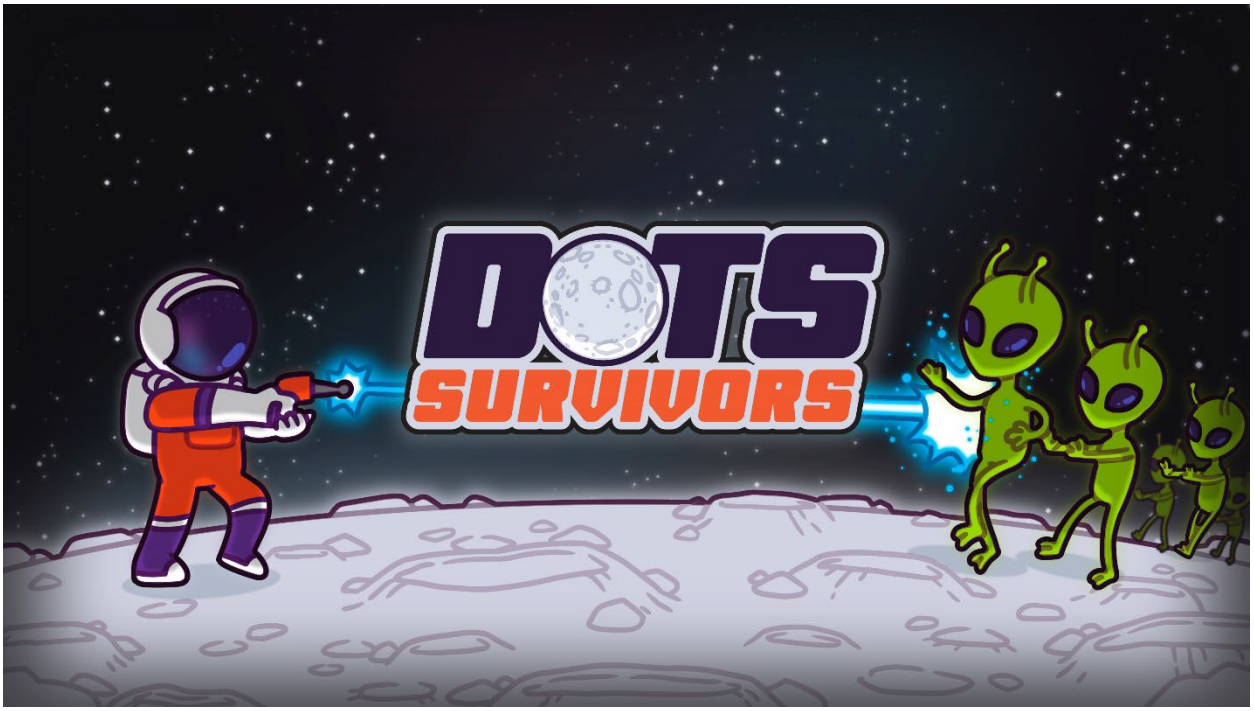


DOTS Survivors Documentation

By: Johnny Thompson – Turbo Makes Games – 2025

Online Documentation: https://survivors.tmg.dev/

Support Discord: https://tmg.dev/support-discord

Support Email: support@tmg.dev

Introduction

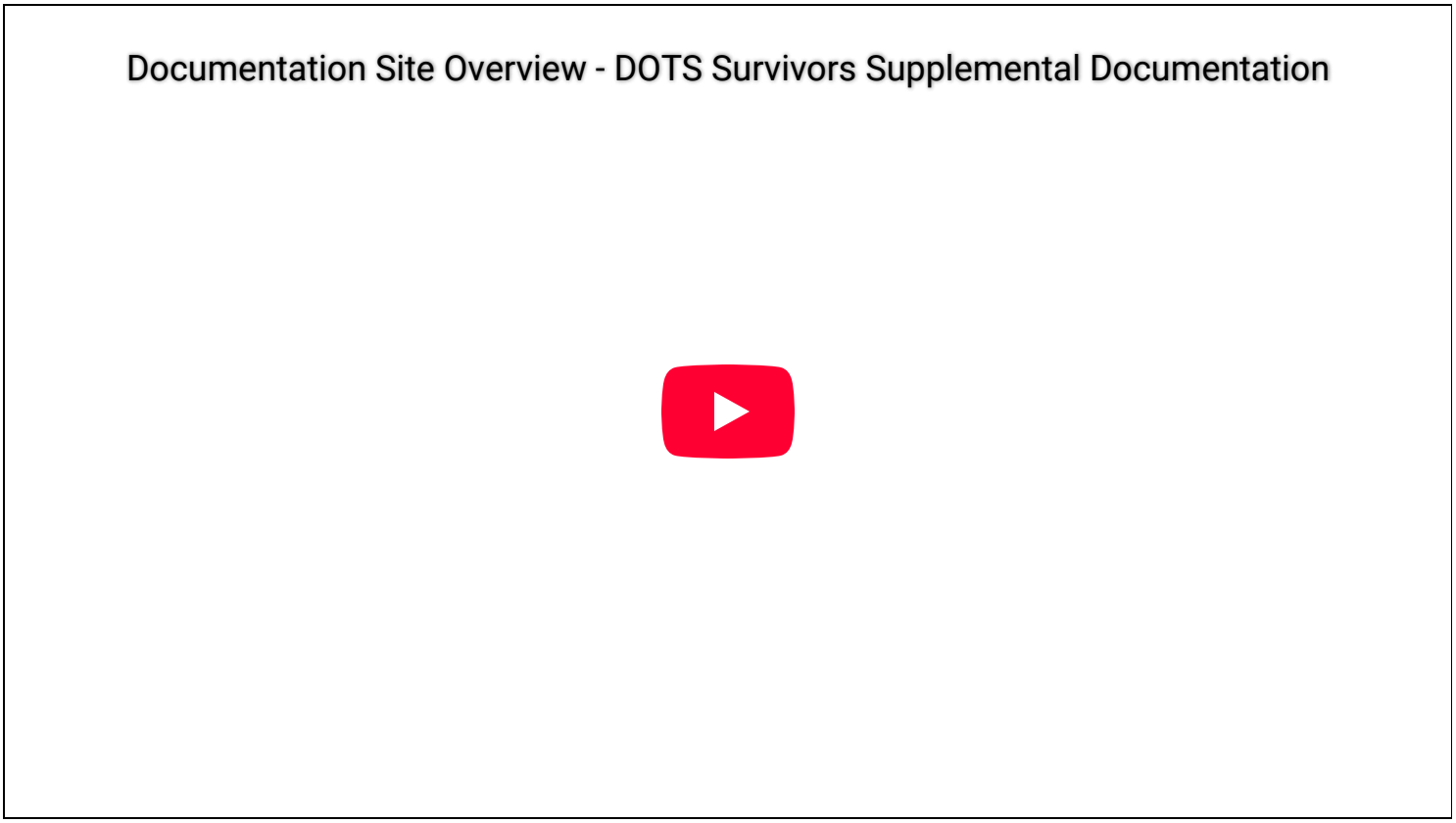
Thank you for purchasing DOTS Survivors, a complete ECS game project and instructional documentation.

DOTS Survivors is a comprehensive example game project that you can use as a starter project or reference for creating games with Unity DOTS and ECS.

This project demonstrates many common features of Unity ECS, while also highlighting drawbacks and workarounds necessary when using Unity ECS in the Unity 6 generation.

Watch the short video below to familiarize yourself with the docs site. After that check out the Setup page for instructions on importing the project into Unity.

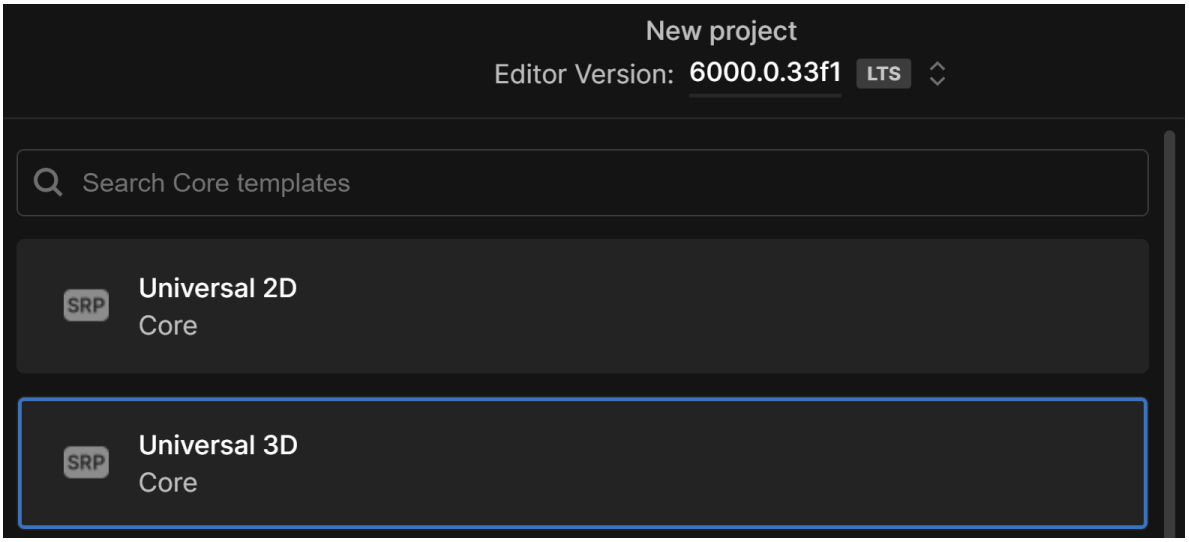
If you have little experience with Unity ECS, check out the New to ECS page for some resources to get you up to speed so you can best learn from these docs.



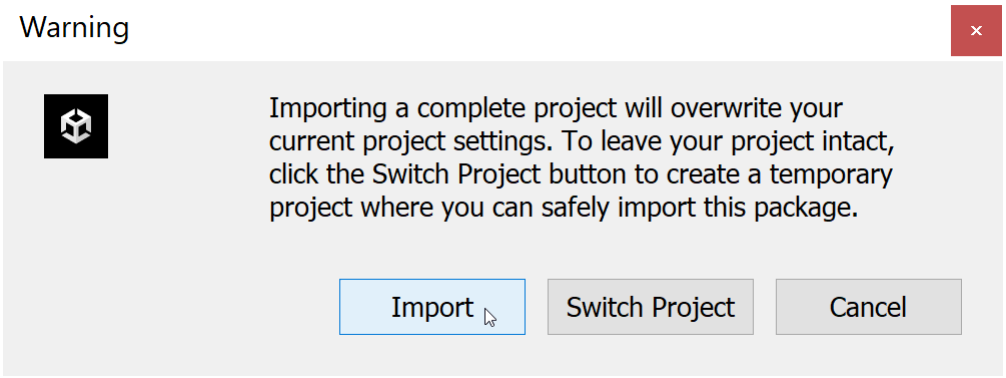
Setup Instructions

As DOTS Survivors is a complete project, it is recommended that you import the project files into a new empty URP project.

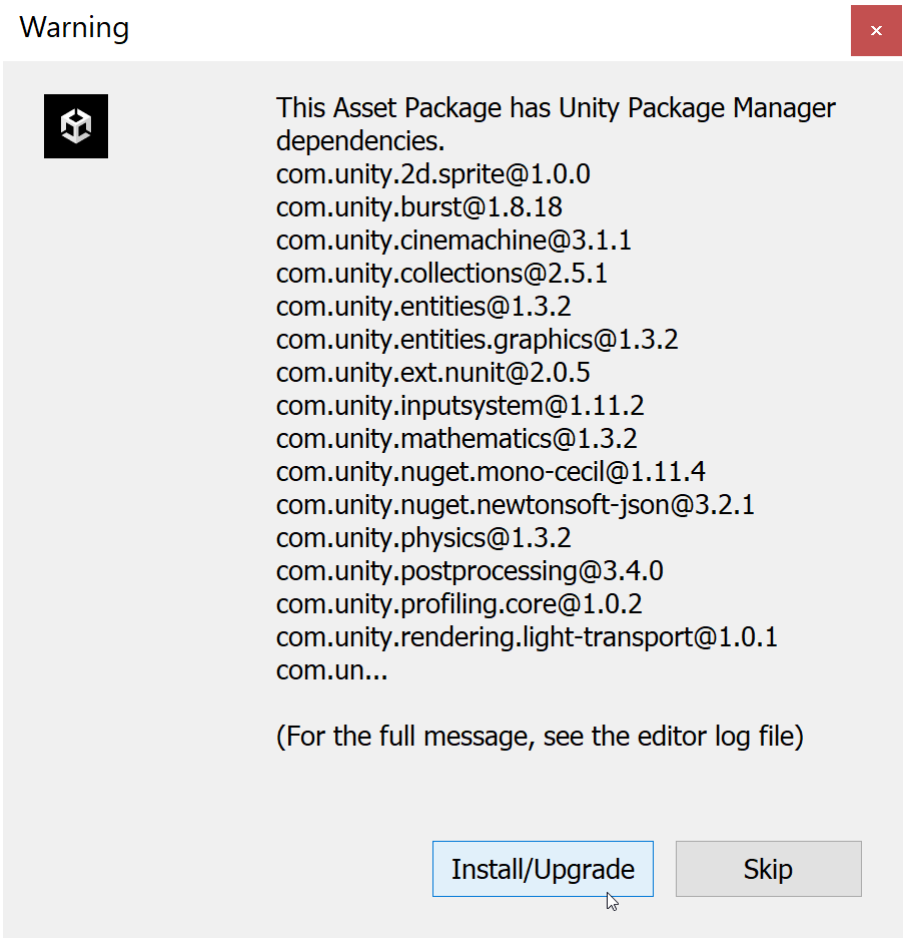
DOTS Survivors was developed using Unity version 6000.0.33f1, however compatibility is expected across all versions of Unity 6000.0.X.



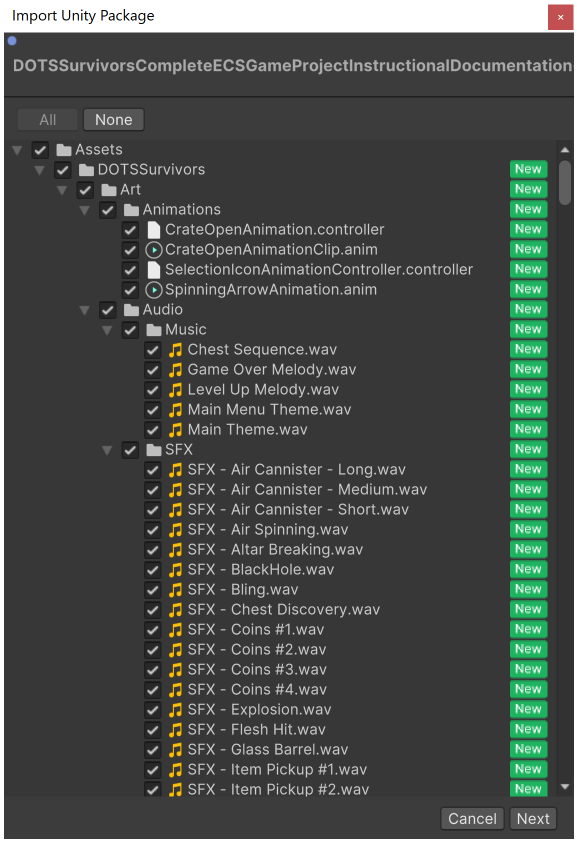
When first importing the package, a message will pop up to inform you that the project settings for the current project will be overwritten. Press "Import" to continue.



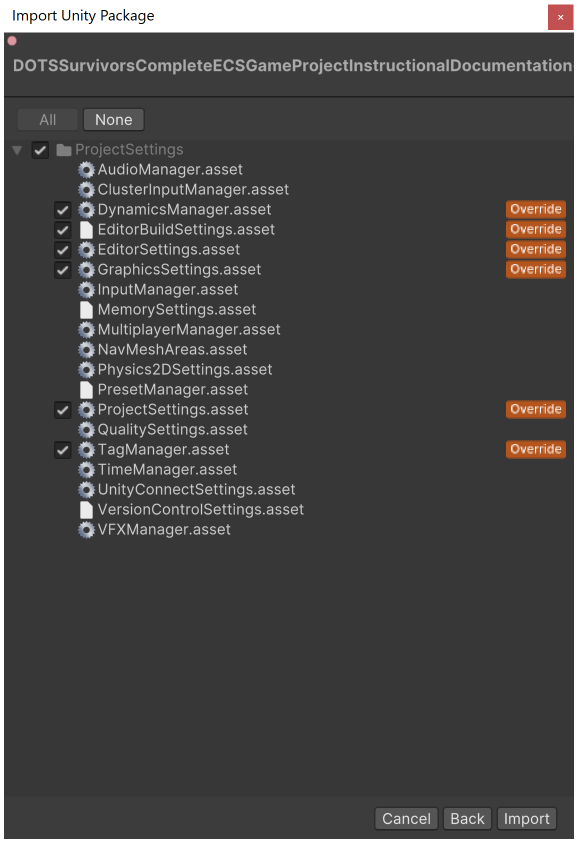
Next, you will be prompted to install dependencies from the Package Manager. Press "Install/Upgrade" to continue.



In the Import Unity Package window, ensure all items are selected, then click "Next" to continue.



Finally, click "Import" on the override project settings window.



The import process may take a few minutes to complete.

Running the Game (Demo Scene)

The main "entry point" of the game is the `TitleScene`, which is stored at `Assets\DOTSSurvivors\Scenes\TitleScene.unity`

Entering the game through the `TitleScene` is what users will experience in the shipped version of the game.

All of the other scenes are stored in the same Scenes folder and can be opened and played directly in the editor without any additional configuration.

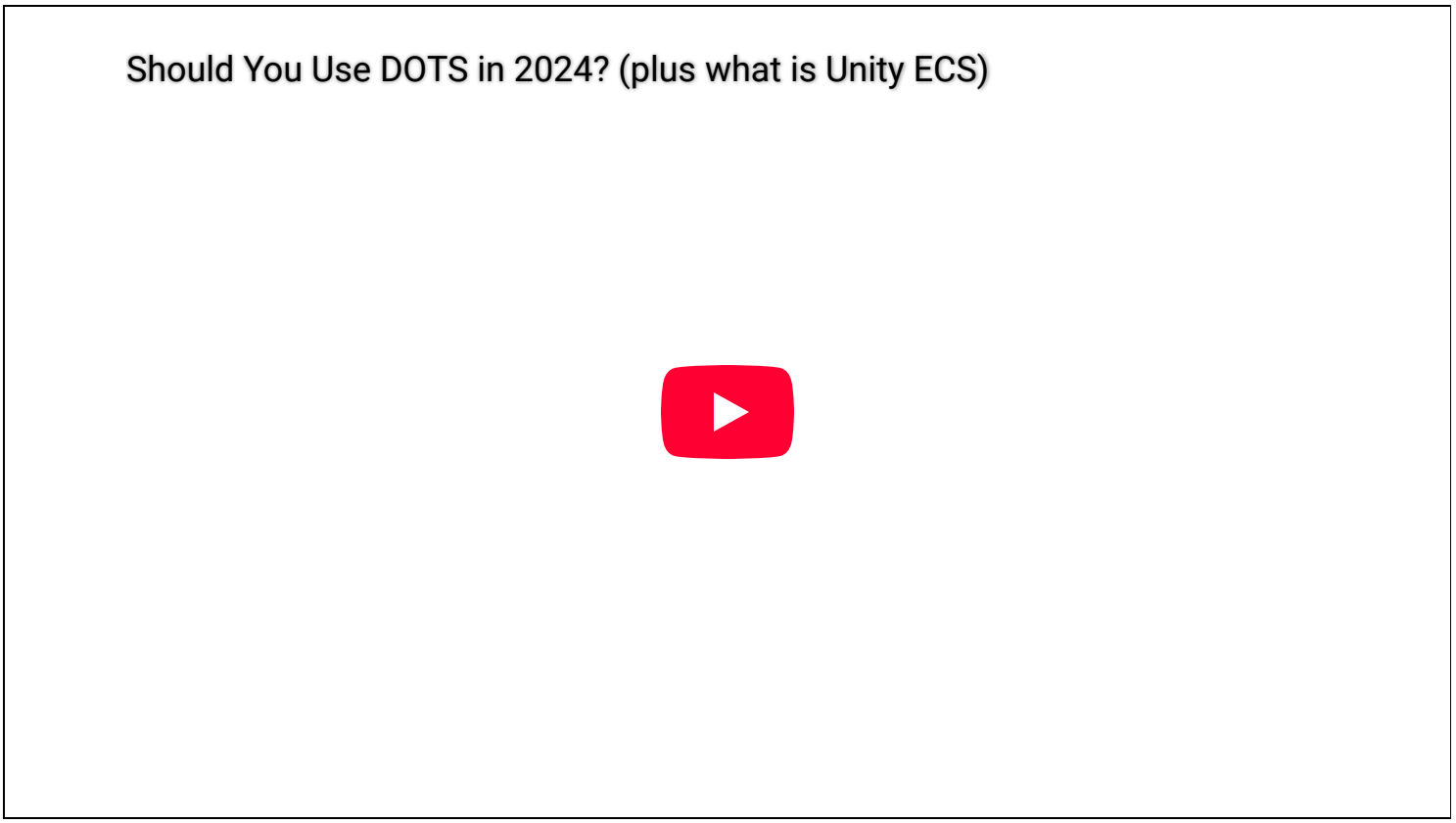
HAVE FUN!

New to DOTS and ECS?

The project files, documentation, and videos of DOTS Survivors are best consumed if you have some understanding behind the theory of Data-Oriented Design and basic structure of a project built with Unity ECS.

If you are new to Unity's Entity Component System, here are a few videos to help get you up to speed.

This video gives a brief overview of all the core pillars of DOTS and provides some additional context of when and when not to use DOTS.



Here is a step-by-step tutorial showcasing how to create a simple survivors-type game in Unity ECS.

Tutorial: SURVIVORS-LIKE w/ Unity DOTS & ECS



Naming Conventions

Types and methods are generally named according to the Microsoft C# standard naming conventions: [Identifier names - rules and conventions - C# | Microsoft Learn]

Here is a list of domain-specific naming patterns used throughout this project:

- Data components with only one field typically have the type name as the descriptive name of what the component is and the underlying value field is simply named "Value"

```
public struct CharacterMoveSpeed : IComponentData
{
        public float Value;
}
```

- Data components with no fields end with the word "Tag"

```
public struct PlayerTag : IComponentData
{

}
```

- Enableable components with no fields end with the word "Flag"

```
public struct InitializePlayerFlag : IComponentData, IEnableableComponent
{

}
```

- Data components with multiple fields that are expected to remain (mostly) unchanged throughout the entity's lifecycle end with the word "Data"

```
public struct CharacterData : IComponentData
{
        public int MaxHitPoints;
        public float MoveSpeed;
}
```

- Data components with multiple fields that are expected to change frequently through the entity's lifecycle end with the word "State"

```
public struct CharacterAttackState : IComponentData
{
        public int AmmoRemaining;
        public float CooldownTimer;
}
```

- All job structs end with the word "Job"

```
public struct CharacterMoveJob : IJobEntity
{
        public void Execute(/* Component data here */) { }
}
```

- All systems end with the word "System"

```
public partial struct CharacterAttackSystem : ISystem
{

}
```

- Authoring scripts end with the word "Authoring" and contain a private class called "Baker"

```
public class PlayerAuthoring : MonoBehaviour
{
        // Public authoring fields go here

        private class Baker : Baker<PlayerAuthoring>
        {
                public override void Bake(PlayerAuthoring authoring)
                {
                        // Baking code goes here
                }
        }
}
```

- ScriptableObjects end with the word "Properties"

```
public class CharacterProperties : ScriptableObject
{

}
```

Prefab Workflow

The prefab workflow is central to this project. Some prefabs are instantiated without modification, while others undergo customization during or shortly after instantiation.

Different Ways Prefabs are Used

This project manages prefabs using two primary methods: direct prefab references in authoring and dynamic loading from the Resources folder. Here are some details on these two implementations:

Direct Prefab References in Authoring

The simplest approach is to store a direct reference to the prefab in a data component that is authored by dragging the GameObject prefab into the script in the editor. During baking, this GameObject is converted to an entity and stored within the data component. When you want to instantiate the prefab, you can just reference it directly via the component.

One place this is done is with the entity game controller where GameObjects are converted to entities in the `GameAuthoring` script's baker and stored in the `GameEntityPrefabs` singleton component.

The drawback to this approach is that this requires you to create a prefab reference for each unique prefab you intend to spawn.

Loading Prefabs from Resources Folder

This approach allows you to be a bit more dynamic with instantiating prefabs as direct references to the prefabs are not required. In this case another key such as an enum can be used to locate and instantiate the prefab.

In this method, GameObjects are loaded from the Resources folder and converted to entities during baking. At runtime, a system collects these prefabs into a dictionary keyed by a unique identifier (such as an enum). This initialization step must occur at runtime as when the GameObject is baked into an entity, it is assigned a certain entity ID, however this is remapped to a different value at runtime. Direct prefab references automatically receive updated entity IDs at runtime. In contrast, if you store these IDs in a dictionary during baking, they remain unchanged and become invalid once remapped at runtime.

When you want to find a prefab, you can use the lookup that you stored the entity prefabs in by passing in the unique key. This lookup is typically stored in a managed data component on a singleton entity for easy access. As this does require a few managed lookups, it is best to minimize the frequency of using this.

Another consideration for this approach is that a unique enum value must be created for each type of prefab you want to spawn. For larger projects where non-technical designers are creating new content

that would require new enum values, looking into a simple source generation solution would be worthwhile.

For a good example of how this is used, this is how enemy prefabs are handled:

- The `GameAuthoring` script loads enemy prefabs from the Resources folder and converts them using a `GetEntity` method.
- The `GameStartSystem` iterates over these prefabs, adding them to a dictionary keyed by the enemy type enum (stored in `ManagedGameData.EnemyEntityPrefabLookup`).
- To optimize performance, the `EnemySpawnSystem` loads only the necessary enemy prefabs into a dynamic buffer at the transition of Enemy Spawn Waves.

> **IMPORTANT**
>
> When adding or changing prefabs stored in the resources folder, changes will not always be immediately reflected in the game. This is due to how Unity triggers the ECS baking process where GameObjects get converted to entities. Unity is unable to detect these changes so baking must be manually retriggered.
>
> The most reliable way I have found to do this for prefabs in the resources folder is to first load into one of the main gameplay stages and open the main entity subscene (check the box next to it in the hierarchy). Next clear the entity cache by going to `Edit > Preferences... > Entities > Clear Entity Cache`. Finally, close the subscene.
>
> Having to frequently do this during development is not ideal so I am investigating alternate solutions.

Video Overview

Prefab Workflow - DOTS Survivors Supplemental Documentation



Authoring Workflow

As this project relies on the prefab workflow, authoring scripts quite prevalent. Rather than using a more "flat" structure where each entity type has a dedicated authoring component for all the components it will ever need, this project uses a "hierarchical" authoring approach. In this method, "higher-level" authoring scripts automatically add "lower-level" authoring scripts that are required for functionality.

Practically, this is achieved by adding a `RequireComponent` attribute to the higher level authoring scripts, passing in the type of the lower level authoring scripts as the arguments.

The primary advantage of this approach is to reduce duplicated code. In these authoring scripts, I often perform simple to moderate manipulation of authoring data during the baking process. Rather than having to duplicate that logic in multiple authoring scripts (which can also be quite error prone), I can take the hierarchical approach and have that logic just live in one place and with the appropriate `RequireComponent` attributes in place, I can be sure that logic is added to all entities of varying archetypes correctly every time. Another advantage is that this avoids situations where multiple authoring components can conflict with each other as they are both trying to add a certain component type to an entity.

The disadvantage of this approach is that now entities have several authoring components (the player entity and enemy entities each have 9 custom authoring components, though most of them have 1 or 0 fields). Another disadvantage is that authoring code must live in multiple class files rather than all being laid out in a single authoring script. While I don't personally do this too much, sometimes there is a bit of script hopping required to follow these hierarchical authoring chains.

ScriptableObjects for Authoring

A common place for data to be authored in DOTS Survivors via the editor is in ScriptableObjects. ScriptableObjects are a Unity asset that can store data and easily be swapped out with other ScriptableObjects of the same type. These ScriptableObjects can be used to store managed and unmanaged data associated with a particular thing. For example, a ScriptableObject for a player's weapon type contains the display name string, associated icon sprite, and all the unmanaged data for leveling up.

ScriptableObjects are managed types that were originally made to work directly with GameObjects. To get these to work with ECS, a `UnityObjectRef<T>` can be used to store a reference to the ScriptableObject for when managed types need to be accessed. Unmanaged data can be extracted from the ScriptableObject and stored in component or blob data during baking or in an initialization system at runtime.

If ScriptableObjects are setting component data during baking it is important to pass the ScriptableObject into a `DependsOn()` method in the baker so that any updates to the data in the

ScriptableObject will retrigger baking to update the entity data with the data changed in the ScriptableObject.

Example:

```
public class WeaponAuthoring : MonoBehaviour
{
    public WeaponUpgradeProperties WeaponUpgradeProperties;

        private class Baker : Baker<WeaponAuthoring>
        {
            public override void Bake(WeaponAuthoring authoring)
            {
                var entity = GetEntity(TransformUsageFlags.Dynamic);

                        // Ensures baking is retriggered when data changes on the Weapon
Upgrade Properties ScriptableObject.
                        DependsOn(authoring.WeaponUpgradeProperties);

                        // Save a reference to the ScriptableObject to access managed data
when needed.
                        AddComponent(entity, new WeaponUpgradePropertiesReference
                        {
                                Value = authoring.WeaponUpgradeProperties
                        });
            }
        }
}

// Data component to reference the original ScriptableObject if needed.
public struct WeaponUpgradePropertiesReference : IComponentData
{
    public UnityObjectRef<WeaponUpgradeProperties> Value;
}
```

Video Overview

Authoring Workflow - DOTS Survivors Supplemental Documentation



Enableable Components

Enableable components are used quite widely in this project. They are often used for scenarios where one system needs to kick off another system - i.e. initialization logic after spawning or visual effects after combat events.

Unity Documentation on Enableable Components - Enableable components introduction | Entities | 1.3.9

Enableable components are a good alternative to using empty data components or data components with a boolean field that needs to be checked every frame. The advantage they have over tag components is that they do not require structural changes to enable or disable, whereas with tag components a structural change is incurred when adding and removing the component. The advantage over boolean fields on a component is that Unity can optimize by determining which entity chunks contain at least one entity with a component enabled or disabled (depending on what the user is querying for). Then it will only "check" the enabled state of entities in certain chunks and typically not for every entity, every frame.

This is commonly used for initialization logic. Because this project uses the Prefab Workflow enableable components can be added to entities in enabled state, which will be baked into the prefab. Then when that prefab is instantiated, a system can run that will set any data components as needed during initialization. Finally the initialization system will set the initialization flag component to disabled, so that system no longer runs on that entity and no additional structural changes are incurred.

They are also used for events like applying knockback or playing damage effects. In these scenarios, additional data is passed into the enableable component so that the system can process the event with the relevant context.

Ways to Enable / Disable Components

Here are a few ways components are enabled and disabled throughout this project

`SystemAPI`

`SystemAPI` is a great way to enable or disable components on a given entity. In the type brackets, pass the enableable type, and pass the entity and true/false into the entity to enable or disable. This method can only be used in `ISystem` and `SystemBase` systems.

```
SystemAPI.SetComponentEnabled<DestroyEntityFlag>(enemyEntity, true);
```

`EntityManager`

For cases when `SystemAPI` cannot be used such as in MonoBehaviours, `EntityManager` can be used to set a component's enabled state.

```
var entityManager = World.DefaultGameObjectInjectionWorld.EntityManager;
entityManager.SetComponentEnabled<InitializeCharacterFlag>(playerEntity, true);
```

EnabledRefRW<T>

`EnabledRefRW<T>` is a handy one as this can be used as a parameter of an idiomatic foreach as another part of the query. This allows you to enable or disable the component directly. Keep in mind that entities will only match the query if the component is enabled. If you do want to query for entities with this disabled, you can set the entity query option for ignoring the component's enabled state.

```
foreach(var destroyEntity in SystemAPI.Query<EnabledRefRW<DestroyEntityFlag>>
().WithOptions(EntityQueryOptions.IgnoreComponentEnabledState))
{
        // Set the enabled state of the DestroyEntityFlag to true
    destroyEntity.ValueRW = true;
}
```

> **NOTE**
>
> Fun bit of trivia for you: most weapon systems use `EnabledRefRW<WeaponActiveFlag>` to deactivate the weapon after its attack. In the case of the `LaserStrikeAttackSystem`, adding this flag to the `SystemAPI.Query` will throw an error due to an excessive number of components. This issue is easily resolved by requiring the component with `.WithAll<WeaponActiveFlag>()`, accessing the entity via `.WithEntityAccess()`, and then using `SystemAPI` to disable the component.

ComponentLookup

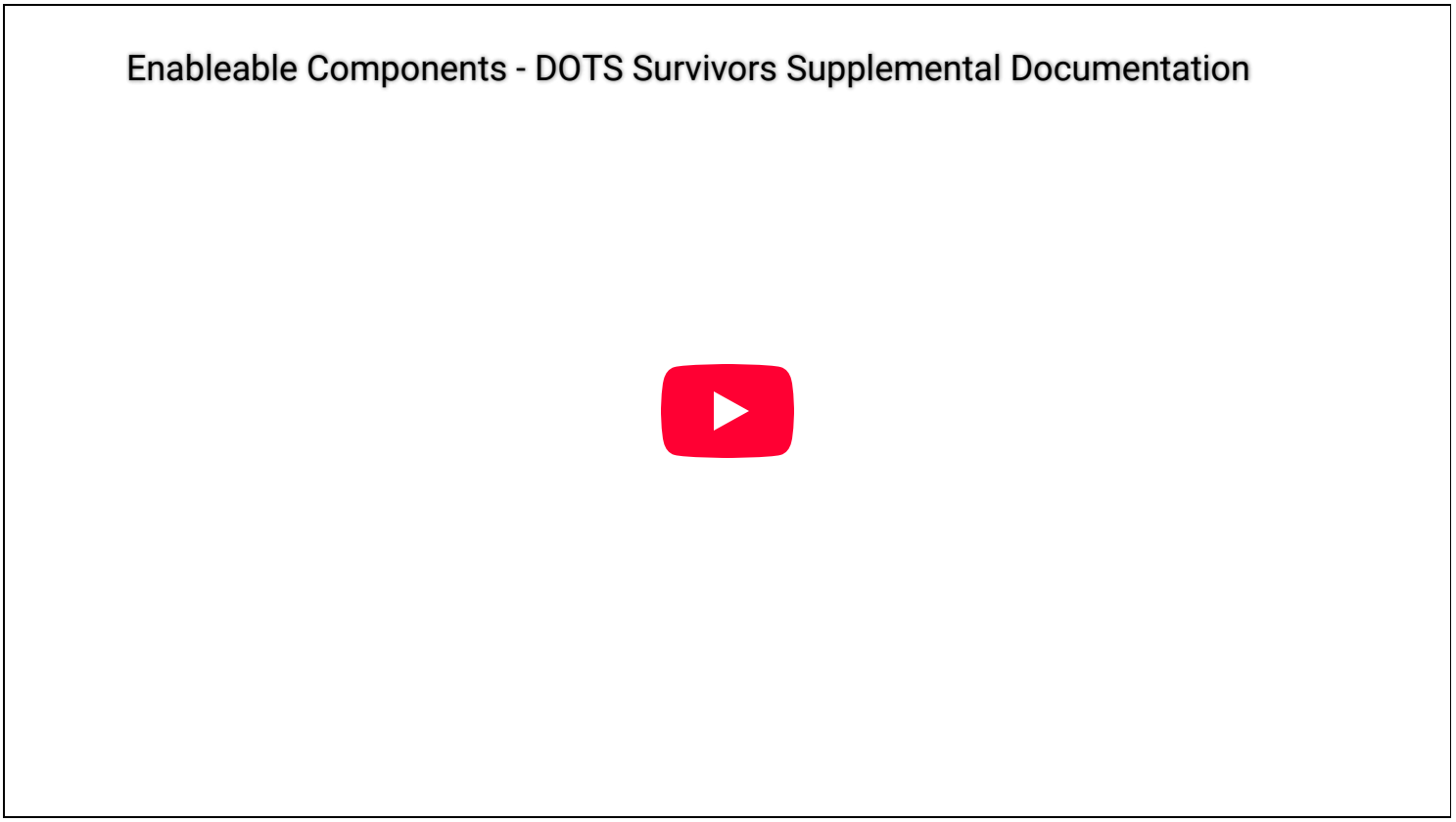
`ComponentLookup` can be used to set the enabled state of an entity, this is especially useful in jobs that do not have access to `SystemAPI` or `EntityManager`.

```
public struct SomeTriggerEventJob : ITriggerEventsJob
{
    public ComponentLookup<DestroyEntityFlag> DestroyEntityLookup;

        public void Execute(TriggerEvent triggerEvent)
        {
                var entityToDestroy = triggerEvent.EntityA;
                DestroyEntityLookup.SetComponentEnabled(entityToDestroy, true);
```

```
        }
}
```

Video Overview



Destruction Logic

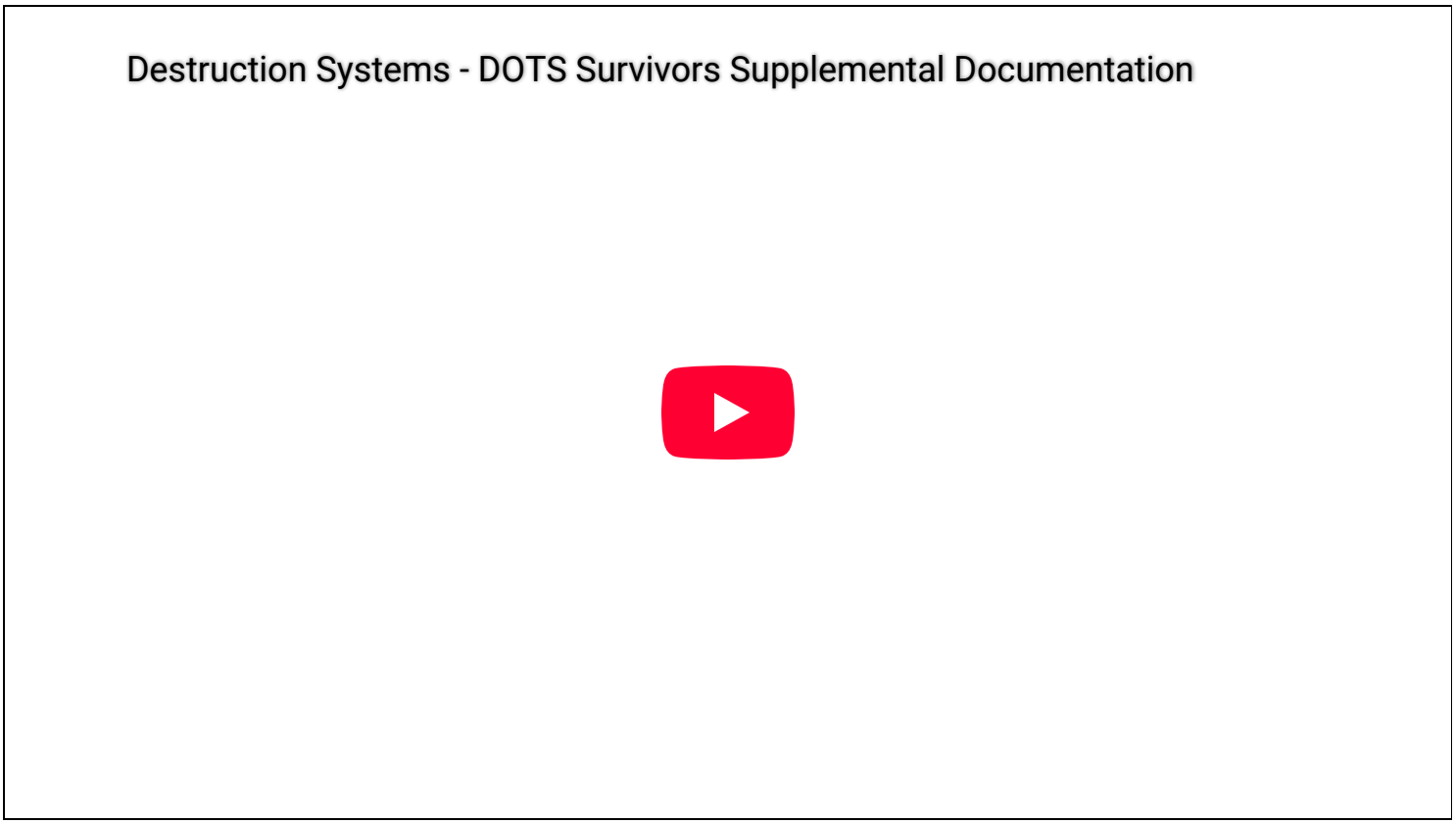
All entities eligible for destruction in the game should have a `DestroyEntityFlag` enableable component. During normal gameplay this component will be disabled, and enabling it will destroy the entity. The `DestructibleEntityAuthoring` script will add and disable the `DestroyEntityFlag` component automatically.

All entity destruction occurs in only two specific systems in the game: `DestroyEntitySystem` and `InstantDestroyEntitySystem`.

All entities that can be destroyed through normal gameplay will have a `DestroyEntityFlag` added to them set as disabled by default. Any system in the game that wants to perform a "standard" destruction of the entity can enable the `DestroyEntityFlag` at any point in the player loop. The `DestroyEntitySystem` will then destroy any entities with the `DestroyEntityFlag` set to enabled at the end of the `SimulationSystemGroup`.

The purpose of a dedicated `DestroyEntitySystem` is so that any additional destruction logic can be executed during entity destruction. Many entities will have certain components that trigger additional logic that should be run when they are destroyed. For example, when an enemy gets destroyed, it will trigger an animation to play and do a dice roll to determine if it should spawn an experience point gem.

The purpose of the secondary `InstantDestroyEntitySystem` is for cases in which the additional destruction logic should not be invoked. One example use case for this is for when an enemy spawned in an earlier spawn wave that is far off screen should be cleaned up. In this case we don't care about the animation playing and we certainly do not want it to drop an experience point gem as the player didn't actually defeat the enemy. For this, we can simply add the `InstantDestroyTag` to the entity and the system will destroy the entity with no fanfare. This system will run just before the standard `DestroyEntitySystem` to ensure the additional destruction logic is not executed in the case that the `InstantDestroyTag` is added the same frame the `DestroyEntityFlag` is set.



Destruction Triggers

Before an enemy is actually destroyed, a certain event or trigger must occur to mark the entity as an entity to be destroyed. Below are the primary destruction triggers in DOTS Survivors, but note that this is not an exclusive list as there is nothing stopping some miscellaneous system not listed here to enable the `DestroyEntityFlag` component or add an `InstantDestroyTag` to an entity.

Destroy After Number Hits

This will trigger destruction of an entity after it has collided with a defined number of entities. This works in conjunction with the Entity Interaction system to determine when a hit should be counted. The `DestroyAfterNumberHitsSystem` will tick down the integer `DestroyAfterNumberHits.HitsRemaining`, once this value reaches zero, it will mark the entity for destruction via enabling the `DestroyEntityFlag`. This system will also clear the remaining elements from the `EntityInteraction` buffer so that damage or other interactions are not applied to any target entities past the point at which the source entity was destroyed.

Destroy After Time

This will trigger destruction of an entity after a certain number of seconds authored in `DestroyAfterTimeAuthoring`. The `DestroyAfterTimeSystem` will tick down the `DestroyAfterTime.Value` by delta time each frame. Once the timer is at or below zero, the `DestroyEntityFlag` will be enabled.

A special consideration to make for this one is for enemies that are set to destroy themselves after time. In this case we don't want the enemy to spawn an experience point gem as the player did not destroy the enemy directly. To do this, the value of `DropExperiencePointOnDestroy.ChanceToDrop` is set to 0, which will not spawn an experience point gem.

Destroy At Position

This will trigger the destruction of an entity when it reaches a certain world position. The way the `DestroyAtPositionSystem` determines if an entity has reached its target position is by comparing the distance between the entity and its target position on the current frame with that of the previous frame. If the value is decreasing, it is still moving towards the target, and if it is increasing, it has passed the target and is moving away from it. If the entity is less than 2 units from the target, and considered moving away from it, at this point it will be considered "at its destination."

The logic behind this method of determining if it is at the destination position is to account for fast moving entities that may not have their position within a small tolerance of the destination position on a given frame.

Destroy Off Camera

This will trigger the destruction of an entity when its world position is out of the camera bounds. This can be configured with some additional padding so that an entity can travel a bit off the edge of the screen before it is destroyed. Further, the bounds of the screen to evaluate (up, down, left, right) can be set so the entity is only destroyed when it goes outside of the camera bounds in only the selected directions.

Related: Camera

Destroy On Environment Contact

This will trigger the destruction of an entity when it contacts an environment element. Environment elements are entities with colliders that live in one place in the world and do not move or have any interaction. To evaluate this contact, a trigger event is evaluated (a trigger event is used as the only entities that utilize this type of destruction are player attacks which are triggers that raise events). If that trigger event is between exactly one environment entity and exactly one entity that has the `DestroyOnEnvironmentContactTag` the trigger event job simply sets the `DestroyEntityFlag` as enabled via a component lookup.

Destroy Previous Enemy Waves

This destruction trigger is used to facilitate enemy variety as it will cycle out enemies spawned on previous waves to favor ones spawning new ones in the current wave. Conceptually, this works by identifying enemies spawned on waves prior to the current one then evaluating those on an individual basis to determine if it is outside of the camera bounds. If these conditions are met, the enemy has an `InstantDestroyTag` added to it. This is so it does not drop an experience point gem and other "on

destroy" functions that may apply in the standard `DestroyEntitySystem` do not execute. This logic does not apply to boss enemies.

This functionality is unique in that it is the only place in this project that takes advantage of shared components. Shared components are similar to data components, however the data values are unique to a given entity archetype. For more in-depth information on shared components see the Unity Documentation on the topic. When each non-boss enemy is spawned, a `WaveIndexOfSpawn` shared component is added with the value set to the current wave index.

The `DestroyPreviousWaveEnemySystem` has a query for all enemies with the `WaveIndexOfSpawn` component. Each frame, this system will iterate from 0 to the current wave index minus 1 and set the shared component filter for the enemy query with the current value in the loop - the query will now only match enemies spawned on a previous spawn wave. If the query is not empty, it will get converted to an entity array, which will then be iterated and evaluated to determine if the enemy is far off screen. If it is, the `InstantDestroyTag` will be added to the enemy to queue it for instant destruction.

With enemies on previous wave destroyed, this creates room for new enemies of the current wave to spawn. See Enemy Spawn Waves for more information on how enemy spawning works.

Destroy Self on Interaction

This will trigger the entity to destroy when an interaction is added to its `EntityInteraction` buffer. This is commonly used for attractable items that will raise an interaction when they come in contact with the player character. This gives the illusion of the player collecting these items.

Instant Destroy on Interact

Entities with this component will add an `InstantDestroyTag` to all target entities in its `EntityInteraction` buffer. This is how the Black Hole attack instantly destroys all enemies on screen without them spawning any experience point gems or doing any other post-destruction logic.

Destruction Triggers - DOTS Survivors Supplemental Documentation



Entity Interactions

Entity interactions are a key concept to DOTS Survivors as they facilitate interactions between multiple entities. Interactions can come in various forms such as an attack projectile that applies damage to an enemy entity, a healing item that grants health to the player, or a projectile that bounces off another entity.

The component at the core of entity interactions is the `EntityInteraction` dynamic buffer. This is added to all entities that can raise interactions. Entities that are the target of an entity interaction do not need (but can have) their own `EntityInteraction` buffer.

```
public struct EntityInteraction : IBufferElementData
{
    public bool IsHandled;
    public Entity TargetEntity;
}
```

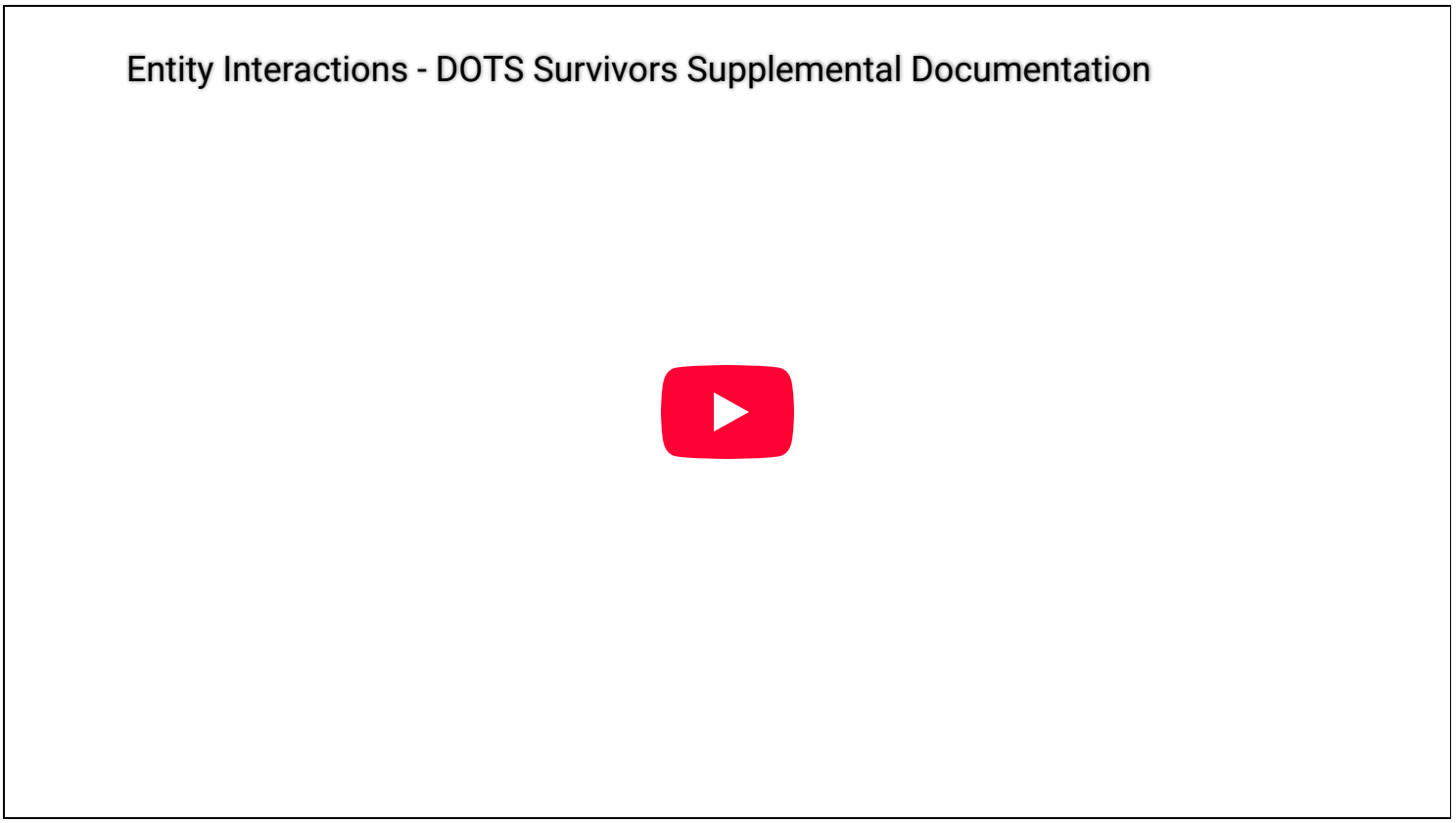
Entity interactions can be raised through collisions, time-based events, or even logical triggers for data-only entities that do not exist physically in the game world. For example, when a damage-dealing entity collides with a damage-receiving entity, a new entry is added to the `EntityInteraction` dynamic buffer. In this case the damage-dealing entity will have the dynamic buffer and the entity receiving damage will be added as the `TargetEntity` for this record. Note that the damage-receiving entity can also have its own `EntityInteraction` buffer, and it may record its own interaction targeting the damage-dealing entity.

When an interaction between two entities has been recorded, various interaction systems can trigger additional behavior based on the component makeup of the entity that holds the interaction buffer. In the example of the damage-dealing entity, a system could execute to deal damage to the entity stored in the `TargetEntity` field. However other interaction systems could trigger behavior that don't use the target entity field - there could be a system to play a sound effect for any new entity interaction, or just outright self-destruct on any interaction raised. The inclusion in these interaction systems and their behaviors are determined by the other data components added to the entity with the `EntityInteraction` buffer.

At the end of the `DS_InteractionSystemGroup`, each interaction's `IsHandled` flag is set to true. This allows subsequent frames to process only new interactions without repeatedly triggering behavior for the same event. Instead of clearing the buffer, all previous interactions remain. This allows systems that raise interactions to scan the buffer and see if the target entity is already in the buffer so it doesn't get added multiple times. This is especially useful for trigger events where one entity may pass through another entity for many frames, but the intended behavior is to only trigger behavior on the first frame the entities overlap.

Depending on the desired behavior, some systems may ignore these checks, as exemplified by `BounceOnEnvironmentJob`, which does not scan for previously added environment entities.

Video Overview



Random Number Generation

Like most video games, DOTS Survivors requires random number generation. Often, entities will need to generate their own unique random numbers, so a `Unity.Mathematics.Random` type can be stored inside a component for an entity to access as needed.

```
using Random = Unity.Mathematics.Random;

public struct EntityRandom : IComponentData
{
        public Random Value;
}
```

This value should be initialized with a seed to begin generating random numbers. To facilitate this, I've created an enum to select the random initialization method.

- None - Seed of 0 (not recommended).
- Manual Override - Uses the value from `EntityRandomAuthoring.ManualOverrideValue` - helpful for consistent debugging.
- System Milliseconds - Uses the system's current millisecond value for (likely) unique randomness each playthrough.

None and Manual Override values set the random seed during baking, while System Milliseconds must be set at runtime in the `InitializeEntityRandomSystem`.

Weighted Random

In DOTS Survivors, there are several instances where one item must be randomly selected from a set. To create more engaging gameplay, weighted randomness is employed so that some items are more likely to be chosen than others.

This is achieved by assigning a weight value from 0 to 100 for each item where 0 means the item will never be chosen and 100 means it has the highest possibility of being chosen. A value of 100 does not mean it will be chosen each time - if there are four items in the set with their weight set to 100, each item has a 25% chance of being selected.

During an initialization system all items in the collection are iterated and each item's weight value is added to a running count. The cumulative value for each item in the collection is stored in another collection along side the item. Consider the following collection of items with random weights:

Item	Weight
Coin	100
Money Bag	30
Blue Gem	100
Green Gem	50

These would be stored in the 2nd collection as:

Item	Weight Key
Coin	100
Money Bag	130
Blue Gem	230
Green Gem	280

With this "weight key" a random number can now be selected from 0 to the max weight key value. This collection can then be iterated in order and the random number will be compared against the weight key. If it is lower than the weight key, that is the item that will be selected.

Random Number	Item Chosen
52	Coin
185	Blue Gem
230	Green Gem
0	Coin

The simplest implementation of this weighted random in the project is the spawn random on destroy, where a random item in a collection is spawned when the entity is destroyed. A more complex implementation is in the selecting of weapons and passive items to display to the player when leveling up.

Video Overview

Random - DOTS Survivors Supplemental Documentation



Blob Assets

Blob assets are a feature that can be used to store data inside a read-only container which can be referenced by many entities. In DOTS Survivors, blob assets are used to store various data types, such as entity prefabs and blob arrays of `WeaponLevelData`.

The creation of blob assets is fairly similar to that described in the Unity documentation on the topic - Create a blob asset | Entities | 1.3.9

In certain cases manual "de-duplication with custom hashes" is implemented. This is used for cases where many entities have the ability to create a unique blob asset, however created blob assets can be shared by multiple entities.

One such case are the Altars that randomly drop an item when destroyed. The data for authoring which items can spawn and the random weights associated with them are done in ScriptableObjects. Altars can have one of these ScriptableObject instances to define which entities that specific altar can drop.

The goal is to implement a solution such that an altar can check to see if a blob asset associated with its ScriptableObject already exists, if it does then a reference is stored to it, if it does not already exist, it will create a new one. This determination is done in the `InitializeSpawnRandomOnDestroySystem` by using a `Unity.Entities.Hash128` type to key into a `BlobAssetStore` to see if it exists. The `Hash128` is created by passing the hash code of the ScriptableObject into a `float4`. When a new blob asset is created, the blob asset will be added to the `BlobAssetStore` with the `Hash128` key.

However, creating the `Hash128` key with just the ScriptableObject's hash code isn't enough. The issue that arises is when a blob asset stores entity prefabs, those prefabs are only valid during the gameplay session in which they were created. Attempting to access a prefab from a previous game session would result in attempting to spawn an invalid entity. As such, a second value is passed into the `float4` when creating the `Hash128`. This is a `gameIndex` integer which increments by 1 each time the game is played. This forces creation of new blob assets each time the game is run.

Video Overview

Blob Assets - DOTS Survivors Supplemental Documentation



System Update Order

System ordering is a very important concept in DOTS Survivors, as many of the systems depend on results of prior systems. As most of these systems run on the main thread (see Job Usage), the order in which they run is important.

Unity ECS has three main phases of the player loop:

- Initialization - Used for setting things up prior to the main simulation
- Simulation - Where systems live by default
- Presentation - Mostly Unity scripts that deal with sending information to the GPU for rendering

Most of the custom systems in DOTS Survivors execute at defined points in simulation system group but there are a few special ones that exist in the initialization system group.

DOTS Survivors Custom System Groups

To further organize my systems, I have created a few custom system groups to ensure systems are updating at the appropriate times in the frame.

`DS_InitializationSystemGroup`

This system group contains systems related to initialization logic that should be executed towards the beginning of the frame. Updates at the end of Unity's `InitializationSystemGroup` to ensure that logic has completed, however it is before the command buffer system at the end of that group in case anything needs to be scheduled in that ECB.

`DS_AttackSystemGroup`

This system group contains systems related to player attacks. Updates in Unity's main `SimulationSystemGroup`. Must update before Unity's `TransformSystemGroup` and `DS_InteractionSystemGroup` as systems in those groups depend on the completion of systems in this group.

`DS_InteractionSystemGroup`

This system group contains systems related to entity interactions. Updates in Unity's main `SimulationSystemGroup`. Must update after Unity's `FixedStepSimulationSystemGroup` and `DS_AttackSystemGroup` as systems in those groups raise entity interactions handled by systems in this group.

`DS_PhysicsSystemGroup`

This system group contains systems that deal with physics such as scheduling collision/trigger event jobs and executing physics overlap queries. Updates in Unity's `PhysicsSystemGroup` (which is part of Unity's

`FixedStepSimulationSystemGroup` that updates on a fixed tick). Will update after Unity's `PhysicsSimulationGroup` and before their `AfterPhysicsSystemGroup` to ensure the appropriate place in the frame for looking at collision/trigger events and overlap queries for the current physics step.

DS_TranslationSystemGroup

This system group contains systems that deal with moving entities via their `LocalTransform` component. Updates in Unity's main `SimulationSystemGroup` and before their `TransformSystemGroup` to ensure that any changes to `LocalTransform` are made before they are transferred to the `LocalToWorld` component which is used to tell the GPU where to render the entity for the current frame.

DS_EffectsSystemGroup

This system group contains systems that deal with triggering visual and audio effects. Updates at the end of Unity's main `SimulationSystemGroup` to ensure any effects that need to be triggered are already known about (typically via enableable component) before executing effects systems. Updates before the `DS_DestructionSystemGroup` to ensure an entity isn't destroyed before it plays necessary effects for the current frame.

DS_DestructionSystemGroup

This system group contains systems related to entity destruction. Updates at the end of Unity's main `SimulationSystemGroup` to ensure all entities marked for destruction in various earlier systems are accounted for and to ensure entities aren't destroyed prematurely. Updates before Unity's `EndSimulationEntityCommandBufferSystem` as systems in this group schedule things in that entity command buffer.

Video Overview

System Update Order - DOTS Survivors Supplemental Documentation



Separate Graphics Entity

In DOTS Survivors, all entities that render graphics have a separate child entity containing a quad mesh and a mesh renderer. Admittedly, the initial purpose of this was to get around a bad choice I made early in development. I discuss this more in the Mistakes section, but the gist is that I chose to make the game play along the X and Z axes with Y serving as depth.

Thus the camera is pointing down in the world and each graphics entity now needs to be rotated 90° along the X axis to face the camera. Having the rotation on the primary entity became cumbersome, as I had to constantly account for the 90° offset when moving or rotating it. To work around this issue, I created a separate graphics entity as a child of each main entity. The child can be rotated 90° to face the camera while the parent entity is moved or rotated freely, with the child automatically following its transform.

Even if I correctly oriented the camera so that graphics entities need to be rotated, I think I would still use a separate graphics entity. Here are some pros and cons with this approach:

Pros:

- Reduced memory footprint on the parent entity. Especially on the characters there are a lot of data components and being able to chip away at that memory usage is a good thing that leads to better chunk utilization.
- Using a separate graphics entity enables independent visual effects and animations, such as rotating or scaling effects during attacks and isolated death animations, without affecting the parent's simulation. For example the plasma blast attack's parent entity will move in a straight line with no changes to scale or rotation, however the child graphics entity rotates and scales up/down for a simple but effective graphical effect.
- For playing death animations I can simply destroy the parent entity and spawn a copy of the graphics entity that plays the death animation. This means that the parent is not affecting the simulation while its death animation plays.

Cons:

- When damaging character entities, I have a system that can flash the color of the sprite black and red to indicate it is taking damage. The built-in `URPMaterialPropertyBaseColor` is used to achieve this effect. The issue is that the damage is being dealt on the parent entity, so now I need to keep an entity reference to the graphics entity to tell it when to flash colors.
- Each entity having a child graphics entity effectively doubles the number of entities that need to have their `LocalToWorld` components calculated each frame. Child entities are also slightly more costly as they need to factor in the parent's transform. While not a significant slowdown, it is extra computing power that wouldn't be required if the graphics entity was on the main entity.

Video Overview

Separate Graphics Entity - DOTS Survivors Supplemental Documentation



Job Usage
(or lack thereof)

When many people think about DOTS and ECS, one of the main things that comes to mind is its highly performant multithreading capability via the job system. The job system is really great for many reasons, however in DOTS Survivors there are not many cases that it makes sense to use it. The job system should be reserved for tasks that can be completed in parallel without forcing the main thread to wait on job completion. It is best suited for work involving large numbers of entities or operations that would be too costly if performed on the main thread alone. These guidelines I tend to follow restrict the usage of jobs in many cases.

For DOTS Survivors, there aren't many extremely costly systems that are constantly running. Much of the application's code follows a procedural flow, where the output of earlier systems directly influences later ones in the same frame. This sequential dependency often negates the benefits of parallel execution. Here, the main thread would need to wait for these to be completed anyway and adding the overhead of job scheduling and safety checking might not even be worth it.

Trigger and collision event jobs are used a few times when relevant, but aside from that there is only one custom job in the game - the `EnemyMoveToPlayerJob`. This is a good candidate for a job as it will run across a large number of entities (majority of the entities will be moving towards the player) and it has a small amount of cost as it needs to normalize the vector from itself to the player.

This is not a recommendation to avoid using the job system entirely - I believe it is an incredibly useful and powerful tool. However, I caution against blindly using it for all cases as that can backfire. Use it only when it makes sense to, and with how I chose to design this project, it did not make sense to use for essentially all of the systems in this game. However I am glad to have an example of where I think the job system is a good fit for this project.

Characters Overview

In DOTS Survivors, there are two types of characters: Players and Enemies. Players are the astronaut entities that are controlled by the player, while enemies are the various aliens that are guided by simple AI logic.

Character Initialization

All characters spawn with an `InitializeCharacterFlag` enableable component that is enabled by default. When active, this flag runs the character through a type-specific initialization system followed by a general character initialization system.

For the player, the `PlayerInitializationSystem` sets up experience point data and triggers events to assign the correct starting weapon and configure all UI elements accurately.

For enemies, an `EnemyInitializationSystem` is executed only on those with an `EnemyHitPointsScaleWithPlayerLevelTag` component. This multiplies the enemy's base hit points by the player's current level. This health scaling is typically used for Boss Enemies.

The `CharacterInitializationSystem` which performs initialization logic on both players and enemies does the following:

- Sets the `PhysicsMass.InverseInertia` value to `float3.zero`, effectively locking rotation along all axes from external forces. This design choice ensures that, following collisions, characters do not rotate - an important consideration for a 2D game where sprites should always face the camera.
- Sets initial stat modification values based on the default stats for a character. See Character Stat Modifiers for more details.
- If the character's max health was changed via a default stat modification, set their current health to match the new max health value.
- Finally, disable the `InitializeCharacterFlag` so that this entity no longer runs in initialization systems.

Video Overview

Characters Overview & Initialization - DOTS Survivors Supplemental Documentation



Character Movement

Character movement is achieved by modifying the `PhysicsVelocity` component instead of directly updating `LocalTransform.Position`. Directly modifying the Position effectively "teleports" the character, bypassing Unity's collision resolution process. Although direct position updates can appear smooth and work for many cases, they have a major drawback: the physics system cannot properly resolve collisions. Constantly overwriting `LocalTransform.Position` prevents the system from resolving overlapping colliders, leading to entities overlapping or warping through walls.

Modifying the `PhysicsVelocity` component instructs the entities to move in a specified direction with a defined force. This method allows Unity's DOTS physics system to accurately resolve collisions with other entities.

A character's velocity is determined by various factors, depending on its type. For instance, a player character gets its move direction from Player Input, while an enemy character obtains its movement direction from Enemy AI Movement systems.

Regardless of whether the character is a player or an enemy, both update a `CharacterMoveDirection` component. The `CharacterMoveSystem` then uses this component to set the character's `PhysicsVelocity`, which Unity's DOTS physics system uses to update the `LocalTransform` while properly factoring in collisions.

It is important to note that this movement is handled in the `FixedStepSimulationGroup`, meaning it updates only on fixed-update frames rather than every rendered frame. If the game runs at a frame rate higher than the fixed time step (typically 60 fps), the characters' motion may appear jittery. To address this issue, Unity provides several components. First, all characters use the `PhysicsGraphicalSmoothing` component with `ApplySmoothing` set to 1, which helps smooth out the jittery motion. However, this approach introduced screen tearing on the player character, so an additional component—the `PhysicsGraphicalInterpolationBuffer`—was added to further address the graphical issues. I still notice slight tearing on enemy characters at times; however, applying the `PhysicsGraphicalInterpolationBuffer` to enemies occasionally causes them to appear at a random position on screen for one frame after spawning, which is undesirable. In the future, I may investigate a method to safely apply this component to enemies, as it appears to reduce graphical artifacts.

Video Overview

Character Movement - DOTS Survivors Supplemental Documentation



Character Movement Graphical Smoothing

Character Movement Graphical Smoothing - DOTS Survivors Supplemental Docu...



Player Input

Player input is obtained using Unity's Input System package. The input system asset, located in `Scripts\Input`, generates a corresponding C# class that allows the input to be read within an ECS system.

The ECS system responsible for handling input is called `GetPlayerInputSystem` and is a `SystemBase` system type. This is because `SystemBase` systems support have managed types as member variables, something that `ISystems` do not allow. Although it is generally discouraged to have member variables on a system, for something like input it is easy to handle this way and doesn't introduce any major issues.

When the application starts, the input actions asset is instantiated and enabled as part of the system's `OnCreate()` method. Every frame, the input system polls the `Vector2` move input as defined in the input asset, which is written to the player's `CharacterMoveDirection`. Additionally, the system tracks aspects of the player's previous input which other systems use to determine the appropriate animations and attack directions.

> **NOTE**
>
> For some other information on input and some of the issues I needed to resolve to get the game working with full controller support and on Steam Deck, see the UI/Controller Navigation section of the documentation.

Video Overview

Player Input - DOTS Survivors Supplemental Documentation



Enemy AI Movement

Most commonly, enemies move in a direct line towards the player. Since this system is expected to run on a large number of entities every frame, and perform slightly expensive calculations that can be easily parallelized, it is one of the few systems in the project that utilizes Unity's job system.

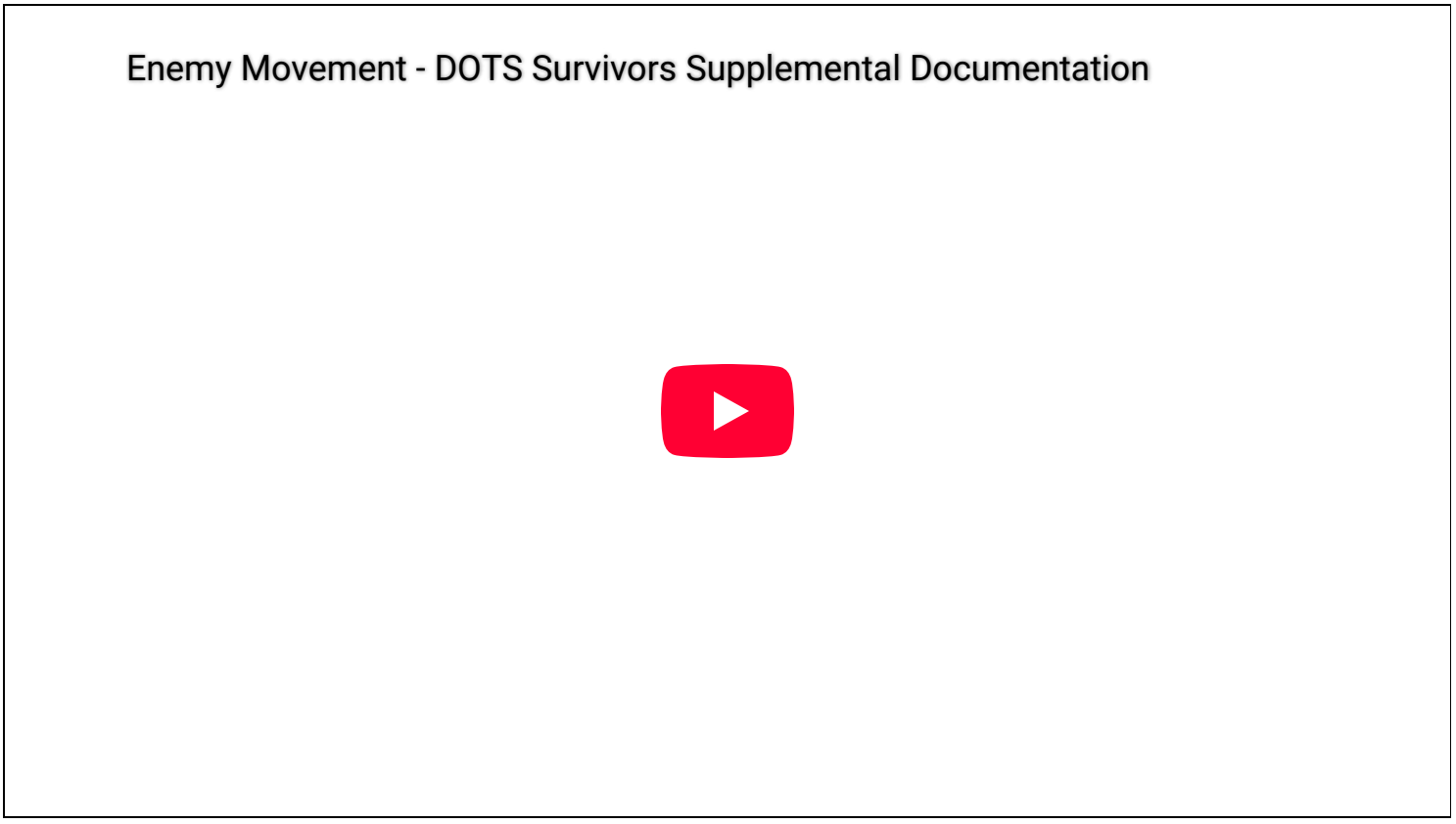
The job itself is extremely simple: it normalizes the vector between the enemy and the player to determine the direction of travel. Although the calculation is simple, the normalize method performs a square root operation, which could slow things down if run on the main thread across many enemies.

There are two other types of enemy movement: one in which enemies move in a fixed direction for their entire lifetime, and another in which enemies move up and down (or side to side) in a sine wave pattern.

Fixed linear movement is very simple, as it only needs to set the `CharacterMoveDirection` one time - when the enemy is instantiated. However, functionality is built in to allow the move direction to be updated by setting a value then enabling an enableable component. Due to the small number of enemies this operates on and the infrequent use, this system runs on the main thread.

Sine wave movement calculates the direction of travel based on a sine wave. The movement remains constant along one axis while varying on the other. Although this system involves some mathematical computation, it only runs on a small number of entities at occasional times in the application. Therefore, it is preferable to run it on the main thread rather than incur the overhead of scheduling a job.

Video Overview



Character Animation





Character animations are achieved with spritesheet and a custom flipbook shader that plays and modifies animations. Players and Enemies use their own unique shader with distinctive features.

Depending on the property to modify, some of these values such as frame count and playback frame rate do not need to change and are set at the material level for a specific character's material.

Other properties such as character facing direction or animation index will change at runtime and are set via material property override components. These are special data components that allow us to directly modify shader properties from ECS in a fully burst compiled context. (See Material Override Properties)

All character animations support the following:

- Playing spritesheet animations with varying sprite counts and frame rates
- Adding a color overlay (used to flash the sprite black/red when taking damage)
- Setting the facing direction of the sprite to match the character's direction of travel
- Applying a time offset to desynchronize animations for characters of the same type
- Dissolve effect to play when the character is destroyed

Only player character animations support:

- Animation index to swap between idle and walking animations

Only enemy character animations support:

- An "enhanced" effect for boss enemies that displays a two-color glowing outline around the enemy sprite





Video Overview

Character Animation - DOTS Survivors Supplemental Documentation

Character Stat Modifiers

Character stat modifiers provide a way for a character's stats to dynamically change during gameplay. Player characters have a few base stat modifications that they spawn with and are unique to each player character. Players can also further modify their stats via Player Passive Abilities.

While enemy characters support stat modification, this feature is not currently implemented. This feature could be used to provide different game modes where enemy characters are enhanced or debuffed to change gameplay.

Stat modifiers are authored in ScriptableObjects located in a dedicated directory within the Resources folder, allowing them to be loaded at runtime. Each stat modifier has a unique enum value associated with it - this makes it easier to author stat modifications applied by passive abilities or other items.

At runtime, the `StatModifierProperties` ScriptableObjects are loaded into a dictionary in `StatModifierController` (singleton MonoBehaviour) which is used as a helper to determine stat modification values and bounds when recalculating stats.

> **NOTE**
>
> It is possible that this all could be stored as blob data which would make it fully burst compatible and minimize the use of managed reference types, but I felt that was a bit cumbersome and didn't provide much benefit in this case as this information is accessed infrequently.

Available Character Stat Modifications

- Additional Attack Projectiles - Increments the number of attacks spawned in an attack group. This stat only impacts attacks that fire attacks in a group of a few at a time before going on a longer cooldown.
- Attack Area - Expands the area of effect for attacks that utilize this stat. Only applies to attacks that deal damage on an area.
- Attack Cooldown - Decreases the cooldown time between attack groups. This stat does not have any effect on the time between shots in a single attack group.
- Attack Duration - Increases the duration that attacks with a "time to live" stat exist in the world.
- Attack Projectile Speed - Increases the speed at which projectile-based attacks move through the game world.
- Damage Dealt - Increases the effective damage output of attacks.
- Damage Received - Decreases the amount of incoming damage from opposing characters.
- Experience Point Gain - Increases the amount of experience points earned from picking up experience point gems.

- Health Regeneration - Increases the amount of hit points per second that are regenerated when the character has less than full health.
- Item Attraction Radius - Increases the radius at which a character can attract attractable items towards themselves.
- Max Health - Increases the maximum health of the character. Increasing this stat typically does not also increase the current health, so that will need to get filled in by health regeneration or health pick ups.
- Move Speed - Increases the character's speed at which they move through the game world.

Modification Calculation Types

- Absolute Integer - Modifications to these types of stats are authored as relative modification values (i.e. +20 or -30 from the base value). However they will display the full stat in the pause menu (i.e. if the base value was 100, then 120 or 70 would be shown based on the previous modifications).
- Increasing Integer - Modifications to these types of stats are authored as relative numbers (i.e. +2 or -3 from the base value). This relative modification is what will be shown for the full stat in the pause menu.
- Absolute Decimal - This stat is authored as a relative floating point number (i.e. +2.5 or -3.3). However the full stat effective stat is shown in the pause menu (i.e. if the base value was 10, then 12.50 or 6.70 would be shown based on the previous modification values).
- Percentage - This stat is authored as a decimal percentage where 0 is 0%, 0.5 is 50%, 1 is 100%, 1.5 is 150%, and so on. In the pause menu, this percentage will be shown.

Each stat modification type has a minimum and maximum value. These bounds are in place to ensure gameplay does not get broken where by some combination of stat modifications, an unexpected scenario is created that makes the player far too weak or too strong... or the game just crashes.

Here is a list of each stat modification type, with their calculation type, and min/max value.

Stat	**Calc Type**	**Min Value**	**Max Value**
Additional Attack Projectiles	Increasing Integer	0	5
Attack Area	Percentage	100%	1000%
Attack Cooldown	Percentage	50%	200%
Attack Duration	Percentage	50%	500%
Attack Projectile Speed	Percentage	50%	500%
Damage Dealt	Percentage	50%	1000%

Stat	Calc Type	Min Value	Max Value
Damage Received	Increasing Integer	0	10
Experience Point Gain	Percentage	50%	500%
Health Regeneration	Absolute Decimal	0.00	2.00
Item Attraction Radius	Percentage	50%	500%
Max Health	Absolute Integer	0	1000
Move Speed	Percentage	50%	500%

Additional Considerations & Requirements for Stat Modifications

- Stat modifications can be permanent or temporary.
- They should stack to support modifications from multiple sources.
- When a temporary stat expires, other modifications must remain intact.

Implementation of Stat Modifiers

Each character entity has a `CharacterStatModificationState` component that contains the current stats for all stat modifications. This is the final "effective" stat that is calculated after combining all stat modifications. Values are stored as `float`s or `int`s depending on the requirements for that stat.

Each character entity has an `ActiveStatModifierEntity` dynamic buffer component. Each time a new stat modification should be added to the player, a data-only entity is created that contains the type and value of the stat modification. This is implemented as an entity, which allows additional components to be attached to temporary stat modifications. These extra components ensure the modifier is destroyed once a timer expires or a specific condition is met. The dynamic buffer has an internal buffer capacity set to 0, meaning all elements live outside the entity chunk. This is a tradeoff because on one hand it reduces the memory footprint of characters by not having to store as much data in the chunk. On the other hand this means that to access stat modification data, this requires a double-random lookup where the program needs to first go outside the chunk to find the entity index, then do another lookup to find data for each entity. I have deemed this as an acceptable tradeoff as accessing stat modification entity data is something that happens infrequently as it is only needed during recalculation.

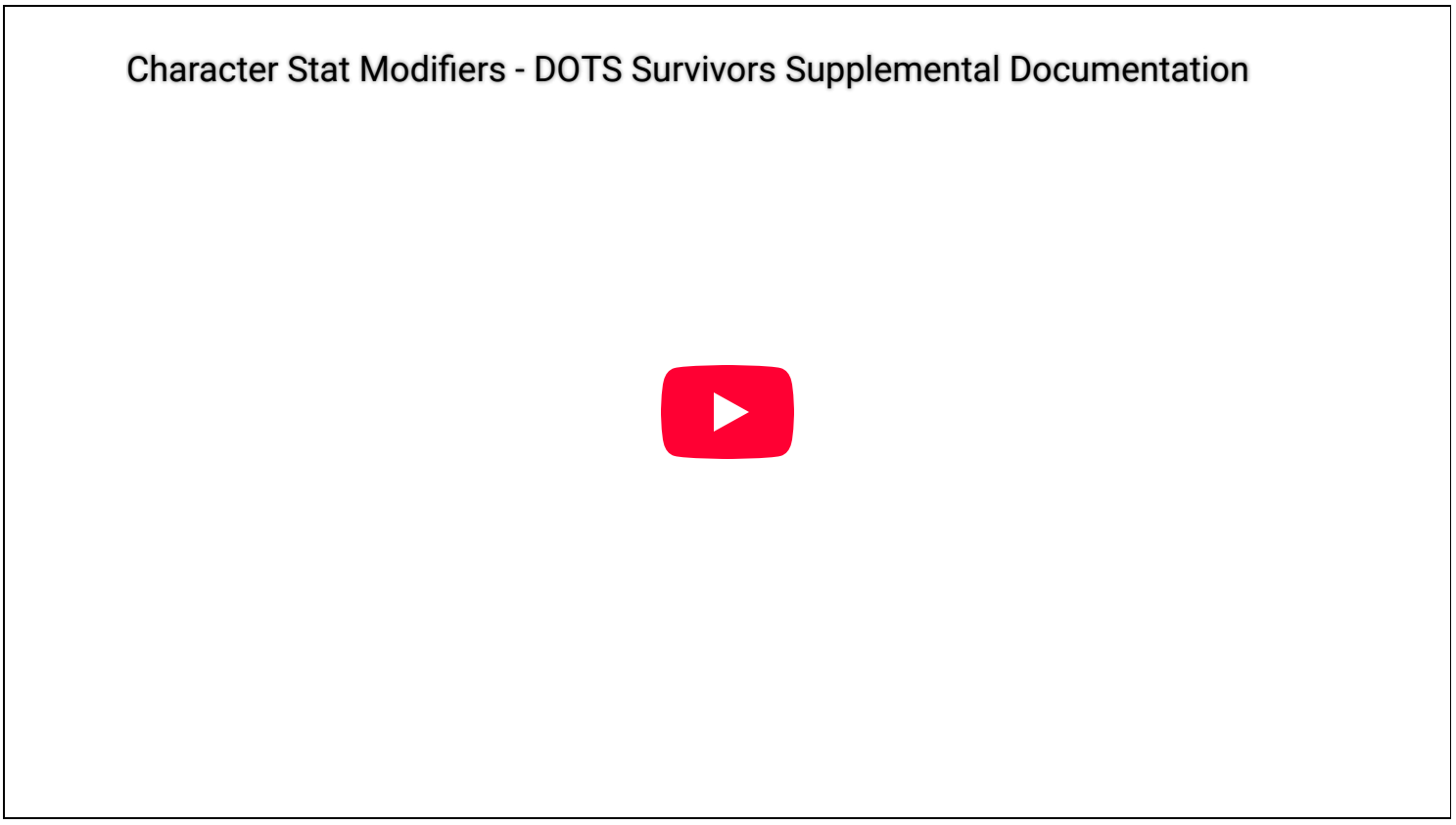
Stat modifications are stored in a `StatModifier` dynamic buffer component on each stat modifier entity. This type consists of a single-byte enum for the stat modification type, and a floating point number for the value. Although some stat modifications are integers, they are still stored as floats at this point and will be cast to an integer when appropriate. This dynamic buffer has an internal buffer capacity size of 12

meaning 12 elements of this type are stored in the chunk alongside each entity. I have chosen 12 as that will be the theoretical-worst case where a single stat modification entity can store one stat modification of every type. Although unlikely and this takes up more memory per entity, I have deemed this as acceptable because these data-only entities typically will not contain any other data (or very little for temporary stat modifiers) so memory footprint is not a concern. Also, if the internal buffer capacity were to be exceeded, that now turns the double lookup into a triple lookup as we would have to do another lookup from the entity to the actual stat modification data, which would be just silly!

Character entities also have a `RecalculateStatsFlag` enableable component. Whenever this component is enabled, the `RecalculateStatsSystem` will iterate entities in the `ActiveStatModifierEntity` buffer for the character, and iterate the `StatModifier` buffer on the stat modifier entity to calculate the effective stats that get assigned to the `CharacterStatModificationState` component on the player. The `RecalculateStatsFlag` for a character gets set any time a new stat modification is added to the character. Also in the `DestroyEntitySystem` any of the stat modifier entities that get destroyed have a reference to the character they are modifying stats for, so they can also set the `RecalculateStatsFlag` here, so when a temporary stat is destroyed, the stats get recalculated.

Any time a stat modification should be considered, the `CharacterStatModificationState` is obtained from the character entity and added, multiplied, subtracted, etc. from the base stat to calculate the current value of that stat.

Video Overview



Character Health Regeneration

Character health regeneration is the ability for a character to regenerate hit points up to their maximum health value.

The data component to facilitate this is `CharacterHealthRegenerationState`, which is an enableable data component with a single float field for the hit point accumulator. This enableable component ensures that the `CharacterHealthRegenerationSystem` runs on an entity only when its `CharacterHealthRegenerationState` is enabled. This component gets enabled in the `ProcessDamageThisFrameSystem` under the following conditions:

- The character has a `CharacterHealthRegenerationState` component
- The character's current health is less than its max health
- The character's `CharacterStateModificationState` has a positive (non-zero) value for the health regeneration stat.

The `CharacterHealthRegenerationSystem` will accumulate hit points over time based on the character's health regeneration stat. The accumulator, stored as a floating-point value in `CharacterHealthRegenerationState`, adds a full hit point to the character's health each time it reaches a whole number.

Hit points are applied to the character by adding a negative entry in the `DamageThisFrame` dynamic buffer. The `ProcessDamageThisFrame` then updates the character's current health accordingly.

Video Overview

Health Regeneration - DOTS Survivors Supplemental Documentation



Player Level Up

Players increase their experience by collecting gems dropped by defeated aliens. When the player reaches a certain experience level, the game pauses and displays a level up screen that prompts the player to acquire a new capability or upgrade an existing one.

When an experience gem is collected, its value is added to the `PlayerExperienceThisFrame` dynamic buffer. As opposed to a data component to accumulate experience points over a frame, a dynamic buffer provides flexibility to add multiple "instances" of experience on parallel threads without overwriting data or experiencing a race condition. Although, experience points are not currently added in any parallel jobs in this project.

The `HandlePlayerExperienceThisFrameSystem` is scheduled to run after the `DS_InteractionSystemGroup`, where experience points are added to the buffer. This system aggregates the experience into the player's total, and if the total meets or exceeds the threshold for the next level, the level up process is triggered.

The level up process starts by updating data components to reflect the player's new level and establishing the new experience target for the next level.

Next, the system attempts to generate a list of up to three random capabilities to either add or upgrade. Here, "capabilities" refer to both Player Weapons and Passive Abilities as they can both be added and upgraded via the level up menu. See Weighted Random for an overview of the selection process. If no upgradeable capabilities are found after a defined fail count, Bonus Items will be displayed as a substitute. This should only occur if all of a player's capabilities are at max level, however it is still possible (though extremely unlikely) that no valid capabilities could be selected with hundreds of consecutive bad dice rolls.

Finally, he system invokes a C# Action event to pass the selected capabilities to the level up UI screen. Upon receiving the event, the UI populates elements that display information about the capabilities and sets up callbacks so that when a player selects a specific capability, it is upgraded accordingly.

The actual upgrade of a capability is managed by the `CapabilityUpgradeController`, a singleton MonoBehaviour. For further details on how capability upgrades are implemented, see Player Attacks and Character Stat Modifiers.

Video Overview

Player Level Up - DOTS Survivors Supplemental Documentation



Enemy Spawn Waves

In DOTS Survivors, enemies are spawned in waves that cycle every minute. The properties for each spawn wave are defined in an `EnemySpawnWaveProperties` ScriptableObject.

During a wave, the number of enemies in the game world is evaluated based on a timer defined by the Spawn Interval in the `EnemySpawnWaveProperties` ScriptableObject. If the enemy count is below the defined minimum enemy count for that wave, the `EnemySpawnSystem` will spawn one of each enemy type listed in the Enemy Types array until the minimum is met or exceeded. If the minimum has already been reached, only one of each enemy type is spawned.

Standard enemies will spawn at a random location offscreen and implement the movement AI to move directly towards the player.

Boss Enemies

Spawn waves can optionally include boss enemies that appear at the beginning of the wave. Bosses have more health than standard enemies, feature a special glow effect, and have may drop a Loot Crate when it is defeated.

In the `EnemySpawnWaveProperties`, if the Boss Type is set to None, no boss will spawn. Otherwise, a single instance of the specified enemy types is spawned offscreen with movement AI directing it toward the player. The boss's base hit points (as defined in the spawn wave properties) are multiplied by the player's level at spawn time. For the Boss Chance to Drop Crate property, a value of 0 means the boss will never drop a crate, while a value of 100 guarantees the boss will always drop a crate.

Enemy Spawn Events

Enemy spawn events are special occurrences that may happen during a spawn wave. During these events, a group of enemies spawn in a specific formation and exhibit a unique movement pattern (see Enemy Movement for more information).

Spawn events are defined in their own `SpawnEventProperties` ScriptableObject, where the spawn formation and other relevant details are set. Currently, only a single enemy type can be spawned in a spawn event.

Here are the spawn event types available to create. These options determine the spawn formation and movement behavior:

- **None** is invalid and no enemies will spawn
- **Linear Move Group** spawns enemies in a cluster at a random location offscreen. Their initial direction will move towards the player at the movement speed defined, however they will keep moving in that same direction for their entire lifecycle

- **Ellipse Around View** will spawn all enemies in an elliptical formation just outside view of the camera. Enemies will move towards the center of the ellipse at the movement speed defined
- **Sine Move Vertical Line** will spawn the enemies in a vertical line off screen either to the left or right (chosen randomly). Enemies will move horizontally across the screen, moving up and down in a sine wave pattern
- **Sine Move Horizontal Line** is the same as the vertical line, except they will spawn horizontally above or below the screen and move down or up respectively while moving side to side in a sine wave pattern

In the `SpawnWaveProperties`, an array of spawn events is defined for the wave. The Delay Time property specifies the interval at which spawn events are evaluated, while the Chance of Occurrence property defines the likelihood of an event - a value of 0 means it never occurs, and 100 means it always occurs.

Spawn events are evaluated for occurrence in order at each Delay Time interval. If an event occurs, enemies are spawned per information stored in the spawn event properties. If it does not occur, that event will be skipped, but subsequent ones are still evaluated individually.

Authoring vs. Runtime Data

Spawn wave properties are authored and stored in ScriptableObjects, which are then added to a stage's properties. This collection of spawn waves is stored in a dynamic buffer with a `UnityObjectRef<SpawnWaveProperties>` field.

The `EnemyWaveSystem` keeps track of the time until the wave should begin to spawn. At the time of spawn wave transition, this system takes managed data from the ScriptableObjects and processes much of that into burst-friendly unmanaged data. This involves populating a dynamic buffer with prefabs for all standard enemies that will spawn. These prefab references are obtained via the `ManagedGameData.EnemyEntityPrefabLookup`. Spawn events also have their data populated into a dynamic buffer. This system is also in charge of spawning boss enemies.

The `EnemySpawnSystem`, which is fully burst compiled and runs more frequently, handles the spawning of both standard enemies and those from spawn events by setting the relevant data for each spawn.

Final Boss Enemy

After all defined spawn waves are complete, a final boss enemy will be spawned to end the game. This final boss has a very high health and damage value making it impossible to destroy and will destroy the player immediately upon contact.

In the shipped game, the Reaper Alien is the boss for all gameplay levels. However this can be changed by setting the `FinalBossType` on the stages `StageProperties` ScriptableObject.

Before spawning the final boss enemy, all existing enemies will be destroyed and the `EnemySpawnerState` component will be removed from the spawner entity to effectively disable further spawning of enemies.



Video Overview

Enemy Spawn Waves - DOTS Survivors Supplemental Documentation



Dealing Damage

In DOTS Survivors, entities can inflict damage on each other by writing damage values to a `DamageThisFrame` dynamic buffer. In the `ProcessDamageThisFrameSystem`, all damage values from this buffer are applied to the target entity's current health stored in the `CurrentHitPoints` component.

The purpose of the `DamageThisFrame` buffer are twofold. Firstly, it centralizes the damage processing to a single system that can account for destroying entities if their health is below zero, triggering SFX/VFX, and managing health regeneration.

Secondly, the buffer permits multiple damage events to be recorded within a single frame, even from parallel threads. Another advantage of using a buffer over a component with a single accumulator field is that each damage element is handled individually allowing us to apply damage reduction from a character's current Stat Modifications. In a more complex game, more information about the damage could be passed on through here such as the damage type - i.e. fire damage deals more damage to grass type ~~Pokémon~~ entities.

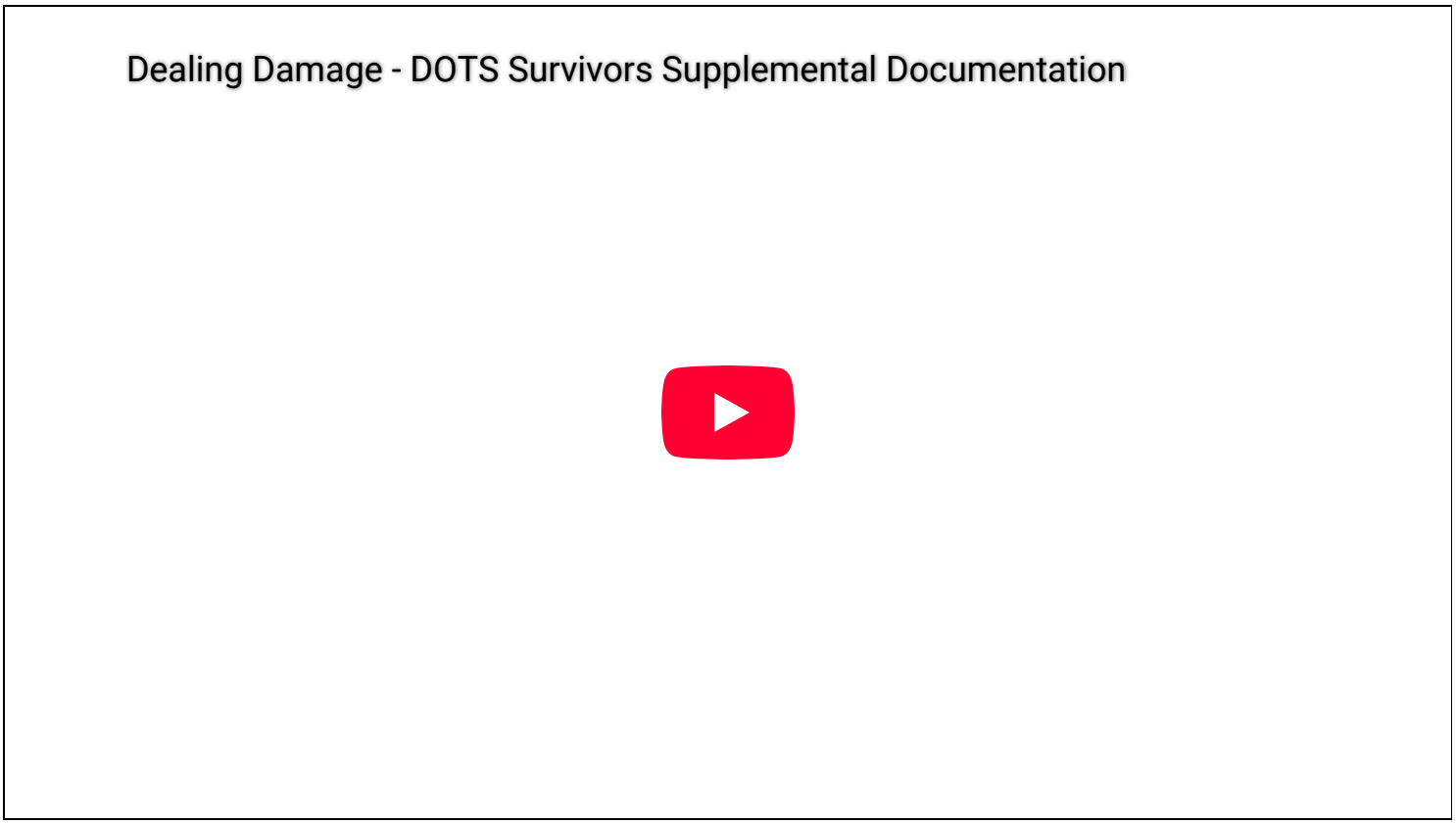
Elements of damage are applied during trigger/collision events (which occur in the fixed update group which is early in the frame) or through Entity Interactions during the `DS_InteractionSystemGroup`. The `ProcessDamageThisFrameSystem` is set to update after the `DS_InteractionSystemGroup` ensuring that all elements of damage for the current frame are accounted for.

For examples of where damage elements are applied to entities, see Player Attacks and Enemy Attacks.

Healing

Since the `DamageThisFrame` buffer only carries the numeric value of damage (without any additional type information) healing is implemented by adding a negative damage value to the buffer. The `ProcessDamageThisFrameSystem` clamps health to the entity's maximum, preventing it from exceeding the upper bound.

Related: Health Regeneration



Knockback

When dealing damage via Entity Interactions, the attacking entity can have a `KnockbackOnInteractionData` component to apply a knockback force. This force affects any target entity with both the `KnockbackTakenMultiplier` and `KnockbackState` components (added via `KnockbackAuthoring`).

When an entity interaction is raised, the `KnockbackOnInteractionSystem` will first confirm the target entity has components necessary to support being knocked back. If so the direction of knockback is calculated by normalizing the vector from the attacker to the target and updates the target's `KnockbackState` with the direction, strength, and end timestamp of the knockback. The component will also be enabled so the proper knockback systems apply.

Any entity with the `KnockbackState` component enabled is excluded from the normal movement in the `CharacterMoveSystem`, allowing the `KnockbackSystem` to temporarily control the character's movement. This system sets the entity's physics velocity to the knockback direction multiplied by the base knockback strength and an additional factor from the `KnockbackTakenMultiplier` (This allows certain enemies to be affected more or less by knockback than others). If the current game time is past the knockback's elapsed time, `KnockbackState` gets disabled and the character will resume normal movement.

Related: Character Movement

Knockback - DOTS Survivors Supplemental Documentation



Player Attacks

In DOTS Survivors, weapons operate automatically on independent timers without player input. Once a weapon's cooldown expires, it executes an attack or spawns a series of attacks called an "attack group." Most spawned attacks collide with enemies to Deal Damage.

Weapon entities originate as prefabs and will be instantiated as a child of the player character when the weapon is unlocked through leveling up or at the start of the game for the starting weapon. Having the weapon as a child of the player makes it easy to spawn attacks from the player's location and follow the player's position if needed.

Weapons data is authored via a `WeaponUpgradeProperties` ScriptableObject which contains high level data about the weapon (display name, icon, etc.) and managed data for the individual upgrade levels.

> **NOTE**
>
> For more information on each of the unique weapons that can be used by the player, see Player Weapons.

Weapon Components

All weapon entities contain the following data components:

`WeaponUpgradePropertiesReference`

This data component simply contains a `UnityObjectRef<WeaponUpgradeProperties>` so the entity can reference managed data from the ScriptableObject.

`WeaponLevelData`

This data component contains all data relevant to the current level this weapon is on. As weapons level up, this component gets updated with new values for the next level. This can be considered a "fat component" as it contains all fields that every weapon entity will need access to. Each weapon does not use all fields, only the ones relevant to itself. This makes authoring and dealing with this commonly used data much easier.

`WeaponUpgradeData`

This data component stores a blob asset reference to a blob array of `WeaponLevelData` components - each element in the array contains data for each upgrade level for the weapon. So when a weapon needs to upgrade, the next element in the blob array is simply copied to the active `WeaponLevelData` component. This blob asset is created at bake time in `WeaponAuthoring`.

`UpgradeWeaponFlag`

Empty enableable component that triggers this entity to run in the `UpgradeWeaponSystem` where its `WeaponLevelData`, and `WeaponState` components will be upgraded with information regarding the new level.

`WeaponState`

Data component containing data relevant to the weapon that is subject to frequently change during gameplay. This keeps track of the cooldown time, time until the next attack in the sequence, number of attacks spawned in the current attack group, and the index of the level the weapon is currently on.

`AttackPrefab`

Stores an entity reference to the prefab of the in-world attacks this weapon will spawn. The GameObject prefab is assigned in the `WeaponUpgradeProperties` and converted to an entity at bake time where it is stored in the `AttackPrefab` data component.

`WeaponActiveFlag`

Empty enableable component to signify the weapon is currently active. The weapon will become active when its cooldown timer expires; the countdown of the timer and activation of this component happens in `WeaponActivationSystem`. When a weapon is active it will begin spawning attacks per its normal attack functionality. Once complete, the flag will become disabled again and the cooldown timer will restart.

Trigger Events

All weapons, except the Black Hole, have physics colliders that interact with enemy entities to deal damage. These attacks are intended to pass through enemies as a single attack instance can often damage multiple enemies. As such these attack colliders are treated as triggers and will not provide any physics forces via `Unity.Physics`. Some attacks can deal a knockback effect, which is handled manually. See Knockback for more information on how this works.

Trigger events are detected in the aptly named `DetectCapabilityTreiggerJob`, which raises an `EntityInteraction` between the attack entity and an enemy. See Entity Interactions for more information. The entity interaction will often apply damage using the `DealHitPointsOnInteraction` data component in the `DealHitPointsOnInteractionSystem`. See Dealing Damage for more information on how damage is applied to the enemy.

When a trigger event is raised, two entities will be part of the `TriggerEvent` type passed into the `Execute()` method of the trigger event. The job confirms that the trigger event is between exactly one player attack and exactly one enemy; it also determines which entity is which as the trigger event just gives a generic "Entity A" and "Entity B" labels. Knowing that information, an entity interaction will be added to the attack's `EntityInteraction` buffer with the enemy as the target entity.

Target Finding

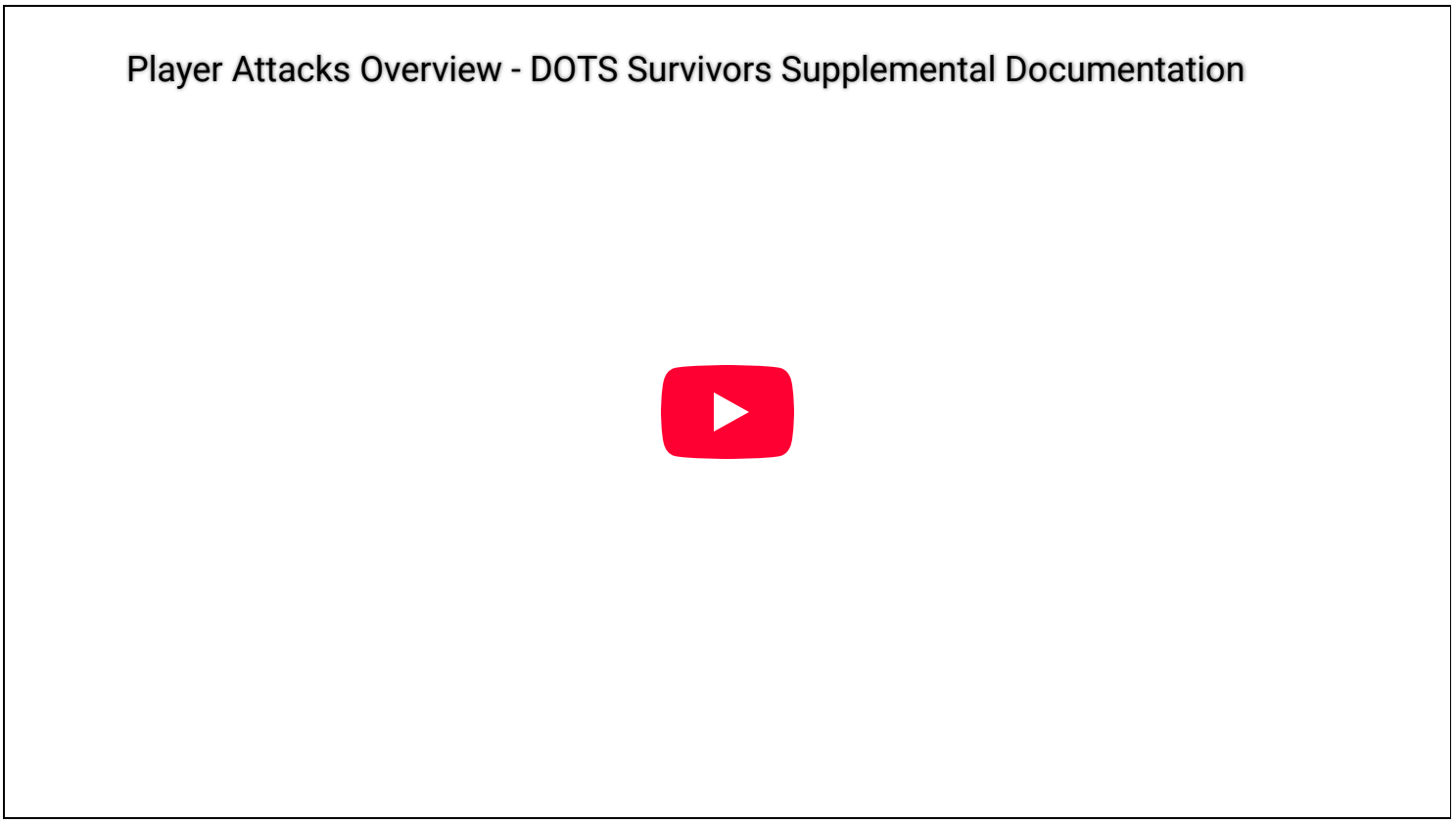
Many weapons will fire attacks towards a target on screen. Some weapons will choose a random enemy while others will select the closest one. The process of determining a target is similar in both cases.

First, all enemies on screen are retrieved using a spatial query, in these cases the `OverlapAABB` method on the `PhysicsWorldSingleton` is used to quickly find all enemies on screen. To achieve this minimum and maximum screen positions are calculated based off camera data stored in the `CameraTarget` unmanaged data component. The collision filter is also used to ensure only enemies are gathered in this query.

For random targets, a random index in the collection is chosen and that will be the target entity. See Entity Random for more information on random number generation.

For closest target, the list is traversed storing references to the current closest entity and its distance. Once all entities have been tested, the closest enemy is known.

Video Overview



Player Weapons

Below is an overview as to how each of the weapons function and some notes on what makes each of them unique from an implementation perspective. Each entity has its own attack system that handles spawning and configuring individual attack instances.

> **NOTE**
> For an overview of how all weapons work, see Player Attacks.

All weapons are impacted by the Attack Cooldown stat modifier.

Black Hole



The Black Hole weapon spawns a massive black hole that consumes everything in sight (except the player!!). This attack is unique in that it has two separate systems that control its behavior. First the `BlackHoleAttackSystem` instantiates a black hole entity with most of its functionality disabled. The `BlackHoleSystem` then takes over using some timers to synchronize the sound effects and visual effects with the actual destruction of entities. This attack is also unique in that it has a random chance stat which is evaluated during the `BlackHoleAttackSystem` - if the random chance is "successful" then it will not destroy experience gems and items on screen, only the enemies. The Reaper enemy is the only enemy in the game that is resistant to black holes as it has the all powerful `EnemyBlackHoleResistTag`.

Black Holes are not impacted by any unique stat modifiers.

Busted Buzz Saw



The Busted Buzz Saw weapon shoots a volley of saw blades all at once in an arc pattern out into the world. The total number of saw blades is calculated by the projectile count from the current level, plus any modifiers the player may have to increase this stat. The direction of travel is chosen by first locating a random enemy on screen and aiming the first shot towards that. Subsequent shots will spawn to the left or right with a slight angle offset. This angle offset is defined in `SawBladeWeaponData` and authored on the `SawBladeWeaponAuthoring` component.

Saw blades move in a straight line, get automatically destroyed after a time period, destroy when they leave the camera bounds, destroy when contacting an environment entity, deal hit points when colliding

with an enemy, and get automatically destroyed after they collided with a specified number of enemies (value set in `WeaponLevelData`).

Saw blades are impacted by the following stat modifications:

- Additional Attack Projectiles
- Attack Area
- Attack Duration
- Damage Dealt
- Attack Projectile Speed

CO2 Vent

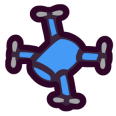


The CO2 Vent weapon spawns a cloud of gas around the player that damages all enemies it comes in contact with. This cloud is spawned as a child of the weapon (which itself is a child of the player) thus it will follow with the player's movements. The cloud will get destroyed after a period of time equal to its cooldown period so a cloud will always be active. A single cloud can only damage a given enemy one time, however it can damage an unlimited number of unique enemies.

CO2 clouds are impacted by the following stat modifications:

- Attack Area
- Damage Dealt

Drone Swarm



The Drone Swarm weapon spawns a group of quadcopter drones that circle the player dealing damage to enemies they contact. All drones are spawned at once and orbit the player as it moves through the game world. The in-world drone entity is offset from the origin of the entity and a rotation is applied via `ConstantRotationData`, the drone graphics child entity has a `CounterRotationData` component to rotate it in the opposite direction so the graphic always appears upright (see Transformations for more detailed explanations).

Drones will destroy after time and can only damage each enemy one time, however they can deal damage to an unlimited number of unique enemies.

Drones are impacted by the following stat modifications:

- Additional Attack Projectiles

- Attack Duration
- Damage Dealt
- Attack Projectile Speed (controls rotation speed)

Laser Strike



The Laser Strike weapon will spawn laser beams down onto the play area striking random enemies on screen. The blast area of the attack deals an area of effect damage and can damage nearby enemies. The weapon system will attempt to damage different enemies on each strike - avoiding the possibility where all laser strikes in an attack group strike one enemy due to bad RNG. The entities already struck by this attack group are stored in an `AlreadyStruckEntity` dynamic buffer. This system also only detects enemies within some inner padding from the screen edges, this is to avoid scenarios where the laser strike occurs on the very edge of the screen and it does not look good.

Laser strikes will destroy after a fraction of a second and deal hit points to all enemies the blast radius comes in contact with.

Laser strikes are impacted by the following stat modifications:

- Attack Area
- Damage Dealt
- Additional Attack Projectiles

O.M.G. Wrench



The "non-copyright infringing" O.M.G. Wrench weapon will spawn large wrenches into the world that move in a boomerang pattern - they will move forward for some amount, then back the way they came for the remainder of their lifecycle. The first wrench in an attack group will fire towards a random enemy and subsequent wrenches will spawn at an angle offset from that which is stored in `WrenchWeaponData` and authored in `WrenchWeaponAuthoring`. `WrenchWeaponData` also stores the angle in radians of the first wrench to spawn so subsequent wrenches know what angle to base of off.

Wrenches will deal hit points on contact with enemy entities, move in a boomerang pattern, and destroy itself when it leaves the camera bounds.

Wrenches are impacted by the following stat modifications:

- Attack Area

- Damage Dealt
- Attack Projectile Speed
- Additional Attack Projectiles

Oxygen Hose



The Oxygen Hose weapon spawns oxygen hoses on either side of the player dealing damage to all enemies they contact. The first oxygen hose in an attack group is spawned in the player's facing direction, the next will fire opposite the player's facing direction, and subsequent attacks will continue to alternate front to back. The player's facing direction is stored in `PreviousPlayerInput` and is used to calculate the orientation of the attack.

Oxygen hoses destroy themselves after a short period of time and damage all enemies they come in contact with.

Oxygen hoses are impacted by the following stat modifications:

- Attack Area
- Damage Dealt
- Additional Attack Projectiles

Plasma Blaster



The Plasma Blaster weapon shoots plasma blasts towards the enemy closest to the player. Plasma blasts will travel in a straight line dealing damage to the first enemy it comes in contact with before destroying itself. Higher levels of the Plasma Blaster weapon allow for plasma blasts to damage up to two unique enemies it contacts. If the plasma blast does not contact any enemies, it will destroy itself after a period of time or when it contacts an environment entity. The weapon waits a short period of time between spawning plasma blasts in a single attack group.

Plasma blasts are impacted by the following stat modifications:

- Attack Duration
- Damage Dealt
- Attack Projectile Speed
- Additional Attack Projectiles

Radioactive Waste Drops



The Radioactive Waste Drop weapon has two phases. In the first phase, a radioactive waste barrel will drop from the top center of the screen towards the enemy closest to the player. Once it reaches its target position (position of the enemy at the time of spawning, not necessarily the current position of the enemy) the barrel will spawn a broken barrel with a radioactive spill zone (giving the illusion of the barrel falling and breaking). The falling barrel does not deal any damage, however the radioactive spill zone will deal damage to all enemies that come in contact with it. The radioactive spill will destroy itself after some time. The radioactive spill is not spawned in any system specific to the Radioactive Waste Drop weapon, rather the barrel is configured to spawn the spill when it is destroyed, thus the actual spawning takes place in the `DestroyEntitySystem`.

After the first barrel, all subsequent barrels will target a random position that is in a defined area nearby the player.

Radioactive waste spills are impacted by the following stat modifications:

- Damage Dealt
- Attack Duration
- Attack Area
- Additional Attack Projectiles

Satellite Toss



The Satellite Toss weapon tosses a satellite in an arc pattern towards the direction the player is facing. Satellites deal damage to unique enemies they contact and will destroy themselves after contacting a certain number of enemies. They will also destroy themselves when they leave the lower screen bounds.

Satellites are impacted by the following stat modifications:

- Attack Area
- Damage Dealt
- Attack Projectile Speed
- Additional Attack Projectiles

Space Screwdriver



The Space Screwdriver weapon shoots screwdrivers in the direction of the player's current input. If the player is not moving, their last positive input value will be used. When playing on keyboard, the screwdriver can only go in 8 directions (up, down, left, right, and all midpoint angles). When playing on controller, the screwdriver can go in all 360 degrees. By default screwdrivers only deal damage to the first enemy they come in contact with, however they can damage more unique enemies they contact by leveling up this weapon.

Screwdrivers travel in a straight line and will be destroyed after some time.

Screwdrivers are impacted by the following stat modifications:

- Attack Duration
- Damage Dealt
- Attack Projectile Speed
- Additional Attack Projectiles

Turbo Jetpack



The Jetpack weapon will spawn jetpacks that travel in a straight line and bounce off edges of the screen and environment elements. Jetpacks are spawned one at a time with a slight delay between each one in an attack group. The first jetpack will fire towards a random enemy and all subsequent jetpacks will spawn at an angle offset from that entity which is stored in `JetpackWeaponData` and authored in `JetpackWeaponAuthoring`. `JetpackWeaponData` also contains the radian angle the first jetpack in the attack group was spawned at so it knows what angle to base subsequent attacks off of.

Jetpacks will destroy after a certain period of time, deal hit points when it contacts an enemy, destroys itself after it contacts a certain number of enemies, moves in a straight line, but bounces off screen bounds and environment elements.

Jetpacks are impacted by the following stat modifications:

- Attack Duration
- Damage Dealt
- Attack Projectile Speed
- Additional Attack Projectiles

Video Overview

Player Weapons Detailed - DOTS Survivors Supplemental Documentation



Player Passive Abilities

Passive abilities are granted to the player upon leveling up. Their effects are defined in `PassiveUpgradeProperties` ScriptableObjects, which are stored in `Resources\ScriptableObjects\UpgradeProperties\PassiveUpgradeProperties`.

Each level of stat modification is applied to the player as a single unit and does not stack with previous levels. Therefore, when designing passive abilities, it is important to specify the total effect value rather than just the incremental increase.

The maximum values listed below only apply to the passive ability itself; the actual values may be vary depending on base stat modifications or other sources. The table displaying the overall minimum and maximum bounds for each stat can be found HERE.

Batteries



Each level increases the duration of weapon effects by 10%, up to a max increase of 50%. Duration is typically the time in seconds an attack entity instance will exist in the game world.

Boots



Each level increases player's move speed by 10%, up to a max increase of 50%.

Glove



Each level increases inflicted damage by 10%, up to a max increase of 50%. When an attack is spawned, its base attack is multiplied by the character's damage dealt stat modifier and then rounded up to the next integer.

Magnet



Each level increases the player's attraction area by an amount ranging from 20-50%, up to a max of 3.98x the base radius. The attraction area is the radius within which the player can pull in items and experience

point gems towards them.

Overclocked Processor



Each level decreases weapon cooldown by 8% for a max decrease of 40%. Weapon cooldown is the time between groups of attacks instantiated by a weapon. This does not have any effect on the time between attacks in an attack group.

Oxygen Flow Valve



Each level increases the character's health regeneration stat by an additional 0.2 hit points per second, up to a max value of 1 hit point per second recovered. When the player's health is below their max health, any positive value in this stat gradually restores hit points until full health is reached.

Oxygen Tank



Each level increases the character's max hit points value by 20 points up to a max increase of 100 hit points.

Payload Expansion Module



Each level increases the area of effect for attacks by 10%, up to a max increase of 50%. When an area of effect attack is instantiated, its transform scale is multiplied by this state to enlarge its effective radius.

Rapid Fire Accelerator Unit



Each level increases the number of projectiles spawned by weapons, up to a max increase of 2 additional projectiles. Increases here will spawn additional attacks in an attack group.

Rocket Assist Module



Each level increases the speed of projectiles by 10%, up to a max increase of 50%.

Space Helmet



Each level increases the experience points received from an experience point gem by 8%, up to a max increase of 40%. Since experience point values are stored as floating point numbers, no rounding is necessary.

Space Suit



Each level reduces the damage taken from enemies by 1 hit point, up to a max reduction of 5 hit points. Note that the minimum damage a player can receive is 0; therefore, if an enemy's attack is less than the damage reduction stat, the player takes no damage and no health is regenerated.

Video Overview

Player Passive Abilities - DOTS Survivors Supplemental Documentation

Enemy Attacks

Enemies execute a melee attack, dealing damage to the player upon contact. To prevent continuous damage during sustained contact, each enemy has a brief cooldown period during which they are unable to attack.

> **NOTE**
>
> Code also exists to support explosive enemies that will explode and deal damage to player characters in the area around the explosion, however no enemies in the game currently use this feature. This works by destroying the enemy after a defined period of time, and its `SpawnOnDestroy` component will spawn an explosion entity with a trigger collider in the world at that location. If that explosion volume contacts the player, it will deal damage in a similar way as standard enemy attacks detailed below.

Enemy entities use standard Unity collider primitives with the 'Provides Contacts' option enabled, ensuring that collision events are raised when they intersect with other colliders. The player, while also using standard colliders, does not have 'Provides Contacts' enabled since it does not need to raise collision events.

When a collision event is raised, the `EnemyAttackCollisionJob` will execute passing along the involved entities as "Entity A" and "Entity B" in the `CollisionEvent` struct. The job then determines which entity is the enemy and which is the player. If the collision event involves a combination of entities other than one enemy and one player, the job exits early.

Once this information is known, the enemy entity can be checked to determine if it is currently on cooldown. If the enemy is off cooldown, it will directly add points of damage to the player's `DamageThisFrame` dynamic buffer (see Dealing Damage for more information on how damage is applied). This also factors in any damage buffs from the enemy's `CharacterStatModificationState.DamageDealt`.

Finally, the enemy's attack cooldown is reset. Instead of updating a countdown timer each frame, the enemy's `EnemyAttackCooldownExpiration` is set to the current game time plus the value from `EnemyAttackData.CooldownTime`. The system then compares this expiration time with the current game time to determine when the enemy can attack again. See Timers for more strategies on implementing timers in ECS.

Video Overview

Enemy Attacks - DOTS Survivors Supplemental Documentation



Audio

DOTS Survivors supports two independent audio types, music and sound effects, each with adjustable volume settings. This setup uses a simple audio mixer that can be found in the `Assets\Settings` directory. Each audio category is assigned an audio mixer group to control the volume. While the music group uses default settings, the sound effects group uses a "compressor" effect to normalize the volume when many sound effects are played simultaneously. The base audio mixer has exposed properties for the volume level of each mixer group.

Each scene has an audio controller MonoBehaviour with a direct reference to the `AudioMixer`, which can be used to set the volume levels for the individual mixer groups. This allows players to set music and SFX volume levels independently.

```
public class GameAudioController : MonoBehaviour
{
    [SerializeField] private AudioMixer _audioMixer;
        private const string MUSIC_VOLUME_PARAMETER = "MusicVolume";

    public void SetMusicVolume(float newVolume)
    {
            _audioMixer.SetFloat(MUSIC_VOLUME_PARAMETER, newVolume);
    }
}
```

Sound Effects

As Unity ECS does not have a native audio solution, audio clips still need to play on audio sources that live on GameObjects. At a high level, this is achieved by pooling many GameObjects with audio sources that will play audio clips stored in `UnityObjectRef<AudioClip>` on a data component. Pooling is used to efficiently manage audio playback by reusing GameObjects rather than constantly creating and destroying them.

There are three types of audio sources that get pooled in the `GameAudioController`:

- Regular SFX Audio Source - used for one-shot audio clips that should play once
- Looping SFX Audio Source - used for effects that will play looping until the source entity is destroyed
- Pause Resistant Audio Source - plays one-shot audio clips that are not affected by game pause

The pooling consists of spawning many new GameObjects with an audio source component with values set relevant to the audio source type. These GameObjects are then stored in an array. To play an audio clip, the `GameAudioController` has public methods for each of the audio source types that take an audio clip as a parameter. When called, the next available audio source in that pool will play the audio clip.

Keeping track of the next available audio source is done with an incrementing index that will loop to 0 once it exceeds the pool length. The other parameter of these methods is a priority value that is used by Unity to determine which audio clips have a higher priority to play.

From the ECS side, audio clips are stored in `UnityObjectRef<AudioClip>` in data components and are authored via authoring scripts. Here is an example of a data component that stores an audio clip to play when the entity spawns:

```
public struct PlayAudioClipOnSpawnData : IComponentData, IEnableableComponent
{
    public byte Priority;
    public UnityObjectRef<AudioClip> AudioClip;
}
```

Then here is what would be called to play the audio clip when the entity is spawned:

```
foreach (var (audioClip, audioClipEnabled) in SystemAPI
    .Query<PlayAudioClipOnSpawnData, EnabledRefRW<PlayAudioClipOnSpawnData>>()
    .WithNone<ShouldLoopAudioClipTag>())
{
    GameAudioController.Instance.PlaySfxAudioClip(audioClip.AudioClip.Value,
audioClip.Priority);
    audioClipEnabled.ValueRW = false;
}
```

Note that `PlayAudioClipOnSpawnData` is an enableable component, so it can be disabled once the audio clip has begun playing.

Looping Sound Effects

Looping sound effects require some additional considerations as their audio isn't "one and done" and they should play through the duration of the entity's lifecycle. To accomplish this I have the return value of `GameAudioController.PlayLoopingAudioClip` be the index of the audio source in the looping audio clip pool. That index value is then stored in a cleanup component - this type of data component will exist after the entity is destroyed and can be used to perform additional cleanup logic, in this case it will stop the audio clip. Here is what that component looks like:

```
public struct LoopingAudioSourceIndex : ICleanupComponentData
{
    public int AudioClipIndex;
}
```

For more information on Cleanup Components, see the Unity documentation: [Cleanup components introduction | Entities | 1.3.9]

Any entity that should play a looping audio clip will have the `ShouldLoopAudioClipTag` component, which will match the query of a foreach like this:

```
foreach (var (audioClip, audioClipEnabled, entity) in SystemAPI
    .Query<PlayAudioClipOnSpawnData, EnabledRefRW<PlayAudioClipOnSpawnData>>()
    .WithAll<ShouldLoopAudioClipTag>().WithNone<LoopingAudioSourceIndex>()
    .WithEntityAccess())
{
    var audioClipIndex =
GameAudioController.Instance.PlayLoopingAudioClip(audioClip.AudioClip.Value,
audioClip.Priority);
    audioClipEnabled.ValueRW = false;
    ecb.AddComponent(entity, new LoopingAudioSourceIndex { AudioClipIndex =
audioClipIndex });
}
```

Then when the entity is destroyed, the `ShouldLoopAudioClipTag` will no longer exist, however the `LoopingAudioSourceIndex` component will still exist as it is a special cleanup component. Thus the entity will match this foreach:

```
foreach (var (loopAudioClip, entity) in SystemAPI
    .Query<LoopingAudioSourceIndex>()
    .WithNone<ShouldLoopAudioClipTag>()
    .WithEntityAccess())
{
        GameAudioController.Instance.StopLoopingAudioClip(loopAudioClip.AudioClipIndex);
    ecb.RemoveComponent<LoopingAudioSourceIndex>(entity);
}
```

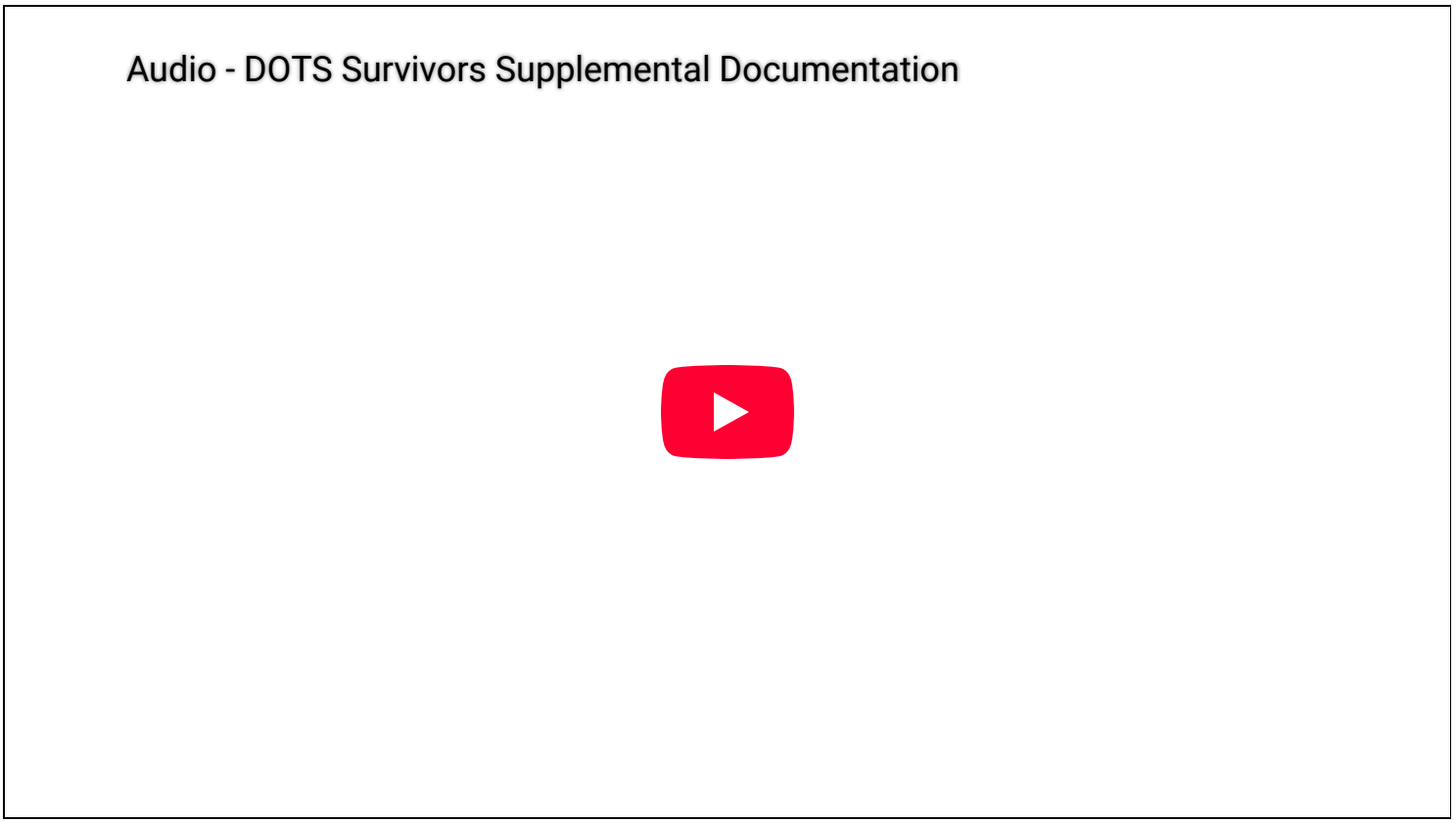
Pausing Game Audio

When the game pauses, sound effects should also pause and then resume when the game resumes. This is accomplished by iterating all of the audio sources and checking if they are currently playing audio. If they are, pause the audio source and add its index to a list of paused indices. When the game resumes, use the list of paused indices to resume only those audio sources.

The pause resistant sound effects work similarly to the regular, non-looping sound effects, however this pool is not iterated during pause/resume. These types of sound effects are used in rare instances where a sound effect should be played when the game pauses, such as when picking up a loot crate or triggering the game over screen.

Related: Gameplay Pausing

Video Overview



Basic Camera Setup

Each gameplay scene has a single camera tagged (via regular Unity GameObject Tag) as the `MainCamera` with the following components:

- Camera projection set to Orthographic with a size of 10
- The sole audio listener for the scene
- Cinemachine Brain component with default settings

The second required GameObject is called `CameraTargetObject` that just has a `CameraTargetObject` singleton MonoBehaviour component on it. This is the GameObject target the Cinemachine camera will be following.

The third required GameObject is called `CinemachineCamera` with the following components:

- `CinemachineCamera` component with the `CameraTargetObject` assigned to the Tracking Target. Position Control is set to Follow and Rotation Control is set to None.
- `CinemachineFollow` component set to a Blending Mode of World Space with Position Damping of (1, 1, 1) and Follow Offset of (0, 0, 0).

On the entity side, the player has a `CameraTargetAuthoring` script with a Y Offset value of 10.

Camera Follow Player

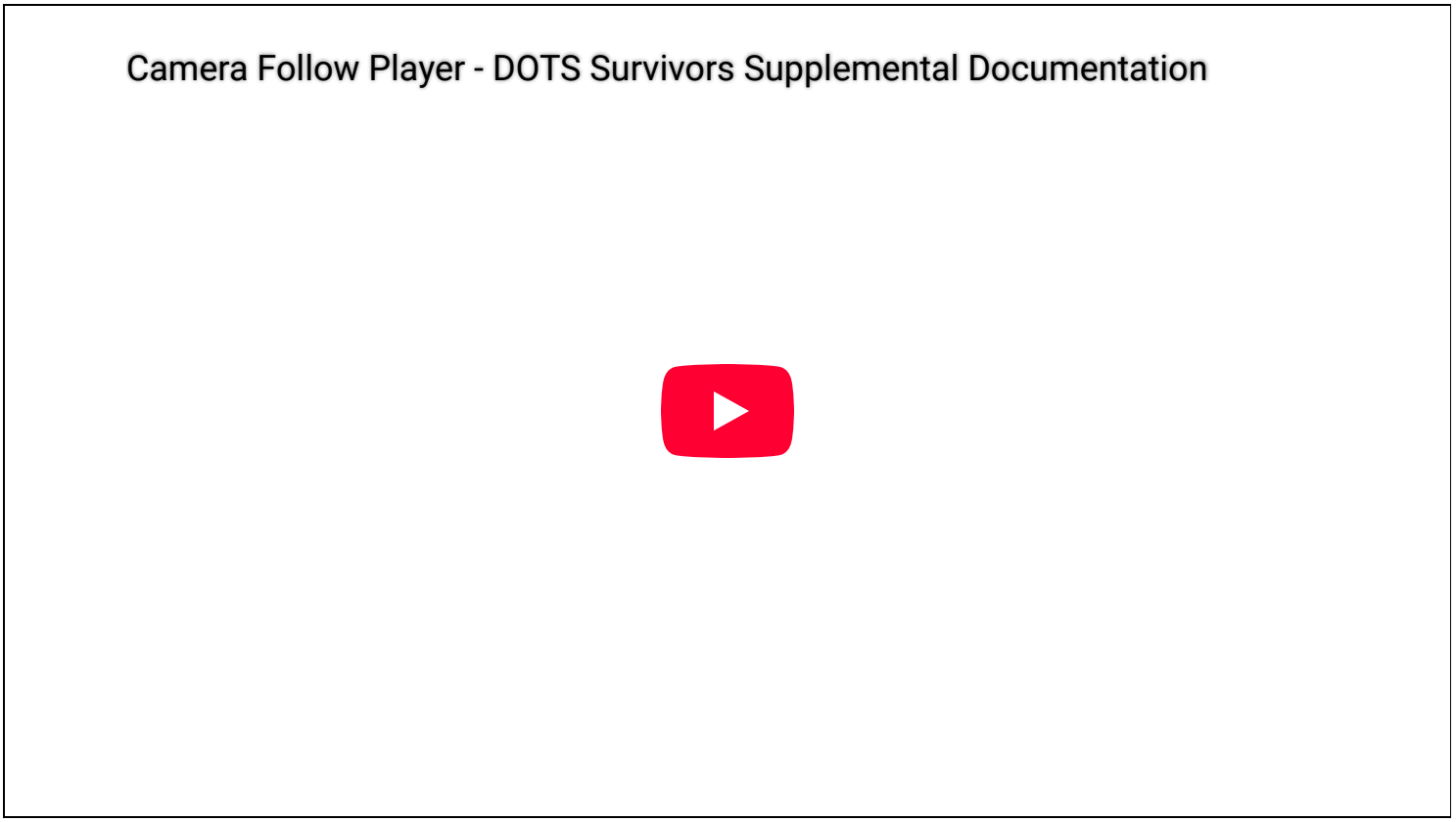
In DOTS Survivors the camera follows the player's movements from a top-down perspective. To set this up, an initialization step is required so that when the player entity spawns, it can locate the camera target. This works as follows:

1. The player spawns with an `InitializeCameraTarget` component
2. The `InitializeCameraTargetSystem` locates the `CameraTargetObject` singleton MonoBehaviour
3. The system assigns the target's transform to the `CameraTarget` data component via `UnityObjectRef<Transform>`
4. The initialization component is then removed

> **NOTE**
>
> This initialization step is kept separate from the player's general initialization logic. Although conceptually related, separating them ensures that if the `CameraTargetObject` isn't loaded when the player spawns, only the camera initialization is retried—keeping the general player initialization from running multiple times.

During gameplay, the `MoveCameraSystem` will get the player's current world position via `LocalToWorld` and use that to update the `CameraTargetObject` through the reference stored in the `CameraTarget` component. Updating the `CameraTargetObject`'s position makes the Cinemachine camera follow that GameObject to effectively follow the player.

Video Overview



Data Persistence

In DOTS Survivors, there is a small amount of progression in the game, where players collect coins in game and those coins persist outside of play sessions. Coins can be used to unlock characters and stages. The coin count and unlocked items should persist through play sessions.

This game uses a very simple, manual binary serialization technique to save data that should remain persistent. This game does not serialize any entity data as it does not have a requirement for it. Save data is stored in a `PlayerSave.bin` file in the `Application.PersistentDataPath`.

Binary serialization is faster than saving data in a human readable format like JSON, however save data is not easily modified outside the game in the save file.

To save game data, a `System.IO.BinaryWriter` is used to write primitive types to the save file.

```
private void SavePersistentGameData()
{
    var fullPath = Application.persistentDataPath + PERSISTENT_FILE_PATH;

    using var fileStream = new FileStream(fullPath, FileMode.Create);
    using var writer = new BinaryWriter(fileStream);

    writer.Write(PERSISTENT_GAME_DATA_VERSION);
    writer.Write(_persistentGameData.CurrentCoinCount);
    writer.Write(_persistentGameData.TotalEnemiesDefeated);
    writer.Write(_persistentGameData.TotalTimeSurvived);
    writer.Write(_persistentGameData.UnlockedCharacterFlags);
    writer.Write(_persistentGameData.UnlockedStageFlags);
    writer.Write(_persistentGameData.MusicVolumeLevel);
    writer.Write(_persistentGameData.SfxVolumeLevel);
}
```

Conversely, to read from the save file, a `System.IO.BinaryReader` is used to read the data and store in the game's memory.

```
private void LoadPersistentGameData()
{
    var fullPath = Application.persistentDataPath + PERSISTENT_FILE_PATH;
    if (!File.Exists(fullPath))
    {        Debug.LogWarning("Save file not found: " + fullPath);
        return;
    }
    using var fileStream = new FileStream(fullPath, FileMode.Open);
    using var reader = new BinaryReader(fileStream);
```

```
    _persistentGameData = new PersistentGameData
    {
        PersistentGameDataVersion = reader.ReadInt32(),
        CurrentCoinCount = reader.ReadInt32(),
        TotalEnemiesDefeated = reader.ReadInt32(),
        TotalTimeSurvived = reader.ReadSingle(),
        UnlockedCharacterFlags = reader.ReadByte(),
        UnlockedStageFlags = reader.ReadByte(),
        MusicVolumeLevel = reader.ReadSingle(),
        SfxVolumeLevel = reader.ReadSingle()
    };
}}
```

During gameplay, save data is stored in memory a `PersistentGameData` struct. Certain values like current coin count or number of enemies defeated are stored plainly, while others like unlocked characters and stages are tracked using bitflags.

```
[Serializable]
public struct PersistentGameData
{
    public int PersistentGameDataVersion;
    public int CurrentCoinCount;
    public int TotalEnemiesDefeated;
    public float TotalTimeSurvived;
    public byte UnlockedCharacterFlags;
    public byte UnlockedStageFlags;
    public float MusicVolumeLevel;
    public float SfxVolumeLevel;
}
```

Each bit of the `UnlockedCharacterFlags` byte is associated with a character. If the bit is 0 the character is locked and if the bit is 1 the character is unlocked. To determine if a character is unlocked, just check the bit of the byte for that character using the enum value assigned to the `CharacterID` enum for that character.

```
public bool IsCharacterUnlocked(CharacterID characterID)
{
    return (_persistentGameData.UnlockedCharacterFlags & (byte)characterID) != 0;
}
```

And to unlock the character, the bit can be set like this:

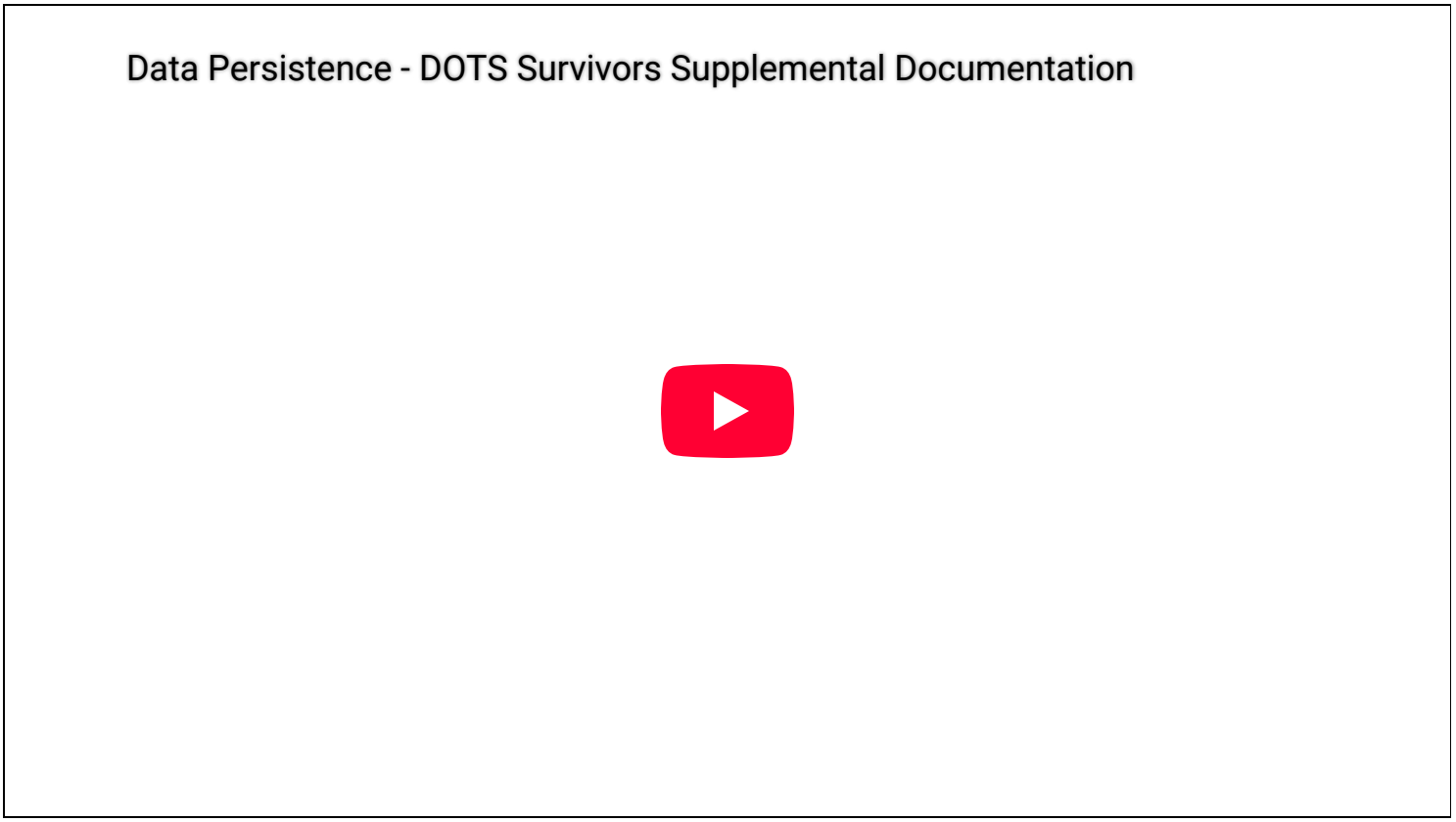
```
public bool TryBuyCharacter(int cost, CharacterID characterID)
{
    if (cost > CurrentCoinCount) return false;
    _persistentGameData.CurrentCoinCount -= cost;
    _persistentGameData.UnlockedCharacterFlags |= (byte)characterID;
    SavePersistentGameData();
    OnUpdateCoinCount?.Invoke(CurrentCoinCount);
    return true;
}
```

Save Data Versioning

You'll notice that the first integer of the `PersistentGameData` struct stores the version number of the save file. This is by design as save data could potentially change over time - data fields get added, rearranged or change type as the game evolves and more persistent data is required to be stored. By first loading in the version of save data, the game can compare that to the current shipped active version of save data - if it is on a previous version, then some upgrade steps can be implemented to translate the previous save data into the current data format.

There is not currently an example of this implementation, but this could be added fairly easily if people ask for it!

Video Overview



Game Management

In the main subscene of each gameplay scene, there is a "game entity" which is a data-only entity that contains many data components pertinent to controlling the flow of the game. Relevant data components are authored in `GameAuthoring` that is attached to the game entity.

The game entity holds the following data:

- Entity prefabs for commonly spawned entities
- Managed game data
  - Lookups for weapon and enemy Entity Prefabs
  - Lookup to obtain the color of an Experience Gem from an input exp. value
  - ScriptableObjects for Bonus Items
- Data components to track game data like coins collected and enemies defeated counter
- Data components to facilitate choosing a player to play as during testing in editor
- A data component to manage Game Time
- Lookup table to determine the amount of experience points required for the player to Level Up
- Lookups for the player's active Weapons and Passive Abilities

Game Initialization

Many of these data components need some runtime initialization steps which are executed in `GameStartSystem.OnUpdate()`. Although this is an update method, it will only execute once as the game entity starts with an `InitializeGameTag` which is required for the `GameStartSystem` to update. Inside this update method, the tag is removed from the entity and thus will never update again.

Game initialization performs the following steps:

1. Increment a game index each time a level is entered in a play session. This is used for Blob Assets to differentiate between blob assets created earlier runs of the game that will no longer be valid or relevant.
2. Iterate through all weapon and enemy entity prefabs and store them in a lookup keyed by their unique type enum.
3. Determine and spawn the appropriate player character. Character determination is done by getting the `CharacterProperties` value from the `SelectedCharacterReference` component. This component is set in the title screen when a player selects an astronaut to play as. However, during testing in the editor the same component on the game entity is used instead. This is so developers can assign a `CharacterProperties` ScriptableObject in the `GameAuthoring` script to test the game with that specific character.
4. Some player initialization steps are taken mostly having to do with getting information from the `CharacterProperties` ScriptableObject and translating it to entity data. This is not all the initialization

the player goes through as they will also have player specific initialization and general character initialization (see Character Initialization).

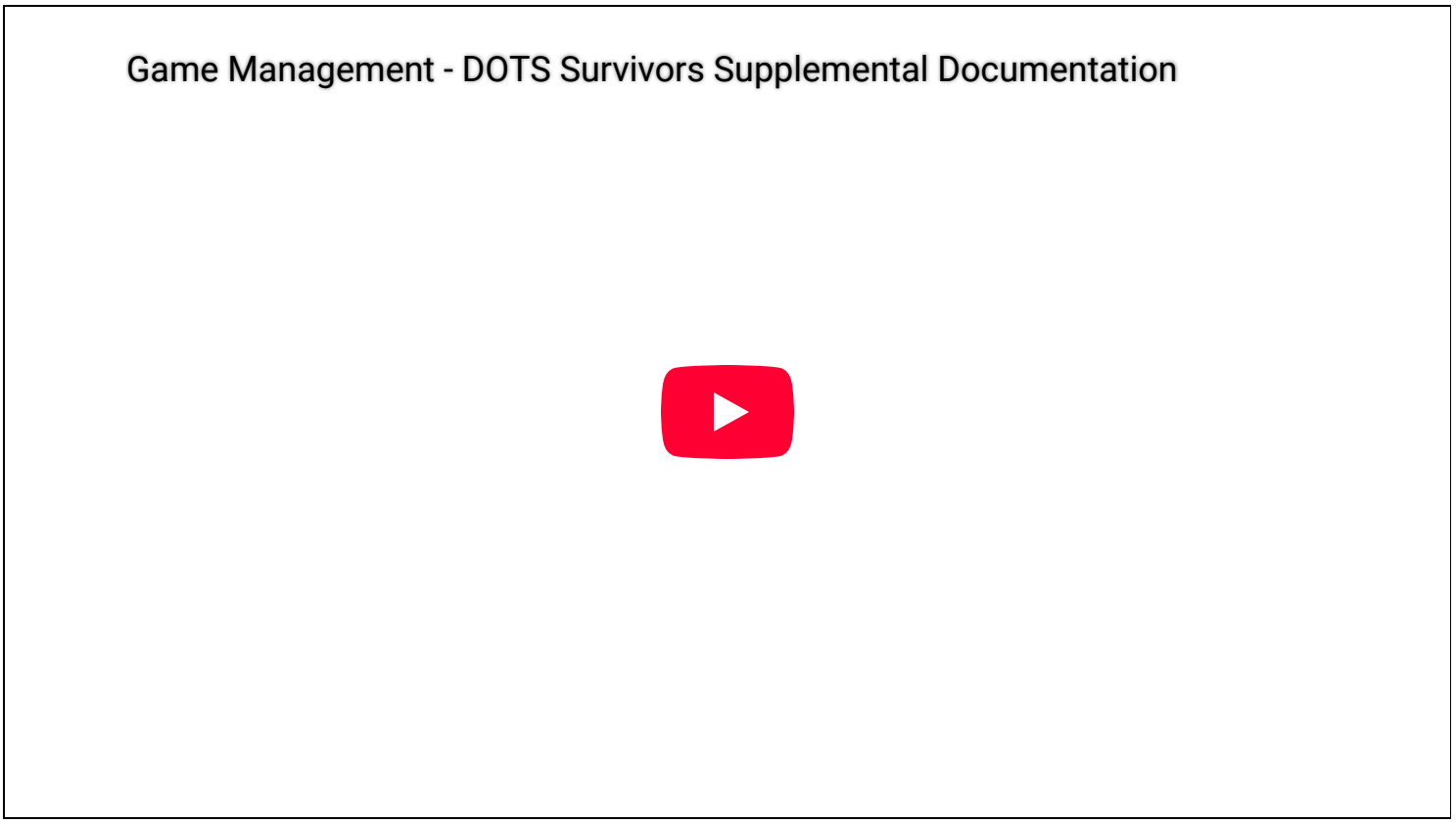
5. Next the `PlayerUpgradeData` struct is populated with Weighted Random values for capabilities the player can upgrade (Player Attacks and Passive Abilities).
6. Finally, the loading screen graphic is hidden and the "beam in" effect is played.

During Gameplay

During gameplay, the game entity is only responsible for running in two systems: one to update the game's Time and one to update the game's UI.

Aside from that, various components of the game entity are accessed in various other systems iterating over other entities. The game entity is easy to access as many components are singletons (only exist on the game entity) such as the `GameEntityTag`.

Video Overview



Items

DOTS Survivors features various spawnable items that the player can collect. These items have different effects on the player. These items take advantage of Entity Interactions to apply their effects to the players.

Collision events handled in `ItemCollectionJob` execute on events between the player and an item which adds an entity interaction to the item's buffer targeting the player. Then, specific systems based off components on the item can run to apply effect(s) to the player.

Attractable Items

Attractable items are items that live in the game world and will become attracted to the player when the player's attraction radius overlaps with the attractable item's collider.

The `DetectItemAttractionSystem` performs a `PhysicsWorldSingleton.OverlapSphere` function to locate items in the player's attraction area. Once inside this area, the `IsAttractedToPlayerData` component gets enabled and initialized with some data to assist with the attraction "animation."

The `AttractableItemSystem` utilizes an "Ease-in-Back" easing function (Easing Functions Cheat Sheet) to move the attractable item first slightly away from the player, then towards the player. This gives a subtle indication of when the item is attracted to the player.

When the item comes in contact with the player, an entity interaction is raised and the effects of the item will be applied to the player.

Here are some examples of attractable items in this game:

Health Pick-Ups (Ice Cream Sandwiches)



If you see an ice cream sandwich laying around, pick it up as it will give you health somehow! On a technical level this happens in the `HealOnInteractionSystem` where a value stored in `HealOnInteraction` is multiplied by negative one and added to the player's `DamageThisFrame` buffer to effectively heal it. See Dealing Damage for more information.

Coins and Money Bags



Coins and money bags are great finds in the world too as they grant the player coins that can be used to unlock new characters and stages. A `GrantMoneyOnInteractionSystem` will grant the player the value of a coin/money bag stored in their `GrantMoneyOnInteraction` component when an entity interaction is raised between the item and the player.

Experience Gems



When an enemy is destroyed, it has a chance of dropping an experience point gem. Data for this behavior is stored in the `DropExperiencePointOnDestroy` component which defines the value of experience point the enemy will drop and the chance it will drop an experience point gem - a value of 0 will never drop an experience point gem and a value of 100 will always drop experience points. The evaluation of this random chance is done in the `DestroyEntitySystem`.

When an experience gem is spawned into the world, it will have its value assigned, but it will not yet have a color. The color is assigned in the `InitializeExperienceGemSystem` where the color value is determined via `ManagedGameData.ExperienceGemColorLookup` and applied to the gem's `URPMaterialPropertyBaseColor` override. This takes place in the `DS_InitializationSystemGroup` immediately after the entity spawns so the entity is never rendered as the white sprite.

Other Item Types

Altars



Altars are items that live in the game world that take one point of damage to destroy. When the altar is destroyed a random item will be spawned. The item selected to be spawned is evaluated using Weighted Random. Authoring the items and weights is handled in a `RandomItemDropProperties` ScriptableObject. An `InitializeSpawnRandomOnDestroySystem` will take the items and weights from the ScriptableObject and store them in a blob asset which is referenced in a `RandomItemToSpawnBlob` component. The `DestroyEntitySystem` will evaluate which item should be spawned and schedule it for instantiation via an entity command buffer.

Loot Crates



Loot crates have a chance to be dropped by boss enemies. When a player comes in contact with a loot crate, the game will pause and a UI window will open prompting the player to open the crate. When the

player opens the crate, a bunch of fun animations and effects play using Unity Timeline. The crate picks a weapon or passive ability for the player and automatically levels it up. Choosing a random upgrade is a uniform (non-weighted) random.

Bonus Items

Bonus items are data-only items that are shown to the player in the level up and loot crate menus when the player has all their capabilities at the max level. These bonus items can either grant additional health or money to the player. They work similar to that of pick-upable health and money items, however as they do not live in the game world, the code that creates them (`CapabilityUpgradeController.CollectBonusItem()`) just automatically raises an entity interaction for them targeting the player.

Video Overview



Level Generation

DOTS Survivors uses random level generation to populate the game world with decorations and items. The seed is currently fixed, so the level is the same layout each time, however this can easily be changed if desired.

Data for Level Generation

`LevelGenerationData` is a regular data component added to the level generation entity. This component contains data for the background prefab, caution prefab (to spawn around the bounds of the safe play area), background scaling factor, size of the level, number of items (decoration & environment entities) to spawn, and the max rarity to use with Weighted Random.

`LevelElementData` is a dynamic buffer containing entity prefabs of items to spawn in the play area. These items use weighted random to spawn more or less frequently and can have a minimum and maximum scale they will spawn at.

Authoring

Data components for level generation are authored through the `LevelGenerationAuthoring` component. Changing the seed value will automatically destroy all level elements and regenerate them with the new seed value. This behavior occurs both when running the game and in the editor.

When the seed value changes, the `OnValidate()` method enables the `RegenerateLevelFlag` on the level generation entity, prompting the `LevelGenerationSystem` to regenerate the level.

Level Generation System

The `LevelGenerationSystem` is a unique system in that it runs both in the editor (outside of play mode) and in the game at runtime. The purpose of it running in the editor is so that changes to the level generation can be viewed in the editor without having to constantly enter/exit play mode to check small changes. This system should also run at runtime so that the level generation can occur for the player to play in.

> **NOTE**
>
> This game will always regenerate the entire level when loading into it, and as mentioned before the seed value is unchanging so the level is the same each time. This is an obvious inefficiency as this entity data should ideally be baked into the subscene. This is something I would consider implementing if enough people show interest.
>
> Another inefficiency is that all level element entities across the map are always loaded into the game. The maps are quite large at 2000x2000 units and contain 17,500 level elements, however only a few dozen at most are relevant to the player. This does have a meaningful effect on performance especially when running the game in editor with burst disabled. Another feature I'd consider implementing with sufficient interest is subscene streaming where only elements nearby the player are loaded in.

Running the generation system in both editor and gameplay worlds is accomplished by adding the following attribute to the system:

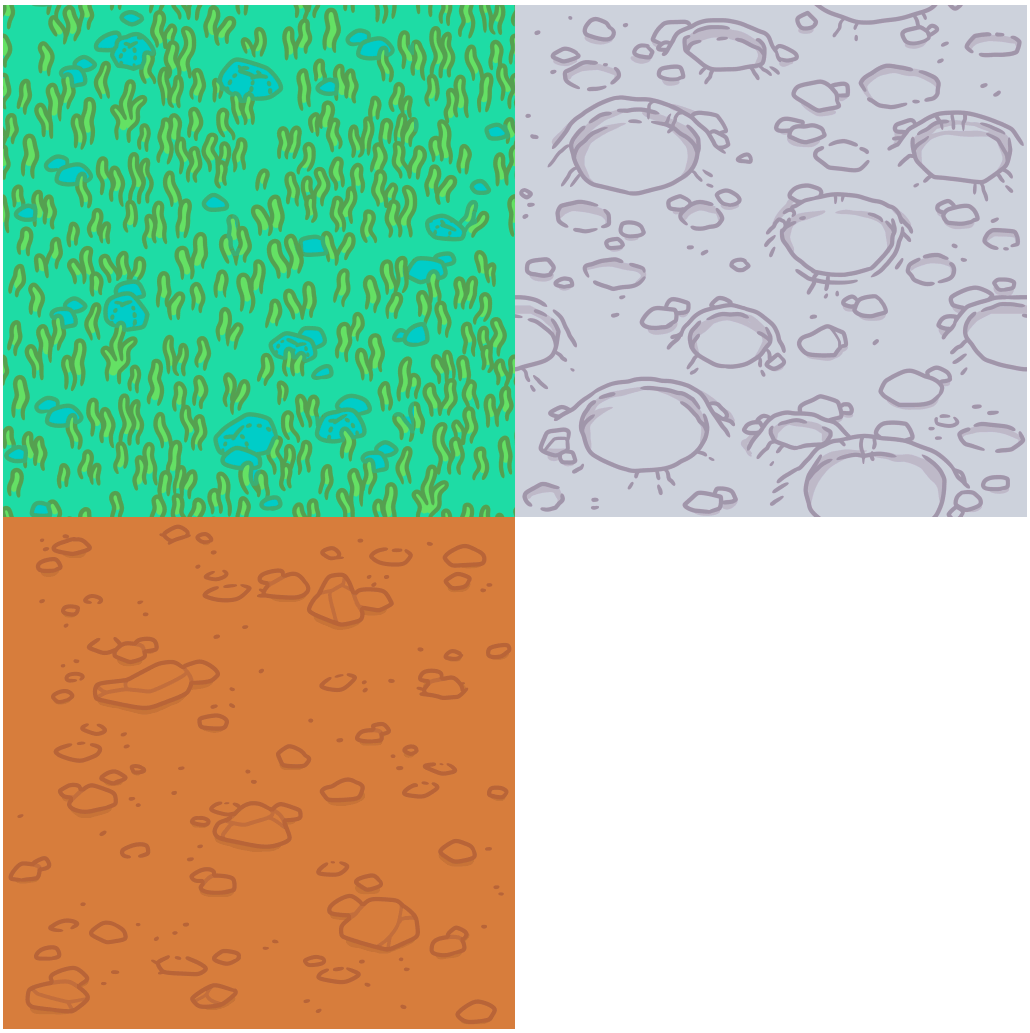
```
[WorldSystemFilter(WorldSystemFilterFlags.Editor | WorldSystemFilterFlags.Default)]
```

This works because when the ECS package is installed in the project, an editor world is created and running when interacting with the editor outside of play mode. This world can have systems that run just like those running in the default game world.

Level generation occurs when the `RegenerateLevelFlag` is enabled on the level generation entity. This flag is enabled when the level is first loaded into and each time the seed value is updated in the authoring script (both in and out of play mode).

Level generation performs the following steps:

- Destroy any existing level elements (background, safe area bounds, environment assets, decoration assets, etc.)
- Spawn a new background entity with appropriate size and scale
- Set material scaling to tile the background to an appropriate size via Material Property Override
- Spawn 4 caution entities to give a visual representation of the safe bounds of the play area
- Set the material scaling on the caution entities for appropriate tiling via material property override
- Set a data component storing the size of the play area safe bounds (used to determine if the player is outside of the safe bounds and will deal damage if so)
- Spawn all the decoration, environment, and item level elements in its `LevelElementData` dynamic buffer using Weighted Random.



Video Overview

> **NOTE**
>
> As of DOTS Survivors Version 1.0.2, the level generation authoring GameObject exists in a separate subscene than the main entity scene. At this point this is primarily for organizational purposes so all the static generated level elements don't clutter up the main entity scene.
>
> In the future, this may be updated such that entities are baked into the subscene so that level generation need not take place on the client device. Another future update may create many subscenes that can be loaded in and out based on the player's position to increase performance.
>
> In this video, the level generation GameObject will exist in the main entity scene, however all functionality remains the same.

Level Generation - DOTS Survivors Supplemental Documentation



Game Time

During normal gameplay of DOTS Survivors, a timer continually counts up to indicate to the player how long they have been playing the current run. This time data is stored in a `GameTime` data component on the main game entity. Each frame, this will increment by delta time and invoke an event to update the HUD UI element.

Gameplay Pausing

During gameplay, the user can press the pause key (ESC on keyboard or START on controller) to pause the game. The UI event is listened to in the `PauseMenuUIController` which determines if the game should pause (the game should only pause if it is not already paused due to level up or a loot crate). This will then call the `PauseManager.PauseGame()` method to pause the game.

The `PauseManager` obtains a reference to the top-level initialization and simulation system groups that Unity ECS creates in the player loop (see System Ordering). These system groups are disabled which disables all ECS systems below them, effectively pause gameplay without altering time scaling. This will also invoke a pause event which can pause non-ECS items such as sound effects (see Audio).

Resuming the game is accomplished by simply reenabling the initialization and simulation system groups and invoking the resume event.

Global Time Update

As noted in the Character Animation section, all character animations are shader-based spritesheet animations. Originally when creating the shader, I used the "Time" node to tick the animation through the animation cycle. This worked during normal gameplay as expected, however when pausing the game as detailed above, this would not stop ticking up the time in the shader graph node, causing the animations to continue although the game was paused.

The solution to this was to create a global material property for time that gets continually incremented in the `GlobalTimeUpdateSystem` which runs in the `InitializationSystemGroup`. This means that when the game pauses and the `InitializationSystemGroup` gets disabled, the `GlobalTimeUpdateSystem` stops ticking up, thus pausing the animations. As soon as the game resumes, the animations will also resume.

Timers

There are two timer methods for entity-based events: countdown timers and expiration timestamp timers.

Timers that tick down start at their target time value and tick down to zero. Once at or below zero, whatever needs to happen takes place, and the timer is reset to the target time if needed. These timers are easy to set up and good for entities that tick down to zero only once. The disadvantage with this

type of approach is that the timer value for every entity with one of these timers needs to be updated every frame.

Timers with an expiration timestamp have a fixed target timestamp that is calculated by adding the current application elapsed time to the time value and stored in a component. Rather than decrementing a timer every frame, this target timestamp is compared to the current application time and will invoke any events when the application time is greater than or equal to the target time. Resetting the timer is done by just again adding the time value to the game's elapsed time and setting it in a data component.

Video Overview



Transformations

DOTS Survivors contains some different custom entity transformations to move, rotate, and scale entities for graphical effects or to facilitate gameplay features. The data-oriented nature of the project means that multiple of these transformations can be used on a single entity for some unique effects.

These transformations modify the `LocalTransform` component directly to effectively "teleport" the entity from one position to another - although this teleportation each frame is very small so the entity looks as if it is moving smoothly.

Linear Movement

The simplest transformation, just move straight forward. This is accomplished by multiplying the entity's forward vector by a move speed and delta time, then adding that to the entity's current position. EZ Clap.

This movement is used by many in-world attack entities to move them through the game world.

Constant Rotation

This transformation will constantly rotate the entity by the `float3` value stored in `ConstantRotationData`. This is useful for graphics entities that should spin around without affecting the main parent entity.

A benefit of Unity ECS's authoring workflow is that this rotation can be authored in degrees, but converted to radians during baking. This is useful because degrees are more easily understood by humans and radians are what most of the `Unity.Mathematics` calculations use by default.

As these rotations are applied in Euler angles, we can also specify the rotation order - the order at which the rotations will be applied to each axis.

This transformation is used by the Plasma Blast attack's graphical entity for a nice little effect.

Counter Rotation

Counter rotation is used by the Drone Swarm attack to ensure the graphics entity remains facing upright as the parent entity rotates around the player character. This operation subtracts the parent's local Euler rotation from a predefined Euler offset and applies the result to the entity.

Linear Scale

The linear scale transformation scales the entity from a starting scale to an ending scale over a set duration of time. This is used by the Black Hole attack to scale it from 0 to larger than the size of the screen.

The `LinearScaleTransformationSystem` has two foreach loops - one for initialization to set the ending timestamp, and one for calculating and applying the scale value to the entity. The scale value is calculated by a simple linear interpolation.

Oscillating Scale

Oscillating scale will constantly scale the entity up and down between two defined scale values in a sine wave pattern. During authoring, the period (frequency), min scale, and max scale are defined. At bake time, the amplitude and y-offset are calculated from the min/max scale. The period, amplitude, and y-offset values are used with a sine wave function to constantly scale the entity up and down.

This transformation is used by the Plasma Blast attack's graphical entity for an appealing visual effect.

Boomerang Movement

Boomerang movement is used by the O.M.G. Wrench attack to move it away from the player in a linear direction, before gradually reversing its direction back towards where it came from until it leaves the screen bounds.

The boomerang motion is created by setting a velocity value each frame determined by a simple quadratic function. In the equation below v is velocity, s is the constant move speed of the boomerang movement, and t is elapsed time.

$$v = t^2 * -s + s$$

Parabolic Movement

Parabolic movement is used by the Satellite Toss attack to move it up and down on the screen in a parabolic arc. This movement is accomplished by determining the starting angle of the movement. Taking the sine and cosine of the starting angle give the x and y "velocity" components that can be used to create the parabolic movement.

The idea is that we will calculate a value for the y velocity using a quadratic function and the x velocity will remain constant. In the equation below v_y is the y component of the velocity, f_g is the gravitational force constant, and t is the elapsed time.

$$v_y = t^2 * f_g$$

The calculated `float2` velocity will be multiplied by delta time and added to the entity's position.

Bounce on Environment

Bounce on environment is used by the Turbo Jetpack attack to bounce off environment entities in the game world. The implementation of this is a bit annoying due to the lack of information Unity allows us to obtain from trigger events.

The functionality begins when an entity with a `BounceOnEnvironmentTag` contacts an environment entity - a trigger event is raised and an Entity Interaction is recorded on the bouncing entity targeting the environment entity.

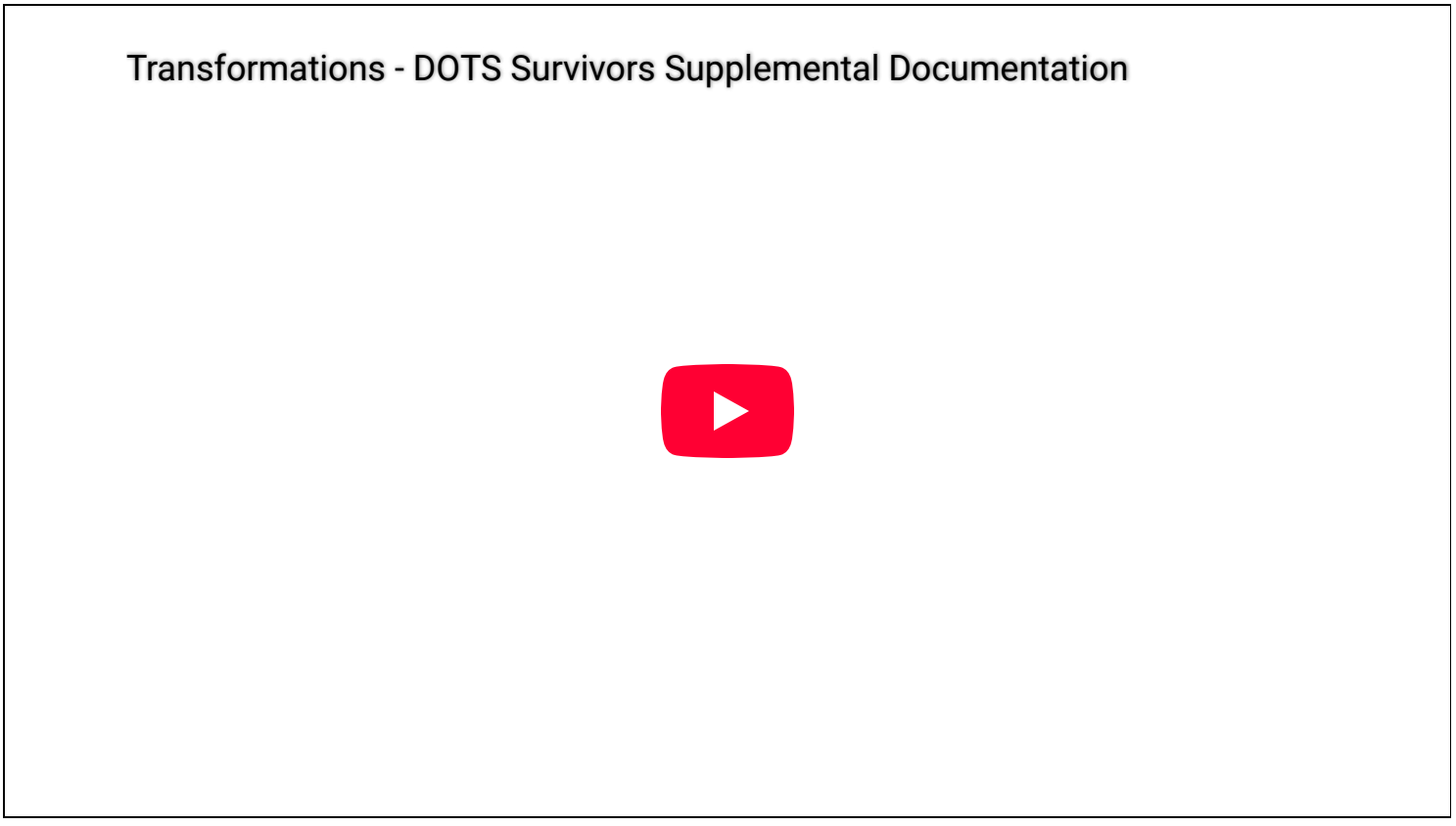
Next the `HandleBounceOnEnvironmentSystem` takes over where the actual bounce takes place. In a trigger event, Unity does not provide the surface normal of the contact between the two entities so an intermediary step is required to determine this information. This is needed so we know at which angle should the bounce occur. To do this, we obtain the collider from the environment entity and execute a collider cast at its world position.

We can then iterate all the entities found in the collider cast until we find the one we are trying to bounce. From this collider cast, we can obtain the surface normal of the surface the bouncing entity is colliding into. With this some vector math can be performed to rotate the orientation about the reflection. We also apply a slight positional offset to avoid cases where it may collide with the entity in multiple successive frames.

Bounce on Screen Edge

Similar to bounce on environment, this transformation is used by the Turbo Jetpack attack to bounce off the edges of the screen. Although this system is much simpler as no collider workarounds are required - it can just simply check to see if the entity is at one of the edges of the screen. If it is, it will reflect its direction of travel.

Video Overview



UI

UI is important to any game as it both provides real-time information about the state of the game and offers a means for the player to interact with and modify that state. Unity's UGUI system works well with standard Unity GameObjects as that is what they were originally designed around. However integrating with ECS requires a few extra steps.

> **NOTE**
> This project uses Unity's UGUI system based off UI Canvases, not the newer UI Toolkit. Although, Unity seems to be pushing things heavily towards UITK and I have seen a few things other people have made with it that make me heavily consider using it for some upcoming projects. If people show interest, this is something I might look into more

Displaying Entity Data in UI

Displaying entity data is important to give the player information about what is happening in the game so they can make informed gameplay decisions.

`UnityObjectRef<T>`

One way ECS data can be "sent" to Unity UI is by use of a `UnityObjectRef` containing a reference to a specific UI element. In this case an entity would store a reference to the UI element and can modify it as needed. For example, the player character has a `PlayerWorldUI` component that stores references to the transform of a world UI canvas and the slider UI element associated with the player's health bar.

```
public struct PlayerWorldUI : IComponentData
{
    public UnityObjectRef<Transform> UICanvas;
    public UnityObjectRef<Slider> HealthBarSlider;
}
```

The `PlayerWorldUISystem` can then just update the position of the canvas (to follow the player) and the health bar to match the player's current and max health values.

```
foreach ( /* Health bar query */ )
{
    playerWorldUIReference.UICanvas.Value.position = transform.Position;
    playerWorldUIReference.HealthBarSlider.Value.value = currentHP / maxHP;
}
```

System.Action

Another way entity data can be displayed in the game is by sending it from an ECS system to a MonoBehaviour in the form of a `System.Action` event. This requires the system to be a `SystemBase` type system as the (managed) `Action` variables will be public members of the system. When needing to send data, the event is invoked passing along the data to display.

Here is how this would look in a system that grants the player money after picking up a coin:

```
public partial class GrantMoneyOnInteractionSystem : SystemBase
{
    public Action<int> OnUpdateCoinCount;
    protected override void OnUpdate()
    {
        foreach ( /* Query for interactions with coins */ )
        {
            coinsCollected.ValueRW.Value += newCoins;
            OnUpdateCoinCount?.Invoke(coinsCollected.ValueRO.Value);
        }
    }
}
```

On the GameObject side, a MonoBehaviour would subscribe to the event and update the UI accordingly. Here is how this would look to update the player's coin count text:

```
public class HUDUIController : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI _coinsText;

    private void OnEnable()
    {
        // Get a reference to the ECS world
        var world = World.DefaultGameObjectInjectionWorld;

        // Get a reference to the system that invokes UI changes
        var grantMoneySystem = world.GetExistingSystemManaged<GrantMoneyOnInteractionSystem>
();

        // Subscribe to the event on the system
        grantMoneySystem.OnUpdateCoinCount += UpdateCoinsText;
    }

    // Method to update the UI element with the new coin count
    private void UpdateCoinsText(int newCoinCount)
```

```
    {
        _coinsText.text = $"Coins: {newCoinCount}";
    }
}
```

MonoBehaviour Singleton

Alternatively, UI controllers can be accessed directly via singleton MonoBehaviours. Note that while this allows using an `ISystem` type, burst compilation remains unavailable.

```
public partial struct ShowDamageNumberOnDamageSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        foreach (/* damage number query */)
        {
            WorldUICanvasController.Instance.DisplayDamageNumber(damageAmount);
        }
    }
}
```

Using UI to Modify Entity Data

UI is not just good for reading data, there are frequently times where games should allow players to modify entity data by clicking a button or entering text into an input field. In DOTS Survivors this is accomplished by UI elements calling methods on a MonoBehaviour singleton, which itself has a reference to the ECS world's `EntityManager`. The `EntityManager` can then be used to directly make changes or enable a component to trigger an "event" system to run (see: Enableable Component Events).

```
public class ButtonController : MonoBehaviour
{
    [SerializeField] private Button _button;

    private void OnEnable()
    {
    _button.onClick.AddListener(() => EntityController.Instance.UpgradePlayer(upgradeData));
    }
}

public class EntityController : MonoBehaviour
{
    public static EntityController Instance;

    private EntityManager _entityManager;
```

```
    private void Start()
    {
        Instance = this;
        _entityManager = World.DefaultGameObjectInjectionWorld.EntityManager;
    }

    public void UpgradePlayer(UpgradeData data)
    {
        // Use _entityManager to create queries, add/remove components, etc.
    }
}
```

Controller Navigation

DOTS Survivors can be played fully using either keyboard/mouse or controller. To facilitate both these control schemes, an animated selection icon will display on both sides of the currently selected element. Interactable UI elements have explicit navigation settings to determine which elements the selection icon will move to next when executing navigational input.

For static menus like the title screen, the explicit navigation settings can be set directly in the editor on the component of the selectable UI element. For dynamic UI like the level up screen, these are set through code.

The animated selection icon consists of two animated UI images. Changes to the selected UI element are detected in the Update method of `SelectionIconUIController` which then starts a coroutine to change the position of the selection icons.

The coroutine is required because certain UI elements that were spawned dynamically register in an incorrect position for a single frame - which means that the selection icon does not show on the sides of selected element correctly.

The solution is to disable the image components then wait one frame before moving to the selected element then setting the width between the two icons to be the width of the UI element plus some padding, then reenable the image components. Without disabling and reenabling the image components, the selection icons lag in their previous position for one frame which doesn't look good when the menu changes. Luckily disabling and reenabling of the UI elements is very hard to perceive.

When no UI element should be selected, i.e. when the user presses a capability to upgrade, the selection icons move to a location far offscreen.

Steam Deck D-Pad Issues

Using the default input actions for UI navigation on Steam Deck causes some strange issues. With this setup, the D-Pad only responds to input a small percentage of times - for instances you can press the D-Pad down 15 times and it will only change the UI selection once. Other times, the D-Pad functions almost as normal where it only misses a few inputs over a short period of time.

This behavior does not occur on PC or Mac with a controller connected. Interestingly enough, if you plug a controller into the Steam Deck, the D-Pad functions just as expected.

The solution to this is to open the input actions asset, edit the UI Action Map, and change the "Navigate" Action to an Action type of "Value" rather than "Pass Through." With this setup the D-Pad functions on Steam Deck just as expected.

Steam Feedback on Controller Support

When I submitted DOTS Survivors to Steam for approval, the game was immediately accepted, however they did provide me with some additional feedback for controller support as recommended steps to implement. I just wanted to mention these and how I chose to solve them for this project.

The first point is that when the user presses the "home" button on the controller to open the Steam Overlay, the game does not pause. Unfortunately with Unity's Input System package, you cannot (at least that I have found) listen to inputs from the controller's home button. Instead I have disabled "Run in Background" in Project Settings > Player > Resolution and Presentation. Now when Steam Overlay opens, the game is considered in the background and will stop playing. This however does not open the gameplay pause menu.

The second point the mentioned was that when the controller is disconnected the game does not pause. To get around this, I have the gameplay `PauseMenuUIController` listen for device change events from the input system. If the device change event is `InputDeviceChange.Removed` and the game is not already paused, I will pause the game. This is a gameplay pause of the game and the game still continues to run in the foreground.

The final point mentioned is that when the player is playing with a controller, the mouse cursor still shows up on screen, forcing the user to move the mouse off screen for a better play experience. For this I have the `SelectionIconUIController` listen for any navigation events from the input system. Navigation events are when the user presses up/down/left/right on any keyboard, d-pad, or analog stick. When one of these events occur, a method is called to hide the cursor.

To make the cursor reappear, the player can move the mouse. This is detected by checking the magnitude of the mouse's delta movement in `SelectionIconUIController.Update()`. If this is greater than some small value, the cursor reappears. This is to provide some form of a dead zone where small readings from the mouse input changing wouldn't make the mouse reappear but any deliberate movement from the player would.

Video Overview

UI - DOTS Survivors Supplemental Documentation



VFX

Visual effects are achieved by two primary methods: material property overrides and companion GameObjects.

Material Property Overrides

Material property overrides allow ECS data components to modify shader properties on a per-entity basis. This can be used to do things like set unique colors or modify animation properties (see Character Animation).

To support this, create a shader using Shader Graph and ensure the "Allow Material Override" box is checked on Graph Settings in the Graph Inspector.

Next, define the property to be overridden and set its "Scope" to "Hybrid Per Instance" in the Graph Inspector's Node Settings. While here, be sure to copy the "Reference" string (typically begins with an underscore).

Finally, create a data component with a single field to override the property. Note that not all data types are supported. Add a `[MaterialProperty("_ReferenceString")]` attribute above the data component, passing the reference string copied from the previous step.

To override the material property for a given entity, just set the value on the data component and Unity will handle the rest!

Unity documentation on Material Property Overrides: Material overrides using C# | Entities Graphics | 1.4.6

Companion GameObjects

For features in Unity that are not ECS-compatible or lack an ECS equivalent, companion GameObjects may be used to implement this functionality. A companion GameObject is a regular Unity GameObject that matches the position and orientation of an ECS entity.

The ECS entity will have a reference to the companion GameObject to set its transform and invoke graphical effects. Basic companion GameObject functionality uses the `CompanionGameObjectData` cleanup component, which stores a `UnityObjectRef<Transform>` referencing the GameObject's transform. The `UpdateCompanionGameObjectSystem` will then push the current position and orientation of the entity to the GameObject each frame. As `CompanionGameObjectData` is a cleanup component, this can be used to automatically destroy the GameObject when the Entity is destroyed.

Unity Documentation on Cleanup Components: Cleanup components introduction | Entities | 1.3.9

The `PlayParticleOnDamageSystem` is an example of a system that sets transform data for the GameObject (containing a standard Unity particle system) and will play the particle system when prompted to do so via enabling the `PlayParticleSystemOnDamageFlag`.

Video Overview



Customizing the Project

The documents in this section provide information on how to expand the project and create your own features and customizations. Since full functionality often requires multiple interdependent components, it is advisable to duplicate an existing element and modify it to suit your needs.

Additionally, familiarize yourself with the core concepts of the feature you wish to customize by reviewing the related documentation pages. This will help you understand the significance of each element.

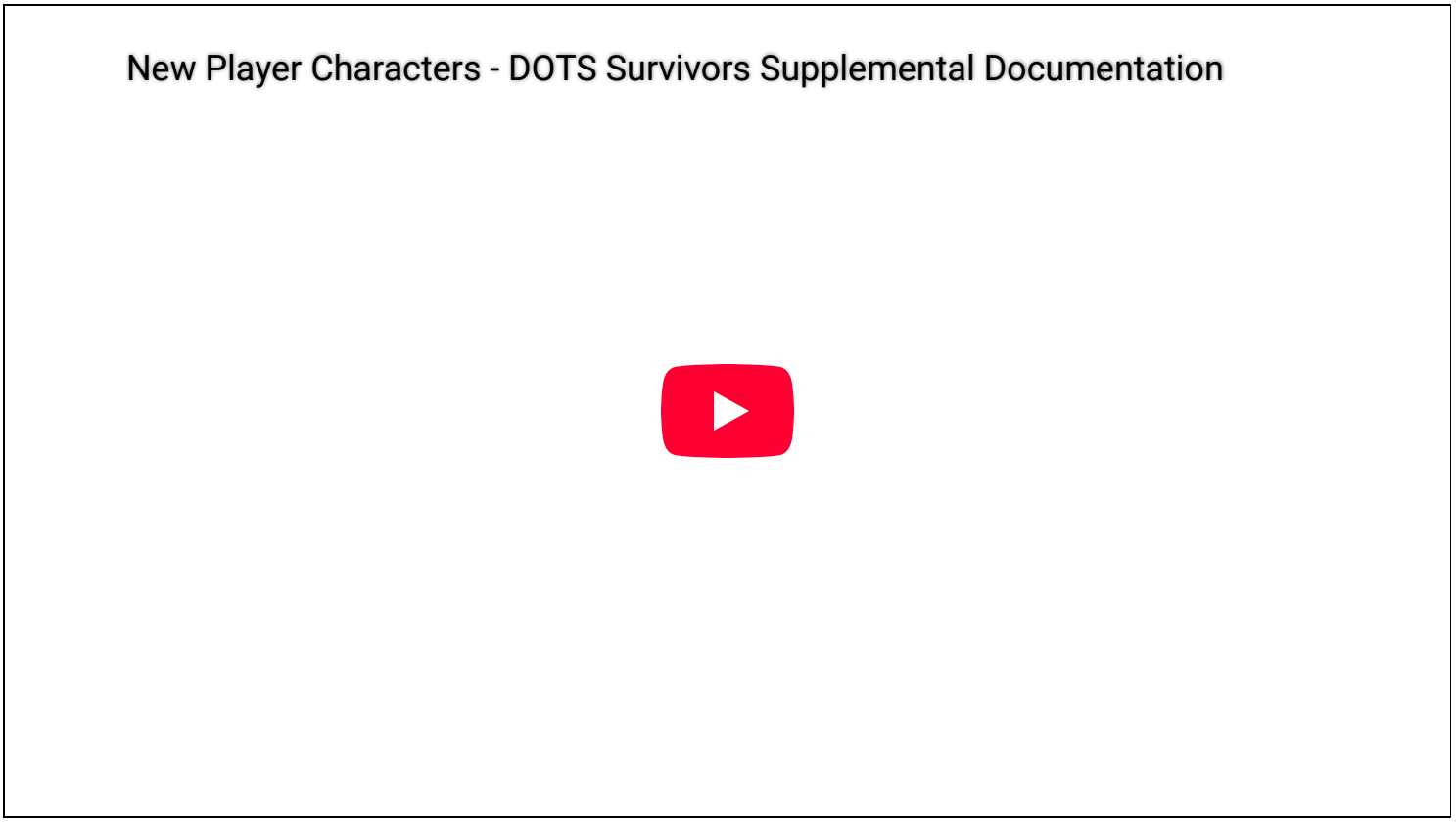
The Asset Store license grants full ability to utilize all code and art assets from DOTS Survivors in any personal and commercial projects. Attribution in your project is not required, but greatly appreciated.

Add New Player Characters

1. Create Player Spritesheet
    - Spritesheet should contain both idle and walking animations in a single PNG.
    - Asset import settings:
        - Texture Type: Sprite (2D and UI)
        - Sprite Mode: Single
        - Wrap Mode: Repeat
    - The custom shader assumes the idle animation is on the top row, walking animation is on the bottom row, and that both animations have the same frame count.
    - Save to `Art\Sprites\Characters\Astronauts`
2. Create Player Icon Sprite
    - Can reuse a single frame of the item animation, but it needs to be in its own separate PNG
    - Save to `Art\Sprites\Characters\Astronauts`
3. Create Space Agency Flag Sprite
    - Recommend copying an existing flag to use as a template so sizing shows up correctly for all uses
    - Save to `Art\Sprites\Flags`
4. Create `SpaceAgencyProperties` ScriptableObject
    - Assign newly create Space Agency Flag
    - Assign name
    - Save to `ScriptableObjects\SpaceAgencyProperties`
5. Create Player Material
    - Use shader: `Shader Graphs/AnimatedShader`
    - Set Frame Count to integer number of animation frames of the animations in the player spritesheet
    - Remainder of the options can be left as default or tweaked as desired
    - Save to `Art\Materials\Astronauts`
6. Add `CharacterID` Enum Value
    - Located at: `Scripts\General\CharacterID.cs`
    - Add a new value for this character with the next bitflag in the sequence - this is important as the game save data relies on having a unique bitflag for each unlockable player character
7. Create `CharacterProperties` ScriptableObject
    - Populate fields with data and assets relevant to the new character
    - Ensure the `CharacterID` is unique to this character or errors will be logged to the console.
    - Stat overrides are the default stat modifiers this character will start with and have through the duration of the game.
    - Save to `ScriptableObjects\CharacterProperties`
8. Assign `CharacterProperties` to `CharacterSelectPanelUIController`
    - In the TitleScene, navigate to the MainCanvas > CharacterSelectPanel

- The current selection UI only supports displaying up to 6 astronauts without going out of bounds so you can either replace one of the "Selectable Characters" with your new one, or modify the UI with a scroll view or otherwise to support displaying more characters.

Video Overview



Add New Enemy Characters

1. Create Alien Spritesheet
    - Spritesheet should contain only a walking animation in a single PNG
    - Asset import settings:
        - Texture Type: Sprite (2D and UI)
        - Sprite Mode: Single
        - Wrap Mode: Repeat
    - Save to `Art\Sprites\Characters\Aliens`
2. Create Alien Material
    - Use shader: `Shader Graphs/EnhancedEnemyMovementShaderGraph`
    - Set Frame Count to integer number of animation frames of the animation spritesheet
    - Set Enhanced Color and Secondary Enhanced Color as desired (this is the outline effect for boss enemies of this type)
    - Remainder of the options can be left as default or tweaked as desired
    - Save to `Art\Materials\Aliens`
3. Add `EnemyType` Enum Value
    - Located at: `Scripts\EnemyLogic\EnemyType.cs`
    - Add a new enum value for each enemy type and color variation to `EnemyType`
    - Ensure the enum value is added at the very bottom of the `EnemyType` enum definition to ensure previously serialized enum values remain unchanged
4. Create New Alien Prefab
    - Easiest to duplicate and modify an existing alien and set the following items:
    - Be sure prefab is located in `Resources\Prefabs\Enemies` so it can be loaded at runtime.
    - Set Move Speed on `CharacterAuthoring`
    - Set attack damage, cooldown, enemy type, and health scaling on `EnemyAuthoring`
        - Enemy type should match the unique ID created in the previous step
        - Health scaling will multiply the base hit points by the player character's level when the enemy spawns
    - Set hit points on `DamageableEntityAuthoring` for the alien's health value
    - Set the amount of experience points to drop and chance of dropping on `DropExperienceOnDestroyAuthoring`
    - Set the audio clip to play when the alien takes damage in `PlayAudioClipOnDamageAuthoring`
    - On the child "Graphics" entity, assign the newly created alien material on the MeshRenderer.
    - On the child "`ShadowGraphics`" entity, scale and position as desired to fit the character. Note that when the character flips facing directions, the shadow will not, so it should be centered along the "flip" axis of the character
    - Physics Configuration
        - Ensure the GameObject layer is set to "Enemy"
        - Set the size and center of the box collider to encompass the character sprite

        - Ensure "Provides Contacts" on the collider is enabled (this will allow the collider to send physics collider events)
        - On the Rigidbody...
            - Set mass to desired value - values less than the player mass of 200 will be pushed around by the player at varying degrees
            - Linear and angular damping should be set to infinity
            - Enable Automatic Center of Mass and Automatic Tensor
            - Disable Use Gravity and Is Kinematic
    - Set the Z value for Base Offset in `ShowDamageNumberOnDamageAuthoring` to represent the relative position the damage numbers will spawn from
    - (Optional) Set knockback taken multiplier on `KnockbackAuthoring`

> **IMPORTANT**
>
> When adding or changing prefabs stored in the resources folder, changes will not always be immediately reflected in the game. This is due to how Unity triggers the ECS baking process where GameObjects get converted to entities. Unity is unable to detect these changes so baking must be manually retriggered.
>
> The most reliable way I have found to do this for prefabs in the resources folder is to first load into one of the main gameplay stages and open the main entity subscene (check the box next to it in the hierarchy). Next clear the entity cache by going to `Edit > Preferences... > Entities > Clear Entity Cache`. Finally, close the subscene.
>
> Having to frequently do this during development is not ideal so I am investigating alternate solutions.

Video Overview

New Enemy Characters - DOTS Survivors Supplemental Documentation



Add New Player Attacks

Before creating a new player attack, review the Player Attacks page for an overview of how attacks work and how existing attacks are implemented.

1. Create necessary assets for attacks
    - In world sprite and material
    - Icon to be displayed on level up screen
    - Audio Clip to play on spawn (looping or non-looping)
    - Additional VFX (Optional)
2. Create an authoring script for the in-world attack entity
    - At minimum add attributes for `[RequireComponent(typeof(EntityInteractionAuthoring))]` and `[RequireComponent(typeof(DestructibleEntityAuthoring))]`
    - Add data components required for desired movement and attack behavior. Often it is not needed to set data in the authoring components as much of that data is determined at runtime in the attack system that spawns it
3. Create the in-world attack prefab
    - Top level GameObject will have the in-world attack authoring script (along with the other required ones) and a Physics Shape component
    - Child "Graphics" GameObject has Quad mesh with material to display the attack sprite. Can also have motion effects such as `ConstantRotationAuthoring` and `ScaleOscillationAuthoring`
    - Store at `Prefabs\AttackEntities`
4. Create a `WeaponType` enum value unique to this attack
    - Located at `Scripts\PlayerAttacks\WeaponType.cs`
5. Create a `WeaponUpgradeProperties` ScriptableObject instance at `Resources\ScriptableObjects\UpgradeProperties\WeaponUpgradeProperties`
    - Assign data and assets relevant to to the weapon as desired
    - Ensure the unique Weapon Type is chosen
    - Assign the attack prefab you just created
    - Fill the Level Properties Array with data for the upgrade path
        - Each element represents a single upgrade level
        - Data fields do not stack - meaning all fields will be applied to the attack as is when the weapon is upgraded to that level
        - Not all fields will be relevant for all attacks
6. Create new weapon authoring script
    - Add `[RequireComponent(typeof(WeaponAuthoring))]` attribute
    - Add any necessary data components - many times a simple tag is sufficient, but sometimes attacks need additional data which can be authored and added to the weapon entity
7. Create an attack system
    - Will run on the weapon entity for the new attack

- Add the `[UpdateInGroup(typeof(DS_AttackSystemGroup))]` attribute to ensure it updates in the appropriate place in the frame.
- At a minimum the system will need:
  - Read/Write access to the `WeaponState` component for current attack timers and attack count
  - Read access to `WeaponLevelData` for fixed attack properties
  - Read access to `AttackPrefab` to spawn prefab attack instances
  - Read access to `Parent` to access the player entity to obtain `CharacterStatModificationState` to apply stat modifications to attacks
- Ensure the system only runs when `WeaponActiveFlag` is enabled.
- Functionality inside the system varies depending on desired behavior. Reference existing attack systems for more information on how to implement common behavior
- Ensure the system is properly updating `WeaponState` and `WeaponActiveFlag` at the appropriate times so attacks function in a consistent and predictable manner

8. Create a Weapon prefab
   - Add your custom authoring component for the weapon entity (not in-world attack)
   - Assign the Weapon Upgrade Properties ScriptableObject on `WeaponAuthoring`
   - Save to `Resources\Prefabs\Weapons`

> ⊗ **IMPORTANT**
>
> When adding or changing prefabs stored in the resources folder, changes will not always be immediately reflected in the game. This is due to how Unity triggers the ECS baking process where GameObjects get converted to entities. Unity is unable to detect these changes so baking must be manually retriggered.
>
> The most reliable way I have found to do this for prefabs in the resources folder is to first load into one of the main gameplay stages and open the main entity subscene (check the box next to it in the hierarchy). Next clear the entity cache by going to `Edit > Preferences... > Entities > Clear Entity Cache`. Finally, close the subscene.
>
> Having to frequently do this during development is not ideal so I am investigating alternate solutions.

Video Overview

New Player Attacks - DOTS Survivors Supplemental Documentation



Add New Stages

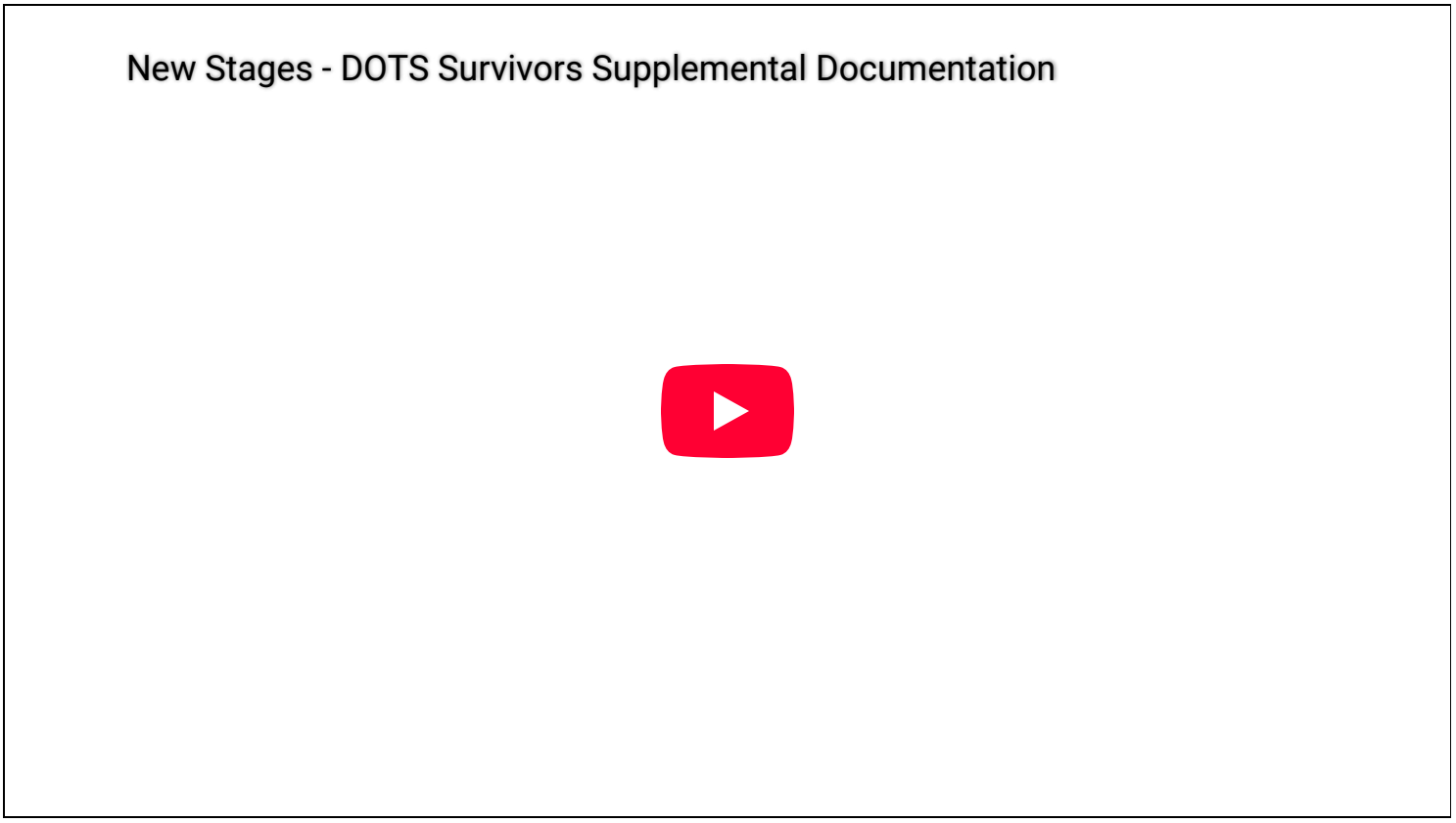
1. Create assets for your level
    - Tileable background sprite, material, and prefab
        - Material should use `Shader Graphs/TileableBackgroundShaderGraph` shader
    - Sprites, materials, and prefabs for decoration and environment elements
        - Graphics should live on a child entity with appropriate orientation (likely just a 90 degree x-axis rotation)
        - Add Physics Shape, Physics Body, and Environment Authoring to any entity that should collide with the player and their attacks
        - Add decoration authoring to any entity that will just exist in the background
    - Stage preview sprite 3:2 aspect ratio
2. Create a unique `StageID` enum value
    - Located at: `Scripts\General\StageID.cs`
    - Value should be the next bitflag in sequence (this is used for Data Persistence to save which stages the user has unlocked)
3. Create `StageProperties` ScriptableObject
    - Save in `ScriptableObjects\StageProperties`
    - Enter information and assets relevant to the current stage
    - Ensure the `StageID` unique to the stage is chosen
    - Populate the Enemy Spawn Waves array with Enemy Spawn Wave Properties ScriptableObjects. Waves will be executed in order at the wave interval defined on this asset
    - Be sure to set the Final Boss type so a boss enemy can spawn at the end of the game[1]
4. Ensure your scene has the following GameObjects:
    - Main Camera with Cinemachine Brain
    - Global Volume Configured as desired
    - CinemachineCamera prefab
    - CameraTargetObject prefab
    - Flagpole prefab
    - BeamInEffect prefab
    - GameUI prefab
    - GameControllers prefab
    - PersistentDataManager prefab
    - StatModifierController prefab
5. Create a new entity subscene with the following:
    - GameEntity prefab
    - EnemySpawner
        - Add Enemy Spawn Authoring and assign the newly created `StageProperties` ScriptableObject
        - Define minimum and maximum spawn distance from camera (default 3 to 10)

6. Create a second subscene for the environment with:
    - Level Generator with Level Generator Authoring
        - Assign data and assets as desired
        - Level Elements define the environment, decoration, and other entity prefabs that will be spawned around the level
        - See Level Generation for more info

---

[1] If you would like to override the end game behavior to spawn something other than a boss enemy - i.e. portal to a new world, loot crate, etc. - you can define some custom behavior in the `EnemyWaveSystem`. Immediately after incrementing the current wave index, the system will check to see if there are no more spawn waves to spawn - if this is the case, the current system destroys all existing enemy entities then spawns the final boss to destroy the player and end the game. This can easily be changed to do something else if you wish.

Video Overview



Add New Spawn Waves

1. Create an Enemy Spawn Wave Properties ScriptableObject
    - Save to `ScriptableObjects\EnemySpawnWaves`
2. Once configured, add the Spawn Wave Properties asset to the Enemy Spawn Waves array within the Stage Properties ScriptableObject for the desired stage. Spawn waves will execute in the listed order and last for the duration specified by the Wave Interval in the Stage Properties.

Spawn Wave Properties Fields

- Enemy Types are the regular enemies that will continually spawn throughout the current wave
- Boss Type is the singular boss entity that will spawn with this wave
- Boss Base Hit Points will be multiplied by the player's current level at the time of spawning to calculate the boss's effective hit points
- Boss Experience Points is the amount of experience points assigned to the gem that will drop when the boss is defeated
- Boss Chance to Drop Crate will be evaluated when the boss is defeated and will determine if the boss should drop a crate. Value of 100 will always drop a crate and value of 0 will never drop a crate
- Min Enemy Count is the minimum number of enemies that will always be present during this wave
- Spawn Interval is the interval at which the enemy count will be evaluated to determine if the minimum enemy count is met. If the minimum enemy count is not met, it will spawn one enemy of each base enemy type until the minimum count is met. If the minimum enemy count is met, only one of each base enemy type will be spawned for that interval
- Spawn Events collection lists all spawn events that can occur during this spawn wave
- Delay time is the amount of time after the spawn wave starts until the first spawn event can occur. Subsequent spawn events will be evaluated for occurrence at this same interval
- Chance of occurrence is the probability that a spawn wave will occur. Value of 100 means all spawn events will always occur, value of 50 means each spawn event has a 50/50 chance of occurrence, and value of 0 means none of the spawn waves will occur

Add New Spawn Events

1. Create an Enemy Spawn Event Properties ScriptableObject
    - Save to `ScriptableObjects\EnemySpawnEvents`
2. Once completed with the data configuration, you can add the Spawn Event Properties asset to the Spawn Events array of an existing `EnemySpawnWaveProperties` ScriptableObject for the spawn wave you wish to have this spawn event occur on.

Spawn Event Properties Fields

- Choose Spawn Formation - this will also determine their behavior
    - **None** is invalid and no enemies will spawn

    - **Linear Move Group** spawns enemies in a cluster at a random location offscreen. Their initial direction will move towards the player at the movement speed defined, however they will keep moving in that same direction for their entire lifecycle
    - **Ellipse Around View** will spawn all enemies in an elliptical formation just outside view of the camera. Enemies will move towards the center of the ellipse at the movement speed defined
    - **Sine Move Vertical Line** will spawn the enemies in a vertical line off screen either to the left or right (chosen randomly). Enemies will move horizontally across the screen, moving up and down in a sine wave pattern
    - **Sine Move Horizontal Line** is the same as the vertical line, except they will spawn horizontally above or below the screen and move down or up respectively while moving side to side in a sine wave pattern
- Enemy count is the number of enemies that will spawn in this spawn event
- Assign data specific to the spawn formation below (will automatically update to display only the relevant properties)

Video Overview

New Enemy Spawn Waves - DOTS Survivors Supplemental Documentation



Postmortem Thoughts

I wanted to set aside a section of the documentation to break down my reflections on development after the game and asset store product has been released for nearly two months. Towards the bottom of this page, there is also a section on some of the major "mistakes" I feel like I made during development and how I might go about doing them differently next time.

Overall, I am very happy with the development of DOTS Survivors both as a game and as an Asset Store product. Development was fun and I wanted to work on it as much as I could over the 8ish months I spent chipping away at it in my free time.

I've been building up my DOTS/ECS development skills over the past few years making tutorials, simple games, and contract projects and I found that development was quite straightforward. For the most part I knew what I needed to do and how to go about doing it with best practices.

As I continued to develop out this project, there were cases found better ways to do things either naturally or through conversations with peers. For some of these things, they already worked just fine and if I were making this project just to ship the game, I may not have gone back and changed how they were implemented. However, with a primary intention of this project being for it to be used as a learning resource, I found it important to go back and rework those things to simplify the code and make it more reliable.

My general approach to development was to essentially create the whole project first and bring it very close to completion before writing any of the documentation. I did this because I knew there were things with the game architecture that would change over time as I find what works best for this project that I wanted to finalize before writing the docs.

This ended up working out really well for me because once I felt like the project was mostly complete, I went through all the code in my project line by line and wrote the API documentation for all the types in the game. This forced me to not only revisit my code from another perspective, but it also forced me to "justify" the inclusions and reasoning of everything in the codebase.

Through this process, I uncovered a bunch of small little issues I may have otherwise missed. This is also where I found ways to simplify much of the code - while this didn't always have a direct impact on performance or reliability, I felt that it made the code easier to follow which is important for a learning resource like this.

This process of creating the API documentation was quite time consuming and took up much of my winter break. But I do think it greatly benefited the project for 1 - having good, detailed reference material for people who purchase the asset and 2 - for uncovering potential issues with the game before shipping. This is one of the main reasons I think I was able to ship this game with very little bugs or issues, despite not doing any form of playtesting (outside myself of course).

Another reason for the lack of bugs/issues in the shipped product is due to the data-oriented architecture. Separating data and systems and having each system run independently makes things much easier to test as there are very few cross-project dependencies to mess things up. Once I implement a new feature and test it thoroughly, I can be pretty certain that it is going to stay solid for the remainder of development.

From time to time, there are instances where new systems step on older systems or issues take longer to reveal themselves, but often once these are discovered the fix is quite simple. Once a problem is discovered you can typically narrow it down to one system or a small number of related systems, and the fix is local to that space you are looking. Again, just fix the issue and you can be pretty confident it won't mess with anything else.

Now, I don't want to toot my own horn too much as I do recognize that another possible reason for the lack of bugs in this game is due to the fact that the game has maybe only been played by a few hundred people and only a fraction of that have gone in-depth with the game. So it is certainly possible that there are some serious bugs in the game in weird scenarios that have yet to be discovered.

And it is also ultimately a quite simple game where the systems can easily be separated out, which lends itself well to isolated testing and catching things early. Not to mention I ripped off the majority of the game design from another game... 😁

In hindsight, I probably should have even done just a small amount of playtesting as there were a few things caught pretty much day 1 of the release of the game that likely would have been brought up from giving it to some friends. There were also a few other small bugs that I knew about, but wasn't sure if anyone would run into them, and sure enough they did, so fixes for those came in a quick update.

Gameplay balance is something that I am not particularly good at, and it is a tedious and time consuming task. I did some small bits of gameplay balance, but I really didn't fully test the game as much as I should have. When I shipped the game, it was almost impossible to complete both Mars and Venus stages. So I ended up just spending a few hours one evening to really dial things in and I'm happy with where it is at now - completing the game is usually possible with a smart build with complimentary weapons and items, but isn't easy to do so it feels good to win.

I'd be remiss not to mention the fantastic art and music created by the great duo at Penzilla Design, Cille and Emil. My friend Walter had picked up some of Cille's asset packs on itch.io and saw that she was available for freelance work. And luckily enough, she was willing to take on this project and make all the art for the game. I'm super happy with the art style and I'm glad that everything fits nicely together to feel like a polished and coherent game.

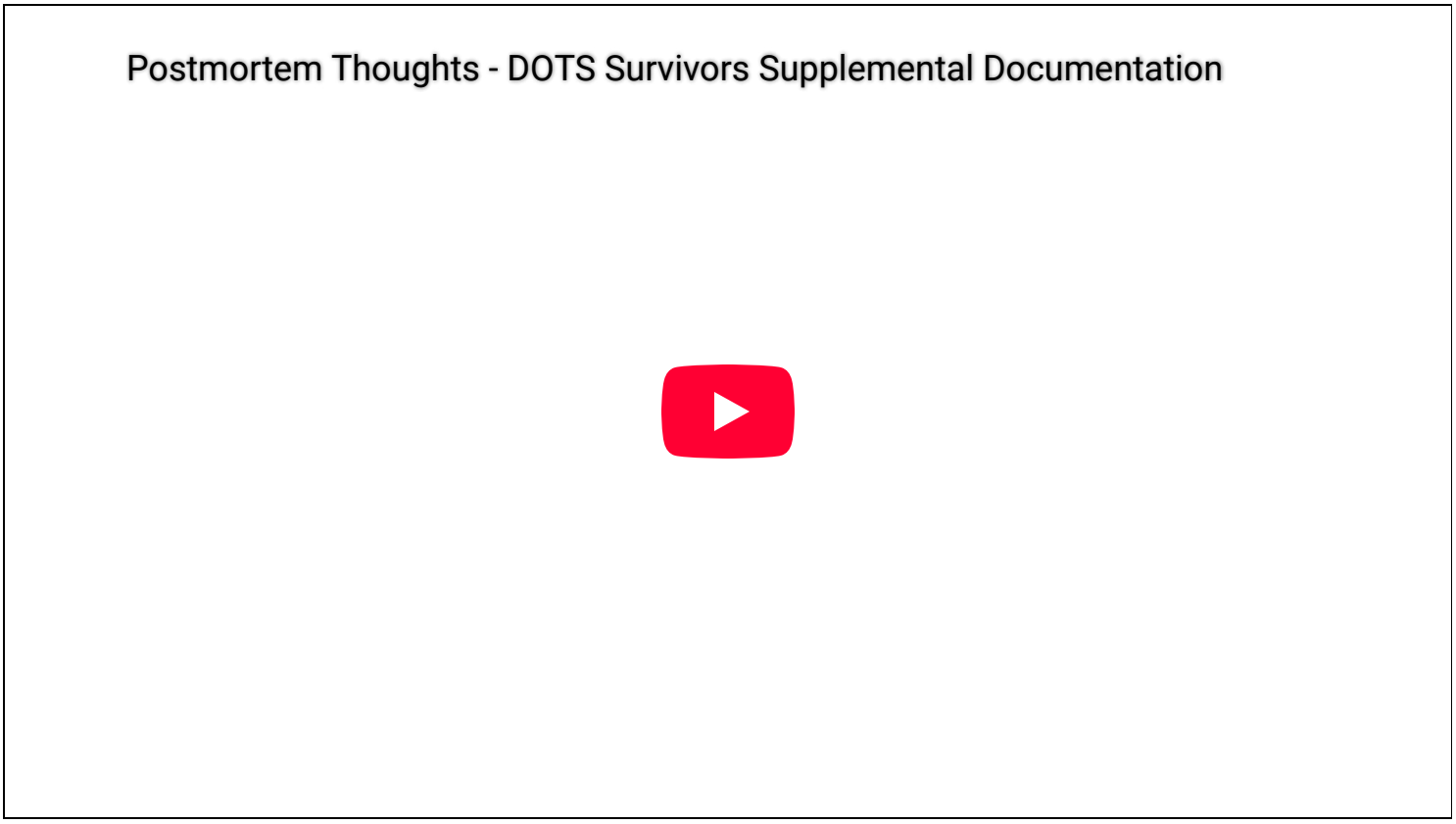
It also so happens that her partner, Emil, is a games composer and was able to create all the music and sound effects for the game. I'm not a musician by any means and with my limited description of the vibe I was going for, Emil was able to far exceed my expectations on this front.

Working with Cille and Emil was such a treat and it makes me want to make another 2D game so I can work again with them some more!

Since the release of the game, I've been very happy to see the response to the game and Asset Store product. People seem to be having fun with the game and learning a lot about ECS from all the learning resources I put together.

As things like this go, I still have a whole list of things I'd like to add to the game and some other learning resources I can provide with this. So I would like to release at leas some of those things over time before I start working on my next big thing.

Video Overview



Mistakes and Things I'd do Different

2D on X-Z Axes

When I started prototyping DOTS Survivors, I made the game with a top-down perspective; moving entities along the X and Z axes and having the camera offset on the Y-axis. While this works fine on a technical level, there are a few things that are a bit annoying with this approach.

> **NOTE**
>
> Community member Bellok pointed out that having the game in the X-Z plane makes it easier to translate the game to 3D, so keep this in mind if you do intend on moving to a 3D perspective.

Unfortunately, by the time I realized this, it was way too late - to switch to having entities move on the X-Y axes would require lots of code changes and reconfiguration of prefabs. I heavily considered making the change as this project is meant to serve as an example of how to accomplish things in a full ECS project, however it doesn't actually change anything with the functionality of the project, just ease of development. And as I was already nearing the end of development on the project, I wouldn't get any of the benefits of the X-Y setup.

The first annoyance with the X-Z setup is that it forced me to use a Separate Graphics Entity. This is because I needed to rotate the quad mesh by 90 degrees along the X-axis to face the camera. While this could be done on the "main" parent entity, I would need to factor in this rotation when spawning or performing other rotations on that main entity. A bit of a drag to have to work around this in all those places in code. Having a separate graphics entity means that I can just have that graphics entity on its fixed rotation on the X-axis and not have to worry about it on the parent entity.

In the separate graphics entity section of the docs linked above, I did mention that even if I used the X-Y approach I would likely still have a separate graphics entity, however this wouldn't be a requirement as it essentially is now.

The second pain point is the conversion between 2D and 3D types. This isn't something particularly difficult to do with the X-Z setup, it is just that with the X-Y setup this is much easier/cleaner to do. For example, let's say I have a `float2` variable storing the current move direction for the player, and I need to convert that to a `float3` to add that to an entity's `LocalTransform` to move it. In the X-Z setup I would have to do this:

```
var moveDirection   = new float2(1f, 0f);
var moveDirection3d = new float3(moveDirection.x, 0f, moveDirection.y);
```

This is a bit verbose as for the X and Z values on the `float3` I need to write out the full variable name and access the X-Y components individually. This is a bit error prone as it is easy to make a typo when writing out the variable names and being sure to include `0f` for the Y-component.

However! Look how easy this conversion is do do with X-Y:

```
var moveDirection   = new float2(1f, 0f);
var moveDirection3d = new float3(moveDirection, 0f);
```

As you can see, we just simply pass in the `float2` type as the first argument of the `float3` constructor to automatically assign the X-Y values of the new `float3`. Then if we need any Z-offset, we can just add that in as the second argument.

Custom Physics Authoring

Another "mistake" I made during development was using Unity's custom DOTS physics authoring components. These are the ones you need to manually import into your project via the "Samples" tab in the `com.unity.physics` package in the Package Manager. These are the ones called "Physics Shape" and "Physics Body" rather than "XXX Collider" or "Rigidbody."

Both the DOTS custom authoring components and regular Unity physics components will add the necessary DOTS data components to the entity during the baking process, however there are some things about the custom authoring components that I prefer to use in ECS projects.

Now although I do prefer using the custom authoring components, I still say that it was a mistake to use these during development. You'll notice that the project files are not using the custom authoring components and rather are using the regular Unity physics components for authoring. This was a change made very late in development as through the course of development I was using the custom authoring components.

The reason for the switch is primarily due requirements imposed by the Unity Asset Store. When I went to submit my project, it got flagged due to the inclusion of namespaces including the word `Unity` which is not allowed. As you might guess, these namespaces were not from my project, but rather from the custom authoring components. This happens because the custom authoring components are "samples" which are added to the project hierarchy and not simply the packages folder.

So my options were to either remove the custom authoring components and ship the project in a broken state with instructions on how to install the custom authoring components, or bite the bullet and convert the whole project to use the regular Unity physics components for authoring. I decided upon the latter as I didn't want to ship the project in a broken state, also Unity seems to be pushing towards using the regular physics components for authoring so there is one unified component type for both GameObjects and ECS projects.

In hindsight, I *may* have been able to work with the people on the Asset Store approval team to explain my inclusion of these components, but I figured that if Unity is pushing people towards the regular physics components, I probably should make the change.

Benefits of Custom Physics Authoring Components

Here is a brief mention on some things that I really like about the custom physics components.

For one, we can create Physics Material Templates that allow us to define some properties and store them in an asset. This asset can then be added to the Physics Shape component to set these properties. This is quite helpful for having various prefabs and entities that should share some common physics settings. Regular Unity authoring does have a concept of a Physics Material asset, however this does not include the things most important to me like collision response and collision filter.

These Physics Material Templates can also be stored on authoring components, providing easy access to the collision filter. There are a few places where I store a collision filter in a data component to execute certain spatial queries. When converting to the regular Unity physics components for authoring, I no longer could use these templates and had to create the collision filter with some bit-level operations. Not particularly difficult, but there just isn't an easy way to do this with regular Unity physics components.

For collision layers on the regular Unity physics components, the collision matrix is defined in the project settings and the layer an entity belongs to is set on the top of the authoring GameObject. With the DOTS physics authoring, these are all set in either the Physics Material Template, or directly on the Physics Shape component - all in one spot.

Also, the custom physics authoring components have options that are named closer to what these are in ECS data components. For example, collision response is quite clear on the custom physics authoring and there is a drop-down to select the exact option you want. With regular Unity components, this is authored through a few checkboxes that are not immediately apparent as to what they do - i.e. if you want to raise collision events you need to be sure "Is Trigger" is not checked and "Provides Contacts" is checked. Makes sense when you know it, but took me a second to figure it out. Then again this all needs to be set on each entity and can't just be stored in a material template.

Ultimately, these are rather small things that don't change much with the actual functionality, but there is a small bit of "ahhh I kinda miss that feature" when using the Unity physics components.

Enemy Prefab Management

The approach I took to enemy prefab management works okay, but there are a few things I may have done differently.

For starters, I have every enemy type and color variation as its own separate prefab. At the current scale of the game, it isn't terrible to deal with but if this game were to grow to have hundreds of enemies each

with 2-4 color variations, it could easily end up with over 1000 individual prefabs.

It might be wise to have these prefabs all be prefab variants of a base "enemy" prefab so if there needed to be high level changes across all enemies those could easily be implemented. Luckily, this isn't something I really needed to do much once all my enemies were created, but again for a growing project this would have been a better base to build off of.

Another thought I had was to do an approach similar to that of how I have the astronauts setup. With those, I am storing all relevant data in ScriptableObjects and loading them onto a base "player" prefab upon instantiation.

For enemies this might require a bit more thought as it would be unwise to be constantly reading from ScriptableObjects and loading data. So maybe when the game loads I could run through all the enemy ScriptableObjects and create prefabs for each of them that could then be spawned as needed.

Another not-so-great-but-still-works-fine thing about enemy prefab management is that I need to have a new enum value for each enemy type and color variation. This enum is nice to have as it allows me to easily specify the types of enemies added to an Enemy Spawn Wave. This does though mean that each time I create a new enemy, I have to manually go in and add a new value to the `EnemyType` enum.

To get around this, I might instead add a bit of code generation to scan the above proposed ScriptableObjects and generate a unique enum value. Though I would need to be careful to add new values after old values and not just put everything in alphabetical order as that would break all the places I am manually setting the enum (this is because Unity serializes the underlying integer value of the enum not whatever text value is set for the enum).

Video Overview

Mistakes During Development - DOTS Survivors Supplemental Documentation



Attributions

Johnny Thompson - Turbo Makes Games - DOTS Survivors Lead Developer

Wrote, recorded, and edited all written and video documentation

Turbo Makes Games - YouTube

Cille Abildhauge - Penzilla Design - Art Assets

Emil Kirchhoff - Penzilla Design - Music + SFX

Penzilla Design | Graphics and Illustrations

Font: Oxanium

Oxanium - Google Fonts